Spread Prediction for Portfolio Optimisation

Evaluating whether spread prediction between cointegrated asset pairs produces more reliable return estimates than historical means for Markowitz optimisation

Candidate Number: XVMK3

BSc Computer Science

Supervisor: Dr. Tomaso Aste

Submission date: 24th April 2026

This report is submitted as part requirement for the BSc Degree in Computer Science at UCL. It is substantially the result of my own work except where explicitly indicated in the text.

The report may be freely copied and distributed provided the source is explicitly acknowledged.

Abstract

This project investigates whether Ornstein–Uhlenbeck (OU) implied spread returns derived from cointegrated asset pairs provide more reliable inputs to Markowitz mean-variance optimisation than traditional historical mean returns. Constructing stable return estimates for portfolio optimisation is notoriously difficult, as historical means are sensitive to the sample period and amplify estimation error. This difficulty motivates the use of cointegration-implied, mean-reverting spread dynamics as a more stable alternative.

A 26-stock universe of US equities is screened using the Engle–Granger procedure over an in-sample period (January 2018 – December 2023), yielding three cointegrated pairs: GS/MS, KO/PEP, and DAL/UAL. For each pair, the spread is modelled as an OU process and its conditional expected return is used as the Markowitz return input. Five portfolios are constructed and evaluated over a strict out-of-sample period (January 2024 – December 2025, 501 trading days), benchmarked against four comparators: historical mean-variance, equal-weight, buy-and-hold, and the S&P 500 index.

The OU Max-Sharpe portfolio achieves an annualised Sharpe ratio of 0.963 versus 0.868 for Historical Max-Sharpe, though a paired t-test ($p = 0.890$) and Wilcoxon test ($p = 0.922$) confirm this difference is not statistically significant; the central hypothesis is therefore only partially supported. The most robust finding concerns risk reduction: optimisation in spread space reduces annualised volatility to 9.2% and maximum drawdown to -9.5%, compared to 22.9% and -27.8% for the passive basket. Universal out-of-sample cointegration breakdown (Chow $F > 269$, $p < 0.001$) identifies regime fragility as the binding constraint. The system is implemented as a modular Python pipeline with an interactive Streamlit dashboard.

Contents

1 Introduction **2**

1.1 Motivation . . . 2

1.2 Research Question . . . 2

1.3 Research Hypothesis . . . 2

1.4 Aims and Objectives . . . 2

1.5 Report Overview . . . 3

2 Background and Literature Review **4**

2.1 Mean-Variance Portfolio Optimisation . . . 4

2.2 The Return Estimation Problem . . . 5

2.3 Pairs Trading and Statistical Arbitrage . . . 5

2.4 Cointegration Theory . . . 6

2.5 Structural Stability and Regime Detection . . . 7

2.6 The Ornstein-Uhlenbeck Process . . . 7

2.7 Spread-Based Return Estimation . . . 8

2.8 Summary . . . 9

3 Methodology **11**

3.1 Data and Universe . . . 11

3.2 Cointegration Screening . . . 12

3.2.1 Unit Root Testing . . . 12

3.2.2 Engle–Granger Procedure . . . 13

3.2.3 OOS Stability . . . 13

3.3 Spread Modelling . . . 13

3.3.1 Spread Construction . . . 14

3.3.2 Half-Life of Mean Reversion . . . 14

3.3.3 Hurst Exponent . . . 14

3.3.4 Rolling Z-Score . . . 15

3.3.5 Z-Score Trading Signals . . . 15

3.4 Return Estimation . . . 15

3.4.1 OU-Implied Expected Returns . . . 15

3.4.2 Historical Mean Returns . . . 16

3.4.3 The Zero-Return Baseline . . . 16

3.5 Portfolio Optimisation . . . 16

3.5.1 Minimum-Variance Formulation . . . 16

3.5.2 Maximum Sharpe Ratio Formulation . . . 17

3.5.3 Covariance Estimation . . . 17
3.5.4 Spread Space vs Asset Space . . . 17
3.6 Backtesting Framework . . . 17
3.6.1 In-Sample / Out-of-Sample Split . . . 18
3.6.2 Evaluation Metrics . . . 18
3.6.3 Statistical Robustness Tests . . . 19
3.7 Summary . . . 19

4 Implementation **20**
4.1 Pipeline Overview . . . 20
4.1.1 Data Ingestion and Caching . . . 21
4.2 Spread Exploration . . . 21
4.3 Cointegration Validation . . . 22
4.3.1 Screening Implementation . . . 22
4.3.2 Out-of-Sample Stability . . . 22
4.4 Return Estimation . . . 22
4.5 Portfolio Optimisation Module . . . 23
4.6 Backtesting Engine . . . 23
4.6.1 Architecture . . . 24
4.6.2 Signal Generation . . . 24
4.6.3 PnL and Transaction Costs . . . 24
4.6.4 Metrics and Benchmarks . . . 24
4.7 Dashboard Interface . . . 25
4.8 Summary . . . 25

5 Testing and Results **26**
5.1 Cointegration Screening . . . 26
5.2 Spread Characterisation . . . 26
5.3 Cointegration Validation . . . 32
5.3.1 KPSS Complement and Chow Structural Break Tests . . . 37
5.4 Return Estimation . . . 37
5.5 Backtesting Results . . . 41
5.5.1 Portfolio-Level Performance . . . 41
5.5.2 Per-Pair Mean-Reversion Backtest . . . 41
5.5.3 Statistical Tests . . . 45

6 Discussion **48**
6.1 Interpretation of Results . . . 48
6.2 Does OU-Implied μ Improve MPT? . . . 48
6.2.1 Predicting Returns vs Assuming Zero: When Does It Help? . . . 49
6.3 Role of Spread Universe Structure . . . 50
6.3.1 Statistical Significance of Performance Gains . . . 50
6.4 OOS Cointegration Instability . . . 51
6.5 Pairs Trading Signal Analysis . . . 51
6.6 Comparison with Market Benchmarks . . . 52
6.7 Hypothesis Evaluation . . . 52
6.8 Limitations . . . 53

6.9 Summary . . . 53

7 Conclusions and Evaluation **54**
7.1 Achievements . . . 54
7.2 Critical Evaluation . . . 54
7.2.1 Out of Sample Cointegration Breakdown . . . 54
7.2.2 Sample Period and Regime Bias . . . 55
7.2.3 Asset Universe and Selection Bias . . . 55
7.2.4 Short Out of Sample Evaluation Window . . . 55
7.2.5 Universe Concentration . . . 56
7.2.6 Methodological Scope . . . 56
7.2.7 Dashboard Scope . . . 56
7.2.8 Internal Consistency . . . 56
7.3 Future Work . . . 57

Bibliography **57**

Appendices **63**

A Use of Generative AI Tools **63**

B Project Plan **64**

C Interim Report **67**

D Supplementary Figures **71**

E Software Implementation **74**
E.1 Repository Structure . . . 74
E.2 Dependencies . . . 74
E.3 Reproducing Results . . . 75
E.4 System Manual . . . 75
E.4.1 Prerequisites . . . 75
E.4.2 Installation . . . 75
E.4.3 LSEG Data Configuration . . . 75
E.4.4 Running the Analysis Notebooks . . . 76
E.4.5 Running the Dashboard . . . 76
E.4.6 Extending the Codebase . . . 76
E.5 Code Listings . . . 76
E.5.1 Cointegration Screening (`cointegration.py`) . . . 77
E.5.2 Spread Characterisation (`spread_analysis.py`) . . . 78
E.5.3 OU-Implied Return Estimator (`return_estimation.py`) . . . 78
E.5.4 Markowitz Optimiser (`optimiser.py`) . . . 79
E.5.5 Pairs Backtesting Engine (`engine.py`) . . . 80
E.5.6 Efficient Frontier & Spread-vs-Asset Comparison (`optimiser.py`, `return_estimation.py`) 81
E.5.7 Performance Metrics (`metrics.py`) . . . 82
E.5.8 Benchmark Suite (`benchmarks.py`) . . . 83
E.5.9 Trade Logging & Aggregate Metrics (`engine.py`) . . . 84

E.5.10 Data Client with Incremental Cache (`refinitiv_client.py`) 85
E.5.11 Universe Configuration (`modelling/config.py`) 86

Chapter 1

Introduction

This project uses only publicly available equity price data and does not involve human subjects, personal data, or data collection. No ethical approval was required for the project.

1.1 Motivation

Markowitz mean-variance optimisation [1] provides a rigorous framework for constructing efficient portfolios, yet its practical performance is highly sensitive to the accuracy of expected return estimates. Historical mean returns are extremely noisy estimators of future returns, with standard errors that often exceed the estimates themselves [2], causing optimisers to produce unstable, concentrated allocations that perform poorly out-of-sample. Cointegration-based pairs trading strategies exploit long-run equilibrium relationships between asset prices to generate mean-reverting spread signals [3]. While extensive literature exists on pairs trading as a standalone strategy, few studies integrate spread-based predictions directly into the MPT framework as a systematic approach to return estimation. This project addresses this gap by investigating whether the mean-reverting dynamics of cointegrated spreads, modelled via the Ornstein-Uhlenbeck process, can provide more reliable return signals for Markowitz optimisation than traditional historical means, thereby improving out-of-sample portfolio performance.

1.2 Research Question

Can spread prediction between cointegrated asset pairs provide more reliable return estimates for Modern Portfolio Theory optimisation than traditional historical mean approaches?

1.3 Research Hypothesis

Spread-based return estimation derived from cointegrated asset pairs produces superior portfolio allocations compared to historical mean-variance optimisation, measured by risk-adjusted returns and portfolio stability, in terms of Sharpe ratio, volatility reduction and maximum drawdown.

1.4 Aims and Objectives

The project has the following objectives:

- Develop a spread-based return estimation approach using cointegration analysis of asset pairs that provides more stable signals than historical mean returns.
- Build an integrated portfolio optimisation system using Markowitz framework and validate through comprehensive back testing against traditional mean-variance approaches using Sharpe ratio, volatility reduction, and drawdown metrics.
- Create an interactive interface to demonstrate real-world applicability for portfolio managers.

1.5 Report Overview

This report is structured as follows. Chapter 2 provides the theoretical foundations underlying the project, covering Modern Portfolio Theory, the return estimation problem, cointegration theory, and the Ornstein-Uhlenbeck process. Chapter 3 presents the methodology, detailing the cointegration screening procedure, spread modelling, OU-implied return estimation, and the minimum-variance portfolio optimisation framework. Chapter 4 describes the implementation of the four-notebook pipeline, portfolio optimisation module, backtesting engine, and dashboard interface. Chapter 5 presents the empirical results across cointegration screening, spread characterisation, out-of-sample validation, return estimation, and portfolio backtesting. Chapter 6 interprets these results, addressing the central research question and examining the role of spread universe structure in driving performance. Chapter 7 summarises the achievements of the project, provides a critical evaluation, and outlines directions for future work.

Chapter 2

Background and Literature Review

This chapter reviews foundational theories underpinning the pairs trading strategy. Key concepts include Markowitz portfolio optimisation [1], the return estimation problem, statistical arbitrage via pairs trading [4], cointegration in the Engle–Granger framework [5], and the Ornstein–Uhlenbeck process for spread dynamics [6]. Together, these establish the theoretical framework for the methodology developed in Chapters 3– 5.

2.1 Mean-Variance Portfolio Optimisation

Markowitz introduced Modern Portfolio Theory (MPT) in his seminal work *Portfolio Selection: Efficient Diversification of Investments* [1], establishing a quantitative framework for constructing optimal portfolios. MPT remains one of the most influential contributions to portfolio theory and optimisation, providing the theoretical foundation upon which much of modern finance is built. The theory characterises portfolio performance using two statistical measures: the mean as a proxy for expected return, and the standard deviation as a measure of risk. Within this framework, Markowitz defines a portfolio as **inefficient** if it is possible to achieve a higher expected return without increasing risk, or to reduce risk without sacrificing expected return. This gives rise to the *Efficient Frontier*, the set of portfolios that offer the maximum expected return for a given level of risk, where any portfolio lying below the curve is considered suboptimal. The selection of an optimal portfolio along the frontier is determined by the investor's individual risk-return preference, reflecting the principle that higher expected returns are generally associated with greater risk. Central to MPT is also the principle of diversification: by combining assets whose returns are not perfectly correlated, investors can reduce overall portfolio risk without proportionally reducing expected return, since assets that do not move in perfect unison partially offset each other's fluctuations. This shifted the focus of portfolio construction from selecting individually attractive assets to engineering a portfolio whose constituent asset interactions collectively minimise risk, which is a conceptual shift that underpins virtually all modern asset allocation practice.

However, MPT is not without limitations. Markowitz himself identified a fundamental shortcoming that the use of variance as the measure of risk was a pragmatic choice rather than a theoretically superior one. In his 1959 book, he explicitly argued that analyses based on semi-variance tend to produce superior portfolios than those based on variance [1], since semi-variance

captures only downside deviations, which are the losses that investors genuinely seek to avoid, whereas variance penalises upside and downside movements equally. Nevertheless, variance was retained on practical grounds of computational cost, convenience, and familiarity, particularly given the constraints of the era [1], as compared to semi-variance. A further, and arguably more practical, limitation of MPT is the instability of return estimation [7]. Fabozzi, Gupta, and Markowitz demonstrate empirically that historical return estimates are highly sensitive to the time period selected: for instance, the MSCI Emerging Markets index returned 16.3% annually over 1991–1995, yet only 0.1% over 1996–2000; they conclude that return estimation is inherently uncertain, with no objectively correct answer [7]. Notwithstanding these limitations, MPT remains foundational to both the theory and practice of finance. The instability of return estimation presents a particular challenge for portfolio construction, which motivates cointegration-based approaches as a more tractable alternative to exploit stable, long-run equilibrium relationships between asset prices rather than relying on point estimates of expected returns.

2.2 The Return Estimation Problem

Historical mean returns are extremely noisy estimators of future returns, with standard errors that often exceed the estimates themselves [2]. As Merton demonstrates, even assuming a constant expected return, it would take a very long history of returns to obtain an accurate estimate due to the high volatility-to-sample-size ratio [8].

In response to these limitations, Jorion proposes a Bayes-Stein shrinkage estimator that improves upon the sample mean by shrinking return estimates toward a common prior, reducing estimation error under a portfolio loss function [9]. Black and Litterman extend this Bayesian perspective further by substituting historical means with Capital Asset Pricing Model (CAPM)-implied equilibrium returns and updating these with investor views, producing more stable inputs for portfolio optimisation [10].

Brealey provides a foundational decomposition of stock price movements into four influences: market-wide, industry-specific, subgroup, and firm-specific factors [11]. This decomposition suggests that stocks within the same industry share both market and industry systematic influences, making sector-paired stocks natural candidates for cointegration analysis.

However, these approaches operate at the level of individual assets and implicitly assume parameter stability over the sample window; they do not exploit long-run equilibrium structure between related securities, nor do they directly address how to extract more reliable return estimates from such cross-series information, which motivates the cointegration-based perspective adopted in this project.

2.3 Pairs Trading and Statistical Arbitrage

Pairs trading is a market-neutral, self-financing hedging strategy widely utilised by hedge funds [3] where a market neutral portfolio can be constructed by identifying an asset pairs: one security to go long, the other to go short. An opportunity arises when the spread between the asset pair deviates away from its mean, where a trader would short the security that increases and long the security that decreases, with the expected behaviour that the spread will revert to its mean.

Krauss provides the most comprehensive survey of the field, categorising over 90 pairs trading papers into five frameworks: distance, cointegration, time series, stochastic control, and other

approaches [12]. The distance, cointegration, and copula methods have been subject to direct empirical comparison. Rad directly compared the three methods on US equities (1962-2014) and identified that cointegration delivers the strongest risk-adjusted returns after transactional costs [13]. Huck and Afawubo corroborate the finding on a decade of S%P 500 and Nikkei data, finding that cointegration consistently outperforms the distance method, which yields negligible risk-adjusted returns after transaction costs [14]. Do and Faff further demonstrate that the profitability of the distance method has declined over time but remains effective during market turbulence. The performance is, however, contingent on transaction costs [15].

Vidyamurthy formalises the cointegration approach to pairs trading, arguing that while individual stock prices are difficult to predict, their relative behaviour can be, provided the pair shares a statistically validated long-run equilibrium relationship [4]. He distinguishes cointegration from correlation, emphasising that correlation alone does not guarantee mean reversion and is therefore insufficient for constructing a tradeable spread. His framework proceeds in three phases: pair selection via cointegration testing, hedge ratio estimation from the cointegrating vector to construct a self-financing mean-reverting portfolio, and signal generation based on spread deviations from long-run equilibrium, with positions unwound upon reversion.

While this literature establishes cointegration-based pairs trading as an effective standalone statistical arbitrage strategy, most studies focus on the performance of individual pairs or simple equally weighted portfolios, and do not analyse how to allocate capital across many spreads within a Markowitz framework or how spread-based signals compare empirically to traditional expected-return estimators for portfolio construction.

2.4 Cointegration Theory

Cointegration was introduced by Engle and Granger in to analyse long-run equilibrium relationships between non-stationary time series, addressing the spurious regression problem identified by Granger and Newbold [5], [16]. A time series is said to be integrated of order d, denoted $I(d)$, if it requires d differences to achieve stationarity; most financial price series are $I(1)$, meaning their first differences are stationary [17]. Cointegration allows the regression of $I(1)$ variables directly, provided a stationary linear combination exists, as compared to differencing series to achieve stationarity, which discards long-run information.

The test follows a two-step approach: Ordinary Least Squares (OLS) estimation of the cointegrating regression, followed by an Augmented Dickey-Fuller (ADF) unit root test on the residuals. A confirmed cointegrating relationship implies the existence of an Error Correction Model (ECM), in which the lagged residual enters as an equilibrium correction term, capturing the speed at which the spread adjusts back to its long-run mean following a deviation [5]. A limitation of this approach is that errors from the first step propagate into the second, and only a single cointegrating relationship can be identified. Johansen extends this to the multivariate case via a Vector Autoregression framework, estimating the number of cointegrating vectors simultaneously using trace and maximum eigenvalue statistics, offering greater statistical power than the bivariate Engle-Granger procedure [18].

In practice, however, empirical implementations of pairs trading often rely on a single Engle–Granger relationship estimated over a fixed window, with limited attention to small-sample bias, error propagation across the two steps, or the presence of structural breaks, despite evidence that such breaks can materially distort cointegration inference and trading performance.

2.5 Structural Stability and Regime Detection

Financial time series are often subject to structural breaks induced by crises such as financial crashes, geopolitical conflicts, major policy shifts, or technological shocks that fundamentally alter growth and pricing dynamics. These breaks can alter the underlying data-generating process and invalidate models calibrated on earlier regimes. Structural stability refers to the constancy of model parameters over time. Whenever this assumption fails, forecasts and risk estimates based on a single pooled regression become unreliable.

The Chow test, introduced by Chow in 1960, provides a classical F-test for a structural break at a known point in time by comparing the fit of a pooled regression against separate regressions estimated on two subsamples [19]. In the context of cointegrated pairs, the test evaluates whether the intercept and hedge ratio of the spread relationship remain stable, where the rejection of the null of parameter equality indicates a regime change in the cointegrating relationship, directly informing the reliability of spread-based return forecasts in pairs trading models. Consequently, formally testing for structural breaks via the Chow test is a crucial prerequisite before relying on cointegration-based signals for portfolio construction. Lu et al. implement a structural break–aware pairs trading strategy that applies the Chow test to the spread relationship and report improved performance stability once breaks are accounted for [20]. Beyer et al. demonstrate that structural breaks in cointegrating relations can cause standard tests to miss cointegration altogether, with the long-run relationship re-emerging once such breaks are modelled explicitly [21].

Within the Fractal Market Hypothesis framework, Peters uses the Hurst exponent to quantify persistence in financial returns [22]. The Hurst exponent H is a scaling exponent used to characterise long-range dependence and persistence in time series [23]. Values $H \approx 0.5$ correspond to an uncorrelated random walk, $H > 0.5$ indicate persistent, trend-following behaviour, and $H < 0.5$ indicate anti-persistent, mean-reverting dynamics, where increases are more likely to be followed by decreases and vice versa . Carbone et al. extend this idea by introducing a detrending moving average (DMA) estimator for a time-dependent Hurst exponent computed over sliding windows, and show that financial series such as the DAX and BOBL exhibit substantially greater local variability in H than synthetic monofractal series, indicating evolving correlation regimes in financial time series [24].

Nevertheless, most empirical pairs-trading implementations do not incorporate these diagnostics into a systematic rule for selecting, dropping, or reweighting pairs, leaving regime detection largely as an ex post diagnostic rather than an integrated part of the trading pipeline.

2.6 The Ornstein-Uhlenbeck Process

The Ornstein–Uhlenbeck (OU) process, introduced by Ornstein and Uhlenbeck as a mean-reverting refinement of Einstein's Brownian motion model, is the canonical continuous-time Gaussian Markov process used to represent mean-reverting dynamics in finance, for example in short-rate models such as Vasicek [25], [26]. Its key property is that the expected change in the process is proportional to the deviation from a long-run equilibrium level, so large departures from equilibrium induce stronger expected pullbacks, while a Gaussian diffusion term continuously perturbs the state around this mean.

Elliott et al. develop a state–space framework for pairs trading, modelling the latent spread as a discrete-time mean-reverting Gaussian Markov process, which can be viewed as a discretised Ornstein–Uhlenbeck process, and the observed spread as a noisy measurement of this hidden state

[6]. The model is calibrated via Kalman filtering and EM-based maximum likelihood estimation, and trading signals are generated when the observed spread deviates sufficiently from the filtered mean, with additional first-passage-time results for the corresponding OU process used to time trade exits. This provides a foundation for treating the spread as a mean-reverting signal, directly motivating the use of OU-based spread models in pairs trading.

These models, however, are typically calibrated once and then used as fixed signal generators for individual spreads; they do not examine the robustness of OU parameter estimates across regimes or their effectiveness as cross-sectional return forecasts within a portfolio optimisation framework.

2.7 Spread-Based Return Estimation

A complementary body of work develops spread-based return estimation by modelling the spread itself as a mean-reverting process and deriving trading rules directly from its dynamics. Bertram presents a general framework for optimal trading in Itô diffusions and specialises it to the Ornstein–Uhlenbeck case, deriving closed-form expressions for the mean and variance of trade length and the rate of profit, and then solving for entry and exit thresholds that maximise expected return or Sharpe ratio when the price (or log-price) follows an (exponential) OU process [27], [28]. Leung and Li extend this optimal stopping approach by formulating a double stopping problem for an OU-modelled spread with transaction costs and stop-loss, and rigorously characterise the optimal entry and exit regions in terms of the OU parameters, transaction costs, and risk limits, further reinforcing the view that these parameters encode the spread's conditional expected return and risk [29]. As discussed in Section 2.6, Elliott et al. likewise model the spread as a latent mean-reverting Gaussian Markov process observed in Gaussian noise, estimated via Kalman filtering and EM, and use the filtered and predicted spread means, effectively OU-like spread forecasts, as the basis for timing trade entry and exit in a pairs trading strategy [6]. Collectively, these contributions provide a theoretical foundation for treating the dynamics of a mean-reverting spread as a return-generating signal, motivating the use of OU-implied or state-space-implied spread forecasts as inputs to portfolio optimisation in this project. A common feature of these studies is their focus on the optimal exploitation of a single mean-reverting spread, with OU parameters taken as given once estimated; they do not investigate how such OU-implied signals should be combined across many spreads or benchmarked against simpler estimators such as historical means in a portfolio setting.

Beyond single-spread trading rules, Zhang, Leung, and Aravkin propose an optimisation-based approach to construct sparse portfolios whose values are well represented by an Ornstein–Uhlenbeck process, jointly estimating portfolio weights and OU parameters via penalised maximum likelihood and using the inferred mean-reversion characteristics as inputs for trading decisions [30]. In parallel, Johansson et al. develop a simple yet competitive method for predicting time-varying covariance matrices of financial returns, showing that lightweight EWMA-style predictors can match or outperform more sophisticated MGARCH models when evaluated through portfolio loss functions [31]. These portfolio-level and covariance results complement the spread-level OU literature, supporting the design of a framework in which OU-implied spread forecasts and shrinkage-based covariance estimates jointly drive Markowitz optimisation in spread space for this project. Zhang et al. emphasise statistical fit and sparsity of OU-representable portfolios, while Johansson et al. focus on stabilising covariance forecasts; neither line of work takes the mean component as an object of study in its own right, or evaluates the out-of-sample performance of OU-based spread return forecasts relative to conventional historical mean estimates when used as inputs to Markowitz optimisation.

Beyond the spread- and portfolio-level OU literature discussed above, several strands of work combine cointegration or pairs trading with mean–variance objectives. Rudoy and Rohrs derive closed-form mean–variance optimal portfolios in a cointegrated VAR setting, showing that the pure cointegration (beta) portfolio is only optimal in the infinite-horizon limit, whereas finite-horizon optimal portfolios mix the cointegrating direction with expected-return components [32]. Yamada and Primbs formulate a model-predictive-control framework in which a vector of cointegrated spreads is modelled by a VAR and, at each rebalance, a conditional mean–variance optimisation is solved in spread space with an explicit prediction horizon and transaction costs [33]. More recently, Naccarato et al. has combined Markowitz portfolio optimisation with cointegrated pairs trading in a long-term investment context [34]. These studies confirm the relevance of Markowitz-style objectives in cointegration and pairs-trading settings but do not use OU-implied spread returns as estimators, nor do they directly compare spread-based forecasts against historical mean returns as inputs to portfolio optimisation, which is the focus of this project.

2.8 Summary

The literature reviewed in this chapter establishes a coherent pathway from classical portfolio theory to spread-based return modelling for statistical arbitrage. Modern Portfolio Theory provides the foundational optimisation framework, casting portfolio choice as a mean–variance trade-off but suffering from a central fragility: its heavy reliance on noisy and unstable estimates of expected returns [1], [7]. Subsequent work on the return estimation problem, including Bayesian shrinkage and equilibrium-based priors, seeks to stabilise these inputs but does not fully resolve the difficulty of forecasting asset-level returns under high volatility and structural regime shifts [9], [10]. This motivates a shift from forecasting individual returns to exploiting more stable long-run relationships, particularly via cointegration among economically related securities [5], [11].

Within this setting, pairs trading and broader statistical arbitrage strategies use cointegrated spreads to construct market-neutral, self-financing positions whose profitability relies on mean-reverting deviations from long-run equilibrium [3], [4]. Engle–Granger and Johansen cointegration frameworks formalise the existence and estimation of such equilibrium relations [18], while structural-break tools such as Chow tests show that these relations can evolve over time and must be monitored before using spreads as predictive signals [19], [20], [21]. Complementary regime-detection methods based on the time-varying Hurst exponent further characterise when mean-reversion regimes shift [22], [24]. Against this backdrop, the Ornstein–Uhlenbeck process and its discrete-time Gaussian Markov analogues emerge as canonical models for mean-reverting dynamics, providing tractable likelihoods for calibration and explicit characterisations of mean reversion, volatility, and first-passage behaviour relevant for trading rules and risk control [6], [25], [26].

Building on these ingredients, a complementary strand of work derives trading strategies and portfolio constructions directly from mean-reverting spread dynamics. Bertram develops a first-passage-time framework for Itô diffusions and solves for optimal OU trading bands under objectives such as expected return and Sharpe ratio [27], [28]. Leung and Li extend this optimal stopping perspective to OU-modelled spreads with transaction costs and stop-loss constraints [29], while Elliott et al. treat the spread as a latent Gaussian mean-reverting state in a state–space model, using Kalman filtering and EM to obtain filtered and predicted spread means that serve as timing signals for pairs trades [6]. At the portfolio level, Zhang, Leung, and Aravkin jointly optimise portfolio weights and OU parameters via penalised maximum likelihood to identify sparse mean-reverting

portfolios, and Johansson et al. show that simple covariance predictors can be competitive with more complex models when judged by portfolio loss functions [30], [31]. Related studies combine cointegration or pairs trading with mean–variance objectives, including closed-form finite-horizon mean–variance portfolios in cointegrated VAR systems [32], model-predictive-control formulations that perform conditional mean–variance optimisation in spread space [33], and Markowitz optimisation built on cointegrated pairs trading for long-term investment [34].

Collectively, this literature supports the central premise of this project: that cointegration-defined, mean-reverting spreads can be treated as structured return signals and embedded in portfolio optimisation frameworks. However, existing work either improves return estimation at the individual-asset level, focuses on single-spread OU trading rules or OU-based portfolio construction, or incorporates cointegration into mean–variance optimisation without directly comparing OU-implied, spread-based forecasts against historical mean returns as inputs to Markowitz optimisation. Addressing this gap—by evaluating spread-based OU return estimates as portfolio inputs relative to traditional historical means—is the primary contribution of this project.

Chapter 3

Methodology

This chapter details the methodology employed by this project. Section 3.1 describes the asset universe and data sourcing. Section 3.2 presents the cointegration screening procedure used to identify suitable pairs. Sections 3.3 and 3.4 develop the spread modelling and return estimation frameworks, respectively. Section 3.5 formulates the portfolio optimisation problem, and Section 3.6 describes the backtesting framework used to evaluate performance.

3.1 Data and Universe

The project uses a **26-asset universe** of US equities (see Table 3.1). Stocks were selected on the basis of market capitalisation and sector-comembership, as economic similarity provides a prior expectation of long-run co-movement, which is a necessary condition for cointegration.

From these 26 stocks, 16 candidate pairs are pre-selected within sectors. Restricting pair formation to within-sector groupings reflects the economic rationale that stocks exposed to common demand drivers, regulatory environments, and supply chains tend to co-move [11] and are therefore more likely to exhibit the long-run equilibrium relationship required for cointegration. (see Section 2.2).

Daily adjusted close prices are sourced from LSEG Data Library [35]. S&P 500 benchmark returns are obtained via the `yfinance` [36] library. The full sample spans 1 January 2018 to 31 December 2025 and is partitioned into:

- **In-Sample (IS)**: 1 January 2018 – 31 December 2023 (cointegration screening, calibration)
- **Out-of-Sample (OOS)**: 1 January 2024 – 31 December 2025 (backtesting)

This IS/OOS split ensures that all model parameters are estimated without look-ahead bias.

Table 3.1: Investment Universe: 26 US Equities by Sector and Market Cap (29 Dec 2023)

Sector	**Ticker**	**Name**	**Mkt Cap ($B)**
Semiconductors	NVDA.O	Nvidia	1,223
	AMD.O	AMD	238
	TSM.N	TSMC	501
	INTC.O	Intel	212
Consumer Staples	KO.N	Coca-Cola	255
	PEP.O	PepsiCo	234
	COST.O	Costco	293
	TGT.N	Target	66
Financials	JPM.N	JPMorgan Chase	492
	BAC.N	Bank of America	266
	GS.N	Goldman Sachs	126
	MS.N	Morgan Stanley	153
Energy	XOM.N	ExxonMobil	400
	CVX.N	Chevron	281
	SLB.N	Schlumberger	74
	HAL.N	Halliburton	32
Airlines	DAL.N	Delta Air Lines	26
	UAL.O	United Airlines	14
E-Commerce/Cloud	AMZN.O	Amazon	1,570
	MSFT.O	Microsoft	2,795
	META.O	Meta	910
	GOOGL.O	Alphabet	1,755
Healthcare	JNJ.N	Johnson & Johnson	377
	PFE.N	Pfizer	163
	MRK.N	Merck	276
	ABBV.N	AbbVie	274

3.2 Cointegration Screening

The cointegration screening pipeline identifies which of the 16 candidate pairs exhibit a statistically significant long-run equilibrium relationship, following the two-step Engle–Granger procedure [5].

3.2.1 Unit Root Testing

A prerequisite for cointegration is that each of the individual price series is integrated of order one, denoted $I(1)$: non-stationary in levels, but stationary in first differences ($\Delta P_t = P_t - P_{t-1}$). For each stock, the Augmented Dickey-Fuller (ADF) test [37] is applied to both the level series P_t and its first order ΔP_t. A series is classified $I(1)$ if

1. ADF on levels **fails to reject** unit root ($p > 0.05$), **suggesting non-stationarity**,
2. ADF on ΔP_t **rejects** unit root ($p \leq 0.05$), indicating stationarity.

The ADF regression is:

$$\Delta P_t = \alpha + \gamma P_{t-1} + \sum_{j=1}^{p} \delta_j \Delta P_{t-j} + \varepsilon_t \tag{3.1}$$

where $\Delta P_t = P_t - P_{t-1}$ (first difference), α is the constant, γ tests the unit root ($H_0 : \gamma = 0$), δ_j are lag coefficients, and $\varepsilon_t \sim N(0, \sigma^2)$ is white noise. Lag order p is selected via Akaike Information Criterion (AIC). Only pairs where both stocks are $I(1)$ proceed to cointegration testing.

3.2.2 Engle–Granger Procedure

For each candidate pair (Y_t, X_t) passing the $I(1)$ filter, the Engle–Granger two-step procedure [5] is applied:

Step 1: Cointegrating regression. With Y_t as dependent variable and X_t as independent variable, estimate the static OLS regression over the in-sample period:

$$Y_t = \alpha + \beta X_t + \epsilon_t \tag{3.2}$$

where β is the hedge ratio and α the intercept.

Step 2: Residual stationarity test. Apply the ADF test to OLS residuals $\hat{\epsilon}_t = Y_t - \hat{\alpha} - \hat{\beta} X_t$. Because residuals are estimated, standard ADF critical values are inappropriate; **MacKinnon critical values** [38] are used instead, accounting for estimation uncertainty.

To avoid ordering effects, both $(Y|X)$ and $(X|Y)$ are tested; the ordering with the lowest residual ADF p-value is retained. A pair is deemed cointegrated if $p < 0.05$.

3.2.3 OOS Stability

To assess whether in-sample cointegration relationships persist out-of-sample, two additional tests are applied to each cointegrated pair.

KPSS Complement: The Kwiatkowski–Phillips–Schmidt–Shin (KPSS) test [39] complements ADF by reversing the null hypothesis: H_0 is stationarity rather than a unit root. Mean reversion is only accepted with confidence if ADF rejects non-stationarity and KPSS fails to reject stationarity simultaneously. Performing both KPSS and ADF tests reduces the risk of false positives, as compared to solely conducting the ADF test. A KPSS p-value ≤ 0.05 indicates rejection of the stationarity null; a value of 0.010 denotes the tabulated lower bound (i.e. $p \leq 0.010$), indicating the statistic is beyond the table range.

Chow structural break test: The test assesses whether the cointegrating regression coefficients $(\hat{\beta}, \hat{\alpha})$ are stable across the In-Sample/Out-of-Sample boundary. Under H_0 (no structural break), the pooled residual sum of squares from separate In-Sample/Out-of-Sample regressions should not differ significantly from the pooled regression. The test statistic is:

$$F = \frac{(RSS_{\text{pooled}} - RSS_{\text{IS}} - RSS_{\text{OOS}})/k}{(RSS_{\text{IS}} + RSS_{\text{OOS}})/(T - 2k)} \tag{3.3}$$

where $k = 2$ (intercept and slope) and T is the total sample size. A large F-statistic (p-value < 0.05) rejects H_0 and confirms a structural cointegration break.

3.3 Spread Modelling

For each cointegrated pair, the spread is constructed and its mean-reverting dynamics are characterised using the metrics: half-life, Hurst exponent and Rolling z-scores.

3.3.1 Spread Construction

The spread is defined as the Ordinary Least Squares (OLS) residual from the cointegrating regression estimated from the in-sample period using the equation below:

$$S_t = Y_t - \hat{\beta} X_t - \hat{\alpha} \tag{3.4}$$

Where $\hat{\beta}$ and $\hat{\alpha}$ is the hedge ratio and intercept respectively. These variables are fixed at their in-sample OLS estimates and applied without recalibration to the out-of-sample period. Under the cointegration hypothesis, S_t is a stationary, mean-reverting process with a long-run equilibrium near zero. The daily log-return from the spread position is computed in the equation below:

$$r_t^S = \ln\left(\frac{Y_t}{Y_{t-1}}\right) - \hat{\beta} \cdot \ln\left(\frac{X_t}{X_{t-1}}\right) \tag{3.5}$$

Using log returns ensures that the spread return r_t^S is in the same units as the expected change $\mathbb{E}[\Delta S]$ derived from the continuous-time OU model, facilitating the use of OU-implied forecasts as predicted expected returns for portfolio optimisation.

3.3.2 Half-Life of Mean Reversion

The speed of mean-reversion is characterised via a discrete-time AR(1) regression on the spread:

$$\Delta S_t = \lambda S_{t-1} + \varepsilon_t \tag{3.6}$$

where $\lambda < 0$ implies mean-reversion.

The half-life h, measured in trading days, (expected time for the spread to revert halfway to its equilibrium) is given in the equation below:

$$h = -\frac{\ln 2}{\ln(1+\lambda)} \approx -\frac{\ln 2}{\lambda} \tag{3.7}$$

where a shorter half-life indicates rapid mean-reversion and stronger trading quality. Pairs with an estimated half-life shorter than the lookback window (typically w=60 trading days) are preferred, as this allows for multiple reversion cycles to occur within a single estimation period.

3.3.3 Hurst Exponent

The Hurst exponent H measures the persistence or anti-persistence of a time-series, by using the variogram method. For a time series S_t, the variogram at τ is:

$$V(\tau) = \mathbb{E}\left[(S_{t+\tau} - S_t)^2\right] \tag{3.8}$$

For a fractional process, $V(\tau) \propto \tau^{2H}$, so H is estimated by regressing $\log V(\tau)$ on $\log \tau$ across lags $\tau = 1, \ldots, 100$ and halving the slope coefficient. Only pairs with $H < 0.5$ are retained as they indicate mean-reverting behaviour, as another confirmation beyond ADF test. Pairs with $H = 0.5$ correspond to a random walk, and $H > 0.5$ indicates trending behaviour.

3.3.4 Rolling Z-Score

To generate trading signals that adapt to time-varying spread dynamics, a rolling z-score is computed:

$$z_t = \frac{S_t - \bar{S}_t^{(w)}}{\sigma_t^{(w)}} \tag{3.9}$$

where $\bar{S}_t^{(w)}$ and $\sigma_t^{(w)}$ denote the rolling mean and standard deviation of the spread respectively over a trailing window of $w = 60$ trading days. The rolling window allows the signal to adapt to structural shifts in spread dynamics while remaining responsive to short-term dislocations.

3.3.5 Z-Score Trading Signals

Trading signals are generated using the following thresholds on the rolling z-score:

- Entry: $|z_t| \geq 2.0\sigma$ (long if $z_t \leq -2.0$, short if $z_t \geq +2.0$)
- Exit: $|z_t| \leq 0.0$ (reversion to rolling mean)
- Stop-loss: $|z_t| \geq 4.0\sigma$

3.4 Return Estimation

This section develops the two competing approaches of estimating returns for portfolio optimisation: the proposed OU-Implied Expected Returns for spreads, and the traditional historical mean returns baseline, which are then used to answer the research hypothesis.

3.4.1 OU-Implied Expected Returns

The Ornstein-Uhlenbeck (OU) process models the mean-reverting spread as a continuous-time stochastic Brownian motion model:

$$dS_t = \kappa(\theta - S_t)\,dt + \sigma\,dW_t \tag{3.10}$$

where $\kappa > 0$ is the speed of mean reversion, θ is the **sample mean of the spread over the in-sample period** (long-run equilibrium), σ is the diffusion coefficient, and W_t is Brownian motion. The parameter κ is related to the discrete-time half-life (Equation 3.7) by:

$$\kappa = \frac{\ln 2}{h} \tag{3.11}$$

The expected change in the spread over a single period, conditional on the current spread level S_t, is:

$$\mathbb{E}[\Delta S_{t+1} \mid S_t] = (\theta - S_t)\left(1 - e^{-\kappa}\right) \tag{3.12}$$

When the spread is below its equilibrium ($S_t < \theta$), the expected change is positive (reversion upward), and vice versa. The magnitude of the expected return is proportional to both the displacement from equilibrium and the speed of mean reversion.

The spread return for a pair (Y, X) with hedge ratio β is defined as:

$$r_t^S = \ln\left(\frac{Y_t}{Y_{t-1}}\right) - \beta \cdot \ln\left(\frac{X_t}{X_{t-1}}\right) \tag{3.13}$$

The annualised OU-implied expected return for spread i is:

$$\mu_i^{\mathrm{OU}} = 252 \cdot \mathbb{E}[\Delta S_{i,T} \mid S_{i,T}] \tag{3.14}$$

Where T denotes the end of the in-sample period. T captures the model's forecast of the spread's tendency to revert at the point of portfolio construction.

The OU-implied expected spread return is then used as the expected return vector $\boldsymbol{\mu}$ in the portfolio optimisation framework (Section 3.5).

3.4.2 Historical Mean Returns

As a baseline estimator, the expected return for spread i is the sample mean of in-sample log spread returns:

$$\hat{\mu}_i^{\mathrm{hist}} = \frac{252}{T} \sum_{t=1}^{T} r_{i,t}^{S} \tag{3.15}$$

This estimator is unbiased but exhibits high sampling variance, a well-documented limitation in portfolio optimisation that motivates the use of model-based alternatives such as the OU estimator [8].

3.4.3 The Zero-Return Baseline

A natural test of the OU return estimates is comparison against a zero-return assumption. Setting $\boldsymbol{\mu} = \mathbf{0}$ asserts that the expected return of every spread is zero. Under this assumption, the maximum-Sharpe objective reduces to:

$$\max_{\mathbf{w}} \frac{0 - r_f}{\sqrt{\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}}} = \min_{\mathbf{w}} \sqrt{\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}} \tag{3.16}$$

which is equivalent to the minimum-variance problem. Therefore, the **minimum-variance portfolio serves as the zero-return baseline** which is the optimal allocation when no return signal is assumed. Any improvement from using OU-implied returns is measured by comparing maximising Sharpe (OU) against minimum-variance in the spread space. The minimum-variance objective is return-agnostic by construction as its weights depend only on Σ and are invariant to the choice of μ. This means that OU-implied returns can only be evaluated through the max-Sharpe objective, not through min-variance weights.

3.5 Portfolio Optimisation

Given a vector of expected spread returns $\boldsymbol{\mu} \in \mathbb{R}^n$ and a covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$ for n spread strategies, the portfolio allocation problem is cast in the Markowitz mean-variance framework [1].

3.5.1 Minimum-Variance Formulation

The minimum-variance portfolio solves:

$$\begin{aligned} \min_{\mathbf{w}} \quad & \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w} + \delta \|\mathbf{w}\|_2^2 \\ \text{subject to} \quad & \mathbf{1}^\top \mathbf{w} = 1, \\ & w_i \geq 0 \quad \forall i \end{aligned} \tag{3.17}$$

where $\delta \geq 0$ is an optional L_2 regularisation parameter that penalises concentrated allocations and improves numerical stability when $\mathbf{\Sigma}$ is near-singular.

3.5.2 Maximum Sharpe Ratio Formulation

The maximum Sharpe ratio portfolio maximises:

$$\max_{\mathbf{w}} \quad \frac{\mathbf{w}^\top \boldsymbol{\mu} - r_f}{\sqrt{\mathbf{w}^\top \mathbf{\Sigma} \mathbf{w}}} - \delta \|\mathbf{w}\|_2^2 \tag{3.18}$$

subject to the same long-only and full-investment constraints. Here $r_f = 0.02$ denotes the annualised risk-free rate and $\delta \geq 0$ is the same optional L_2 regularisation parameter as in the minimum-variance problem, discouraging extreme weight concentration.

Expected returns are annualised from daily log returns via the transformation $\mu_{\text{ann}} = \exp(252 \cdot \mu_{\text{daily}}) - 1$, and volatilities are annualised as $\sigma_{\text{ann}} = \sigma_{\text{daily}} \cdot \sqrt{252}$.

3.5.3 Covariance Estimation

The covariance matrix $\mathbf{\Sigma}$ is estimated using the **Ledoit–Wolf shrinkage estimator** [2], which linearly combines the sample covariance matrix with a scaled identity target:

$$\hat{\mathbf{\Sigma}}_{\text{LW}} = (1 - \rho)\, \hat{\mathbf{\Sigma}}_{\text{sample}} + \rho \cdot \mu_{\text{tr}} \mathbf{I} \tag{3.19}$$

where $\rho \in [0, 1]$ is determined analytically by minimising the expected Frobenius norm of the estimation error. Shrinkage improves conditioning relative to the raw sample covariance, which is important when the number of assets is non-trivially large relative to the time series length. All inputs to the optimiser are expressed in annualised units: $\boldsymbol{\mu}_{\text{ann}}$ and $\mathbf{\Sigma}_{\text{ann}} = 252 \cdot \mathbf{\Sigma}_{\text{daily}}$.

3.5.4 Spread Space vs Asset Space

A distinctive feature of this methodology is that optimisation is performed in *spread space* rather than directly over individual assets. Each dimension of the portfolio vector $\mathbf{w}$ corresponds to a spread strategy (a cointegrated pair), not an individual stock. This has several advantages:

- **Dimensionality reduction**: the number of spreads (up to 16) is smaller than the number of assets (26), yielding a lower-dimensional and better-conditioned optimisation problem.
- **Built-in hedging**: each spread is by construction a hedged position (long one leg, short the other by β units), so the portfolio inherits partial market-neutrality.
- **Signal coherence**: the OU-implied return estimates are naturally defined at the spread level, avoiding the need to decompose spread-level predictions back into asset-level expected returns.

The weight w_i allocated to spread i determines the fraction of capital deployed to that pair strategy. The resulting portfolio return on day t is $r_t^P = \sum_i w_i \cdot r_{i,t}^S$.

3.6 Backtesting Framework

The backtesting engine simulates a z-score-based mean-reversion strategy for each pair, then evaluates the optimised portfolio of pair strategies against benchmarks.

3.6.1 In-Sample / Out-of-Sample Split

The temporal split is strict: the in-sample period (1 January 2018 – 31 December 2023) is used for all estimation, namely cointegrating regressions, hedge ratios, OU parameters, spread statistics, and portfolio weights. The out-of-sample period (1 January 2024 – 31 December 2025) is used exclusively for performance evaluation. No re-estimation or recalibration is performed during the OOS period, ensuring a clean test of whether in-sample model parameters generalise.

Pair-level signal generation. For each pair, a position signal is generated using a z-score state machine with three thresholds:

- **Entry** ($|z_t| \geq z_{\text{entry}} = 2.0$): open a position when the spread is sufficiently dislocated. If $z_t \leq -z_{\text{entry}}$, go long the spread (expecting reversion upward); if $z_t \geq z_{\text{entry}}$, go short the spread.
- **Exit** ($|z_t| \leq z_{\text{exit}} = 0.0$): close the position when the spread has reverted to its rolling mean.
- **Stop-loss** ($|z_t| \geq z_{\text{stop}} = 4.0$): force-close the position if the spread diverges further, limiting downside risk.

Position sizing and transaction costs. The full initial capital ($C_0 = \$100{,}000$) is deployed into each active spread position. One spread unit costs $Y_t + |\beta| \cdot X_t$ in notional terms, and the number of units is $n_t = C_0/(Y_t + |\beta| X_t)$. Transaction costs of $c = 10$ basis points are incurred proportionally on each position change. The daily strategy return is:

$$r_t = \frac{\text{pos}_{t-1} \cdot n_{t-1} \cdot (\Delta Y_t - \beta \cdot \Delta X_t) - \text{TC}_t}{C_0} \tag{3.20}$$

where $\text{TC}_t = |\Delta \text{pos}_t| \cdot C_0 \cdot (c/10{,}000)$.

3.6.2 Evaluation Metrics

Portfolio performance is assessed using the following metrics:

Annualised Sharpe ratio. The ex-post Sharpe ratio is computed as:

$$\text{SR} = \frac{\bar{r} - r_f/252}{\sigma_r} \cdot \sqrt{252} \tag{3.21}$$

where $\bar{r}$ is the mean daily return, $r_f = 0.02$ is the annualised risk-free rate, and σ_r is the standard deviation of daily returns.

Maximum drawdown. The maximum peak-to-trough decline in cumulative returns:

$$\text{MDD} = \max_t \left(\frac{\max_{\tau \leq t} V_\tau - V_t}{\max_{\tau \leq t} V_\tau} \right) \tag{3.22}$$

where V_t is the cumulative portfolio value at time t.

Volatility reduction. The percentage reduction in annualised volatility relative to an equal-weight buy-and-hold benchmark of the constituent stocks:

$$\mathrm{VR} = 1 - \frac{\sigma_{\text{strategy}}}{\sigma_{\text{benchmark}}} \tag{3.23}$$

Additional metrics. Win rate (fraction of active trading days with positive returns), profit factor (ratio of gross profits to gross losses), number of round-trip trades, and total return are also reported.

Benchmarks. Three benchmarks are used for comparison: (i) an equal-weight buy-and-hold portfolio of all universe constituents, (ii) individual buy-and-hold of each pair leg, and (iii) the S&P 500 index as a broad market proxy.

3.6.3 Statistical Robustness Tests

Four additional tests are applied to the OOS results to assess statistical robustness:

Bootstrap confidence intervals. Given the short OOS window ($T \approx 501$ trading days), point Sharpe ratio estimates carry high sampling variance. Bootstrap confidence intervals are constructed by resampling daily returns with replacement 10,000 times, providing empirical 95% CIs [40] for each strategy's Sharpe ratio.

Paired significance test. A paired t-test and non-parametric Wilcoxon signed-rank test are applied to the daily return difference between OU Max-Sharpe and Historical Max-Sharpe. This directly tests whether the observed Sharpe advantage of the OU-Implied approach is statistically significant or within normal sampling variation.

Sensitivity analysis. The per-pair backtest is re-run across a grid of entry z-scores $\{1.5, 2.0, 2.5, 3.0\}$ and transaction costs $\{0, 5, 10, 20, 30\}$ bps, with exit fixed at $z_{\text{exit}} = 0.0$ and stop-loss at $z_{\text{entry}} + 2.0$. This assesses whether conclusions are robust to parameter choice or depend critically on the default configuration.

Leave-one-pair-out (LOPO). Each pair is removed in turn, the OU Max-Sharpe portfolio is re-optimised on the remaining spreads using IS data, and OOS Sharpe is recomputed. This tests whether the portfolio result is dominated by a single spread or reflects genuine multi-pair diversification.

3.7 Summary

This chapter has presented the complete methodology for the project. The pipeline proceeds from data sourcing and cointegration screening, through spread modelling and OU-implied return estimation, to Markowitz portfolio optimisation and rigorous out-of-sample backtesting. The key methodological contribution is the use of OU-implied expected returns derived from cointegrated spreads as inputs to the mean-variance optimiser, compared against a traditional historical mean baseline. Chapter 4 describes how this methodology is implemented in software.

Chapter 4

Implementation

This chapter describes the software implementation of the methodology presented in Chapter 3. The system is implemented in Python and organised into a modular pipeline of reusable library code, analysis notebooks, a backtesting engine, and an interactive dashboard. All source code is available at the project repository.

4.1 Pipeline Overview

The project follows a four-stage notebook pipeline, each building on the outputs of the previous stage, supported by three library packages (`src/modelling`, `src/backtesting`, `src/data`) and an interactive dashboard (`src/dashboard`). Figure 4.1 illustrates the data flow.

1. **Spread Exploration** (`01_spread_exploration.ipynb`) — data ingestion, spread construction, and visual characterisation.
2. **Cointegration Validation** (`02_cointegration_validation.ipynb`) — unit root testing, Engle–Granger screening, and out-of-sample stability checks.
3. **Return Estimation** (`03_return_estimation.ipynb`) — OU-implied and historical mean return estimation, covariance estimation, and comparison.
4. **Backtest Results** (`04_backtest_results.ipynb`) — portfolio optimisation, pair-level backtesting, portfolio-level evaluation, and benchmark comparison.

The library packages expose a clean public API through their `__init__.py` modules, allowing notebooks to import functions such as `screen_pairs()`, `compute_half_life()`, and `maximum_sharpe_weights()` without knowledge of internal file structure. This separation of concerns ensures that analytical logic is unit-testable independently of the notebook execution context.

Each notebook produces two classes of visualisation. Static publication-quality figures are generated using `matplotlib` [41] with the `SciencePlots` [42] style package configured for IEEE and LaTeX rendering, ensuring consistent typography and line weights suitable for the report. Interactive exploratory charts are produced using `Plotly` [43], providing zoom, hover tooltips, and export capabilities during analysis; these are also reused directly within the Streamlit dashboard.

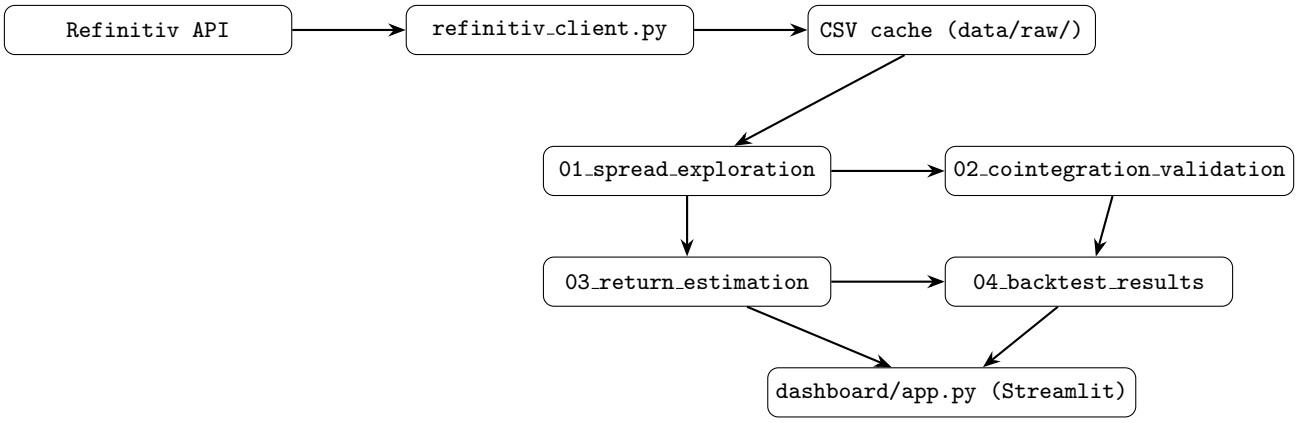

Figure 4.1: Data flow through the four-notebook pipeline and dashboard.

4.1.1 Data Ingestion and Caching

Price data are retrieved from the LSEG Refinitiv Workspace API via a custom client (`src/data/refinitiv_client.py`). The client implements an incremental caching strategy: on first request, the full date range is fetched and persisted as a CSV file in `data/raw/`. Subsequent requests check the cache and only fetch the date gaps not yet covered, merging new data with the existing cache. Cache keys are derived from a hash of the ticker list, interval, and field set, ensuring deterministic file naming regardless of request order. This design minimises API calls during iterative development while guaranteeing reproducibility.

The client also handles column normalisation (mapping LSEG field codes such as `TRDPRC_1` to `Close`), forward-filling of up to five consecutive missing values, and automatic dropping of columns with more than 10% missing data. All functions return tidy `pandas` [44] `Series` or `DataFrame` objects indexed by date, providing a consistent interface for downstream notebooks and library modules. S&P 500 benchmark prices are sourced separately via the `yfinance` library.

4.2 Spread Exploration

The first notebook (`01_spread_exploration`) loads in-sample close prices for all 26 universe constituents and constructs the 16 candidate spreads using the OLS hedge ratios. For each pair, it produces:

- Normalised price overlays to visualise co-movement.
- Spread level and distribution plots.
- Rolling half-life and Hurst exponent time series.
- Rolling z-score plots with entry/exit threshold annotations.

These are implemented in the `spread_analysis` module, which provides functions `compute_spread()`, `compute_half_life()`, `compute_hurst_exponent()`, and `compute_rolling_zscore()`. The half-life function fits the AR(1) regression (Equation 3.6) using ordinary least squares and returns the half-life in trading days. The Hurst exponent function computes the variogram across lags up to `max_lag` (default: 100) and estimates H via log-log regression on $\log(\tau_\ell)$ against $\log(\ell)$. Diagnostics are aggregated in the notebook into a summary DataFrame for tabular comparison across pairs.

4.3 Cointegration Validation

The second notebook (`02_cointegration_validation`) applies the full Engle–Granger screening pipeline to all 16 candidate pairs on the in-sample data, then assesses out-of-sample stability.

4.3.1 Screening Implementation

The `cointegration` module implements the screening procedure as follows:

- `adf_test(series)` applies the Augmented Dickey–Fuller test using the `statsmodels` implementation with automatic AIC-based lag selection [45]. It returns the test statistic, p-value, number of lags, and critical values.
- `engle_granger_test(y, x)` performs the two-step procedure: it fits the OLS cointegrating regression, extracts residuals, and applies the ADF test to the residuals. Both orderings (y on x and x on y) are tested, and the ordering with the lower residual p-value is retained. The function returns the hedge ratio $\hat{\beta}$, intercept $\hat{\alpha}$, ADF statistic, p-value, and the selected ordering.
- `screen_pairs(prices, pairs)` iterates over all candidate pairs, verifies that both legs are $I(1)$, runs the Engle–Granger test, and returns a DataFrame of results with a boolean `cointegrated` column (true if $p < 0.05$).

4.3.2 Out-of-Sample Stability

To assess whether in-sample cointegration relationships persist, the notebook re-estimates the cointegrating regression on the OOS period and compares hedge ratios, ADF statistics, and spread dynamics. Rolling half-life plots spanning both periods provide visual confirmation of parameter stability. This validation step informs which pairs are included in the final portfolio.

Two additional formal tests are also implemented. The **KPSS test** (`statsmodels.tsa.stattools.kpss`) complements the ADF test by reversing the null hypothesis: stationarity is accepted only when ADF rejects non-stationarity *and* KPSS fails to reject stationarity. This dual-consensus approach reduces the risk of false positives from either test alone. The **Chow structural break test** formally tests whether the cointegrating regression coefficients $(\hat{\beta}, \hat{\alpha})$ remain stable across the IS/OOS split by comparing the pooled residual sum of squares against the sum from separate IS and OOS regressions, yielding an F-statistic under H_0: no structural break.

4.4 Return Estimation

The third notebook (`03_return_estimation`) computes expected returns under both the OU-implied and historical mean approaches.

The `return_estimation` module provides:

- `compute_spread_returns(y_prices, x_prices, hedge_ratio)` — computes daily log spread returns as $\ln(Y_t/Y_{t-1}) - \beta \cdot \ln(X_t/X_{t-1})$.
- `build_spread_return_matrix(prices, coint_pairs)` — constructs the full $T \times n$ matrix of spread returns for all cointegrated pairs.

- `ou_implied_spread_return(spread, half_life)` — implements Equation 3.12, computing $\kappa = \ln 2/h$ and the conditional expected spread change.
- `build_ou_implied_returns(prices, coint_pairs)` — applies the OU-implied estimator to all pairs and returns an annualised expected return vector.
- `historical_mean_return(returns)` — computes the sample mean, optionally over a trailing window.
- `shrinkage_covariance(returns)` — estimates the covariance matrix using the Ledoit–Wolf shrinkage estimator from `scikit-learn` [46], which shrinks the sample covariance towards a scaled identity target.

The notebook produces a grouped bar chart comparing OU-implied, historical mean, and EWMA return estimates across pairs, along with a correlation heatmap of spread returns to visualise diversification potential.

4.5 Portfolio Optimisation Module

The `optimiser` module implements the Markowitz mean-variance framework in spread space.

- `minimum_variance_weights(returns, cov_matrix, l2_reg)` — solves the minimum-variance problem (Equation 3.17) using Sequential Least Squares Programming (SLSQP) from `scipy.optimize` [47]. The long-only constraint ($w_i \geq 0$) and full-investment constraint ($\sum w_i = 1$) are enforced explicitly. The optional L_2 regularisation term `l2_reg`$\cdot\|\mathbf{w}\|_2^2$ is added to the objective to improve numerical stability and reduce weight concentration.
- `maximum_sharpe_weights(returns, expected_returns, cov_matrix, rf_annual)` — solves the maximum Sharpe ratio problem (Equation 3.18) by minimising the negative Sharpe ratio subject to the same long-only and full-investment constraints. Custom expected returns (e.g. OU-implied) can be passed via `expected_returns`; if omitted, the historical sample mean is used.
- `compute_efficient_frontier(returns, expected_returns, cov_matrix, n_points, rf_annual)` — generates the efficient frontier by solving the minimum-variance problem for a range of target returns, anchored between the global minimum-variance portfolio return and the maximum single-asset expected return.

Return and volatility figures are annualised from daily values. Log-return expected returns are converted to annualised arithmetic returns via $\mu_{\text{ann}} = \text{expm1}(252 \cdot \mu_{\text{daily}})$, i.e. $\exp(252\,\mu_{\text{daily}}) - 1$, which correctly compounds the daily log return. Volatilities are scaled as $\sigma_{\text{ann}} = \sigma_{\text{daily}} \cdot \sqrt{252}$. The optimiser accepts either OU-implied or historical expected returns, enabling direct comparison of the resulting allocations.

4.6 Backtesting Engine

The backtesting engine (`src/backtesting/engine.py`) simulates pair-level trading and computes performance metrics.

4.6.1 Architecture

The engine uses a dataclass-based configuration pattern:

- `BacktestConfig` stores all strategy parameters (entry, exit, and stop-loss z-score thresholds, lookback window, transaction cost rate, and initial capital) with validation in `__post_init__` to enforce $z_{\text{exit}} < z_{\text{entry}} < z_{\text{stop}}$.
- `BacktestResult` is a container holding daily returns, cumulative returns, position history, trade log, summary metrics, spread, and z-score series.
- `PairsBacktestEngine` orchestrates the full pipeline: spread and z-score computation, signal generation, PnL calculation, trade logging, and metric aggregation.

4.6.2 Signal Generation

The signal generator implements the z-score state machine described in Section 3.6. It iterates sequentially through the z-score series, maintaining a position state variable that transitions between $\{-1, 0, +1\}$ based on threshold crossings. The sequential iteration is necessary because the position on day t depends on the position on day $t-1$, precluding vectorised computation.

4.6.3 PnL and Transaction Costs

The PnL computation deploys the full initial capital into each active position. On each day, the number of spread units is recalculated as $n_t = C_0/(Y_t + |\beta| X_t)$ to reflect changing notional values. The dollar PnL is $\text{pos}_{t-1} \cdot n_{t-1} \cdot (\Delta Y_t - \beta \cdot \Delta X_t)$, from which transaction costs (proportional to the absolute position change) are deducted.

4.6.4 Metrics and Benchmarks

The `metrics` module provides three core functions: `compute_ex_post_sharpe_ratio()`, `compute_max_drawdown()`, and `compute_volatility_reduction()`, each implemented as described in Section 3.6. The `benchmarks` module constructs comparison return series: a constant risk-free rate, equal-weight buy-and-hold of all universe constituents, per-pair buy-and-hold of both legs, and the S&P 500 index. A `historical_mpt_returns()` function computes the out-of-sample returns of a traditional max-Sharpe portfolio estimated on in-sample asset returns, serving as the primary baseline for comparison.

Four additional robustness procedures are implemented in `04_backtest_results.ipynb`:

- **Bootstrap confidence intervals**: 10,000 bootstrap resamples of daily returns are used to construct empirical 95% confidence intervals for each strategy's annualised Sharpe ratio. The implementation uses vectorised NumPy resampling for computational efficiency.
- **Paired significance tests**: a paired t-test and Wilcoxon signed-rank test compare daily returns of OU Max-Sharpe and Historical Max-Sharpe portfolios, directly assessing whether the observed Sharpe difference is statistically significant.
- **Sensitivity analysis**: the per-pair backtest is re-run across a grid of entry z-score thresholds $\{1.5, 2.0, 2.5, 3.0\}$ and transaction costs $\{0, 5, 10, 20, 30\}$ bps, assessing robustness to parameter choice.

- **Leave-one-pair-out robustness**: each pair is removed in turn, the OU Max-Sharpe portfolio is re-optimised on the remaining spreads, and OOS Sharpe is recomputed to test whether the portfolio result is driven by a single spread.

4.7 Dashboard Interface

An interactive Streamlit dashboard (`src/dashboard/app.py`) provides a user-facing interface for exploring the full pipeline [48]. The dashboard is organised into six tabs:

1. **Cointegration** — horizontal bar chart of Engle–Granger p-values for all candidate pairs, with a 5% significance threshold line.
2. **Spread Analysis** — spread level with $\pm 1\sigma/\pm 2\sigma$ bands, rolling z-score with entry/exit/stop-loss threshold annotations, rolling half-life and Hurst exponent plots, and position timelines.
3. **Return Estimation** — grouped bar chart comparing OU-implied, historical mean, and EWMA return estimates across pairs; correlation heatmap; and rolling return estimate stability plots.
4. **Backtesting** — per-pair z-score backtest results including cumulative return, drawdown, rolling Sharpe, and metrics summary.
5. **Portfolio Comparison** — multi-strategy cumulative return chart, benchmark comparison table, drawdown comparison, and bootstrap confidence intervals for Sharpe ratio.
6. **Portfolio Optimisation** — efficient frontier visualisation with annotated min-variance and max-Sharpe portfolios for both spread-space and asset-space approaches, alongside portfolio weight bar charts.

The visualisation layer is implemented in a separate `components.py` module using Plotly [43], providing interactive zoom, hover tooltips, and export capabilities. Chart functions are stateless and accept data as arguments, ensuring reusability across both notebooks and the dashboard. Bootstrap confidence intervals are computed within the dashboard using vectorised NumPy resampling [49], allowing 10,000 resamples to be evaluated interactively without an explicit Python loop.

4.8 Summary

This chapter has described the implementation of the pairs-trading portfolio optimisation system. The codebase is organised into a four-notebook analysis pipeline supported by three library packages (`modelling`, `backtesting`, `data`) and an interactive Streamlit dashboard. Key design decisions include incremental data caching for API efficiency, a clean modular API for testability, dataclass-based configuration for the backtesting engine, and separation of visualisation logic into reusable Plotly chart components. Chapter 5 presents the empirical results produced by this implementation.

Chapter 5

Testing and Results

5.1 Cointegration Screening

Table 5.1: Engle-Granger Cointegration Screening Results (16 candidate pairs, in-sample: Jan 2019 – Dec 2023)

Dependent	Independent	Hedge Ratio ($\hat{\beta}$)	Intercept ($\hat{\alpha}$)	ADF Stat	p-value	y is $I(1)$	x is $I(1)$	Cointegrated
GS.N	MS.N	0.780	2.351	−3.667	0.020	Yes	Yes	**Yes**
KO.N	PEP.O	0.641	0.789	−3.470	0.035	Yes	Yes	**Yes**
DAL.N	UAL.O	0.672	1.068	−3.460	0.036	Yes	Yes	**Yes**
CVX.N	XOM.N	0.698	1.813	−3.234	0.064	Yes	Yes	No
AMZN.O	GOOGL.O	0.630	1.942	−2.442	0.305	Yes	Yes	No
HAL.N	SLB.N	0.997	−0.344	−2.370	0.339	Yes	Yes	No
AMD.O	NVDA.O	0.811	1.999	−2.310	0.368	Yes	Yes	No
AMZN.O	MSFT.O	0.516	2.027	−2.141	0.455	Yes	Yes	No
MRK.N	ABBV.N	0.431	2.366	−1.980	0.539	Yes	Yes	No
INTC.O	AMD.O	−0.081	4.154	−1.958	0.550	Yes	Yes	No
JNJ.N	PFE.N	0.423	3.466	−1.908	0.576	Yes	Yes	No
INTC.O	TSM.N	−0.087	4.201	−1.890	0.585	Yes	Yes	No
META.O	GOOGL.O	0.543	2.953	−1.681	0.685	Yes	Yes	No
TSM.N	NVDA.O	0.476	3.071	−1.340	0.817	Yes	Yes	No
BAC.N	JPM.N	0.854	−0.666	−1.230	0.850	Yes	Yes	No
TGT.N	COST.O	0.856	−0.125	−0.869	0.925	Yes	Yes	No

Dep./Indep.: Dependent/Independent variable in the cointegrating regression $Y_t = \hat{\alpha} + \hat{\beta} X_t + \epsilon_t$. $\hat{\beta}$: hedge ratio; $\hat{\alpha}$: intercept. ADF Stat and p-val: residual ADF test statistic and MacKinnon (2010) p-value. y:$I(1)$, x:$I(1)$: whether each leg passes the unit root filter. Coint.: cointegrated at 5% significance ($p < 0.05$). Source: Author's calculations on `02_cointegration_validation.ipynb`.

5.2 Spread Characterisation

Based on the results in Table 5.1, three pairs were confirmed cointegrated at the 5% significance level: GS.N/MS.N ($p = 0.020$), KO.N/PEP.O ($p = 0.035$), and DAL.N/UAL.O ($p = 0.036$). CVX.N/XOM.N was borderline ($p = 0.064$) and excluded. Spreads for each confirmed pair are computed as the OLS residual $S_t = \ln P_t^y - \hat{\beta} \ln P_t^x - \hat{\alpha}$ and characterised by their half-life, Hurst exponent, z-score, and spread statistics.

Table 5.2 and Figures 5.1 present the spread characterisation results for the three cointegrated pairs.

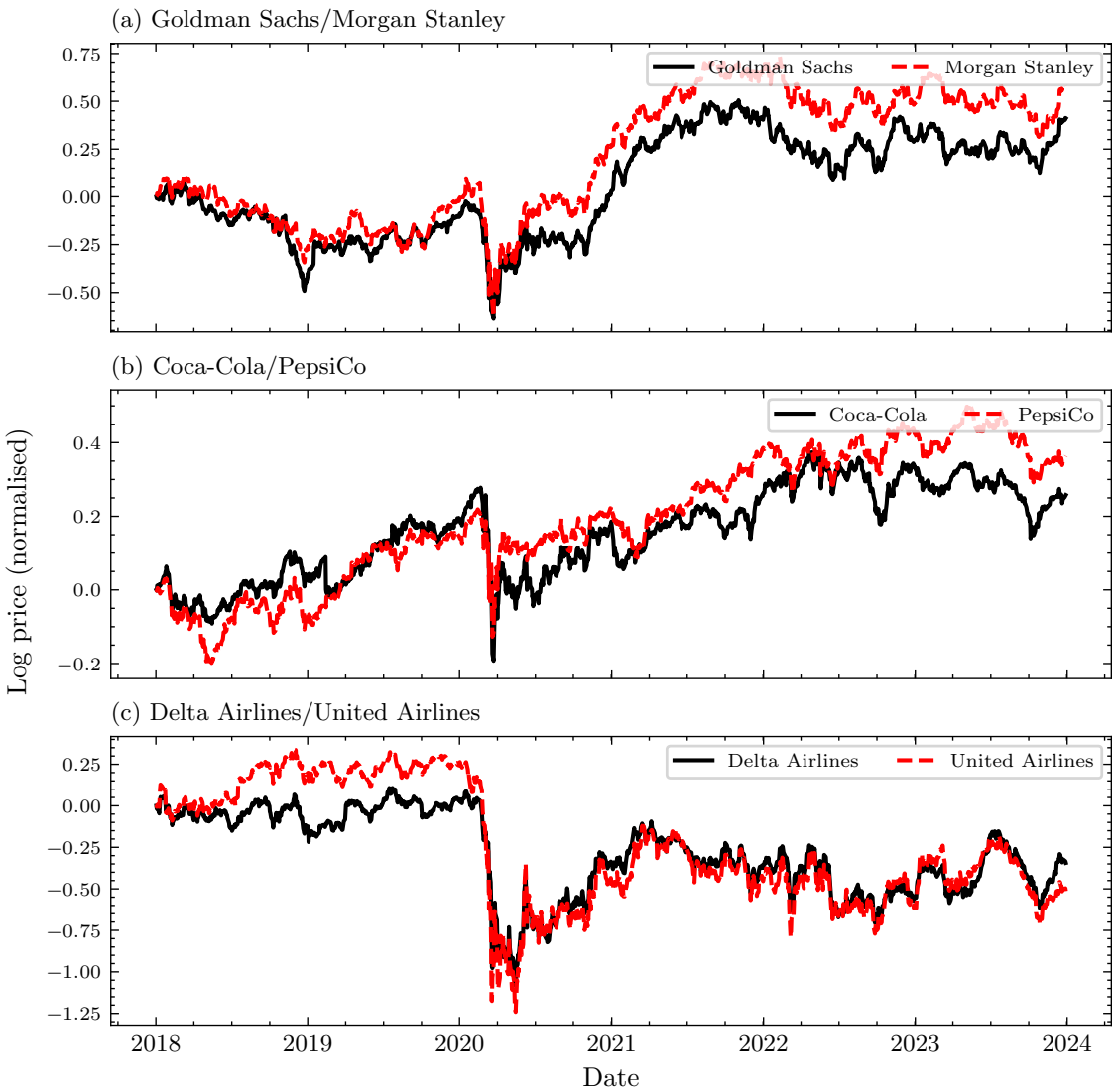

Figure 5.1: Normalised log price series for the three cointegrated pairs over the in-sample period (January 2019 – December 2023). Each series is normalised to zero at the start of the sample period to highlight co-movement. (a) GS/MS (Financials); (b) KO/PEP (Consumer Staples);(c) DAL/UAL (Airlines).

Table 5.2: Spread Characterisation Summary — Cointegrated Pairs (In-Sample: Jan 2019 – Dec 2023)

Dep.	Indep.	Half-Life (days)	Hurst Exp.	z-Score (current)	μ_S	σ_S	ADF Stat	ADF p
GS.N	MS.N	41.8	0.403	0.675	0.000	0.056	−3.666	0.005
KO.N	PEP.O	40.3	0.421	0.809	0.000	0.047	−3.469	0.009
DAL.N	UAL.O	52.9	0.414	1.315	0.000	0.076	−3.459	0.009

Dep.: Dependent leg; Indep.: Independent leg of the cointegrating regression. μ_S: spread mean; σ_S: spread standard deviation; ADF Stat/ADF p: residual ADF test statistic and MacKinnon (2010) p-value on the spread.
Half-life estimated via AR(1)/OU regression: $\kappa = \ln(2)/\text{half-life}$ gives $\kappa \approx 0.0166$ (GS/MS), 0.0172 (KO/PEP), and 0.0131 (DAL/UAL).
Hurst exponent $H < 0.5$ indicates mean-reverting (anti-persistent) dynamics. z-score uses 60-day rolling window at end of training period.
Spread mean $\mu_S = 0$ by construction (demeaned OLS residuals). Source: Author's calculations on `01_spread_exploration.ipynb`.

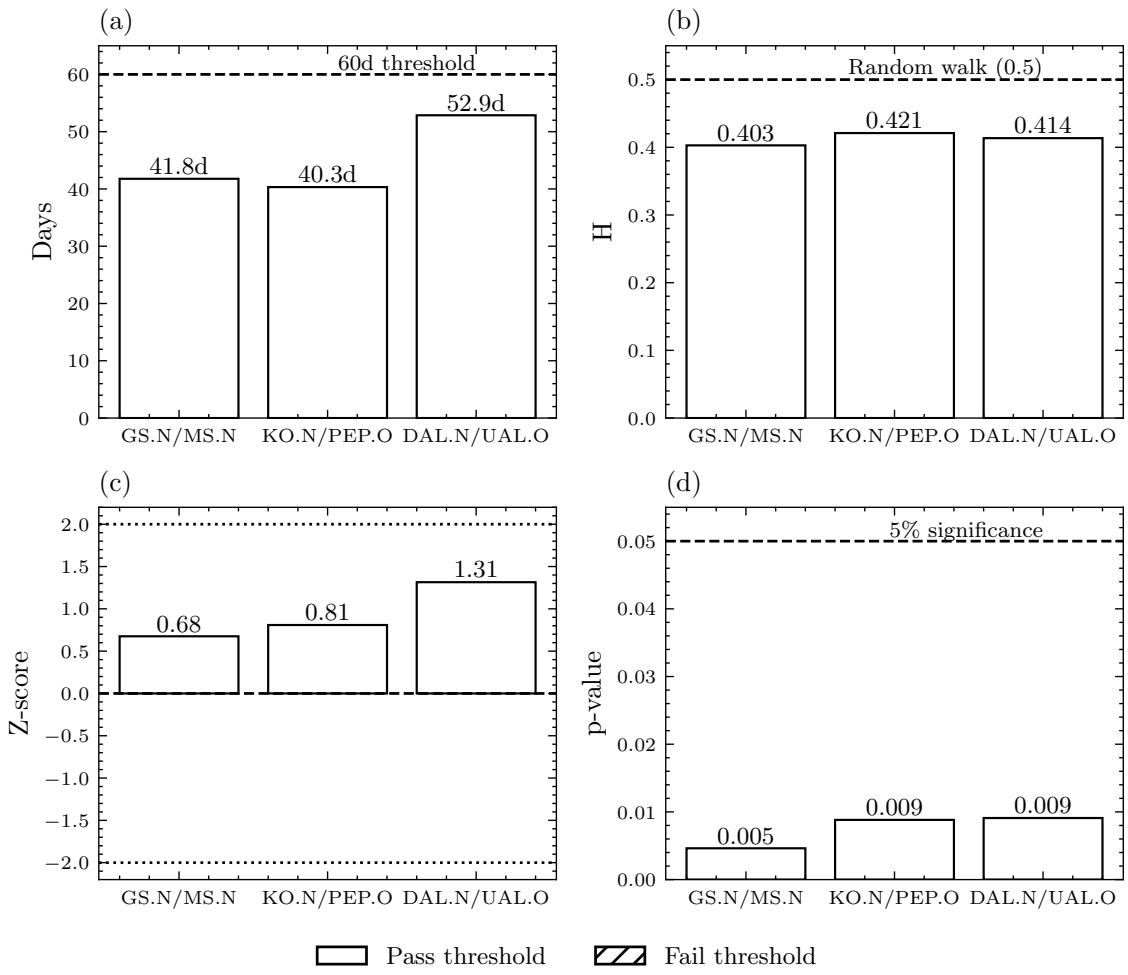

Figure 5.2: Spread characterisation summary for the three cointegrated pairs (January 2019 – December 2023). (a) Half-life of mean reversion in trading days; dashed line marks the 60-day threshold. (b) Hurst exponent; dashed line marks the random walk boundary of 0.5. (c) Current z-score at the end of the training window; dotted lines mark $\pm 2\sigma$. (d) ADF p-value on the spread; dashed line marks the 5% significance level. Hatched bars indicate pairs that fail the respective threshold.

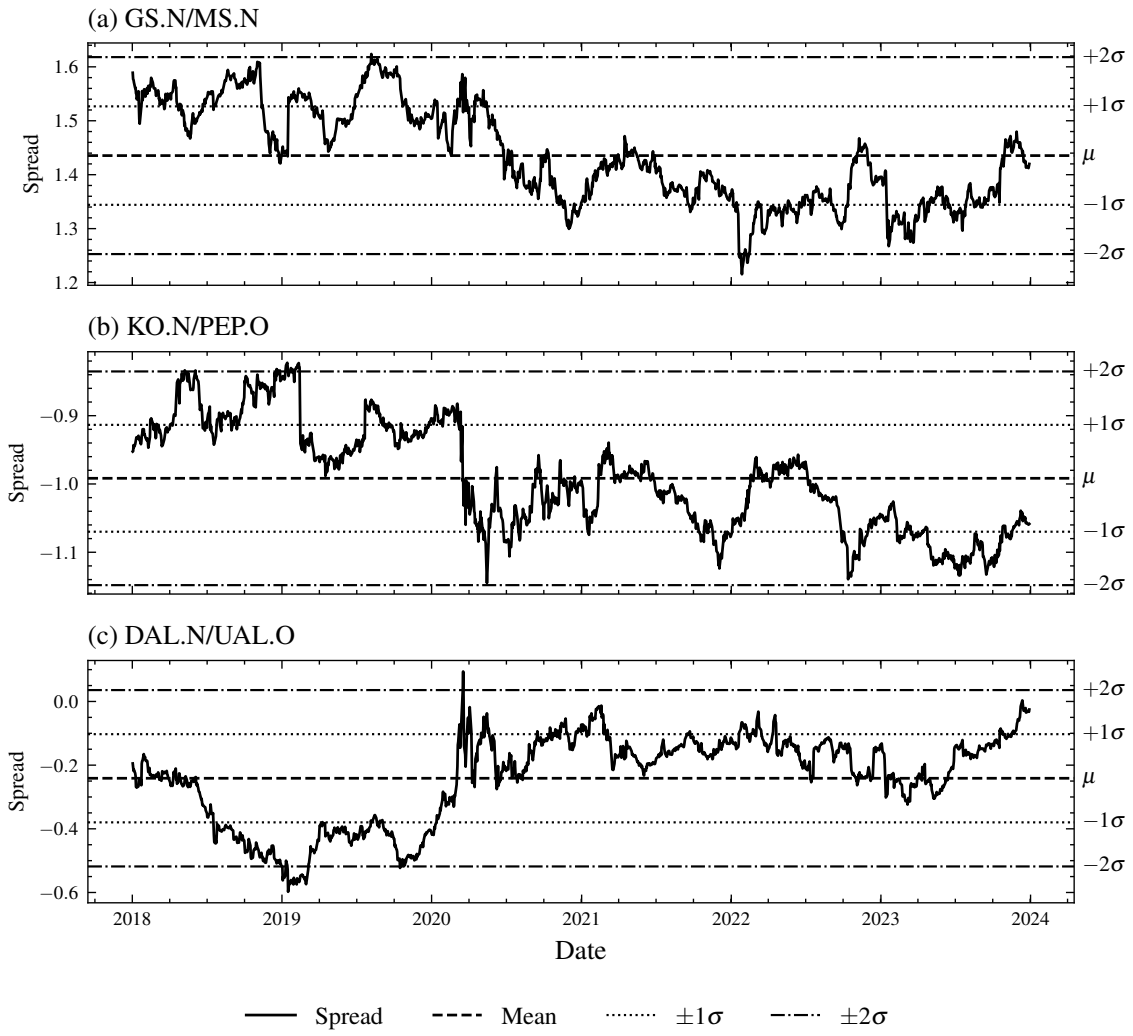

Figure 5.3: Spread levels with $\pm 1\sigma$ and $\pm 2\sigma$ bands for each cointegrated pair over the in-sample period (January 2019 – December 2023). The spread is defined as $S_t = \ln P_t^y - \hat{\beta} \ln P_t^x - \hat{\alpha}$. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

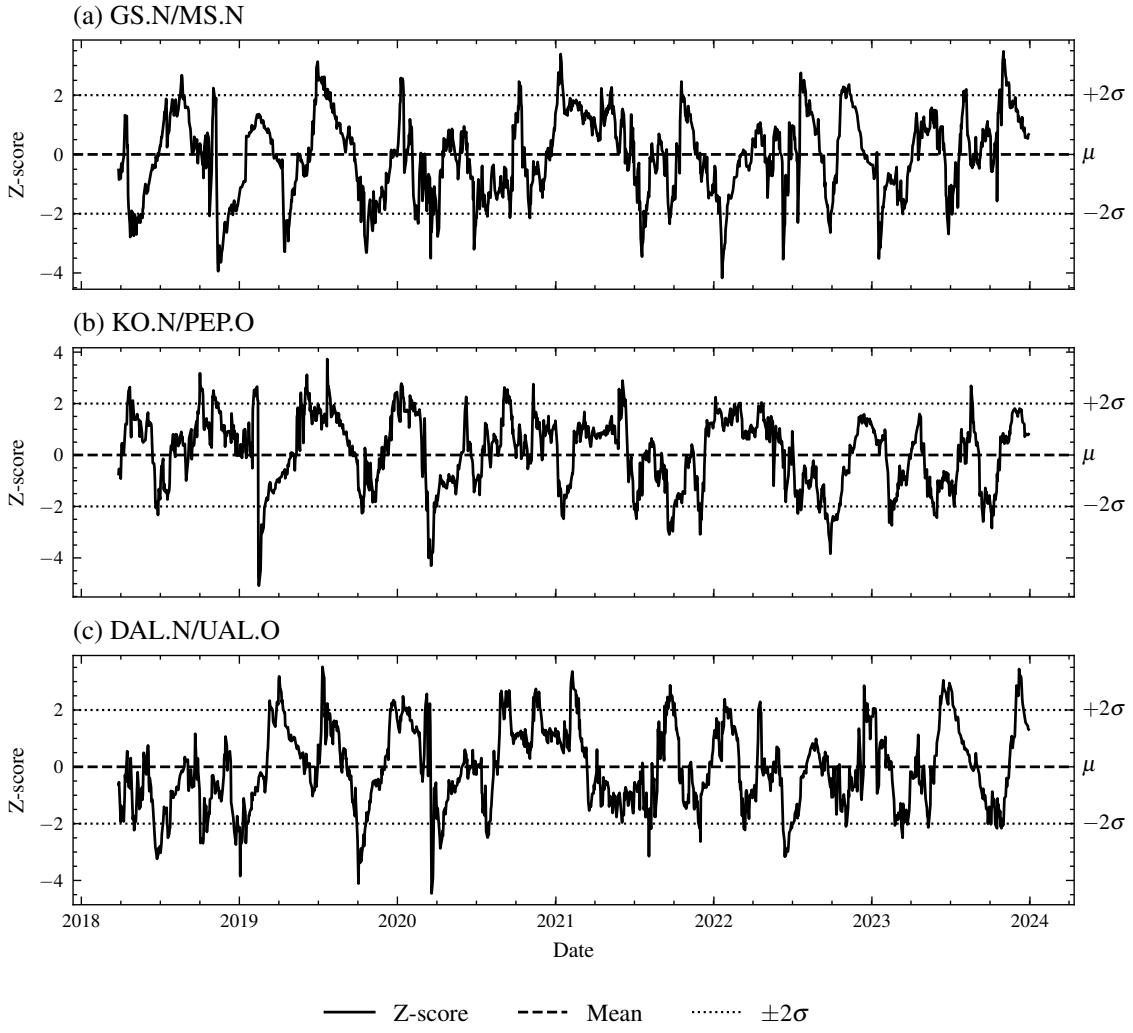

Figure 5.4: Rolling z-scores (60-day window) for the three cointegrated pairs over the in-sample period (January 2019 – December 2023). Dotted lines mark the $\pm 2\sigma$ entry thresholds. The z-score at the end of the training window feeds directly into the OU-implied return estimation. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

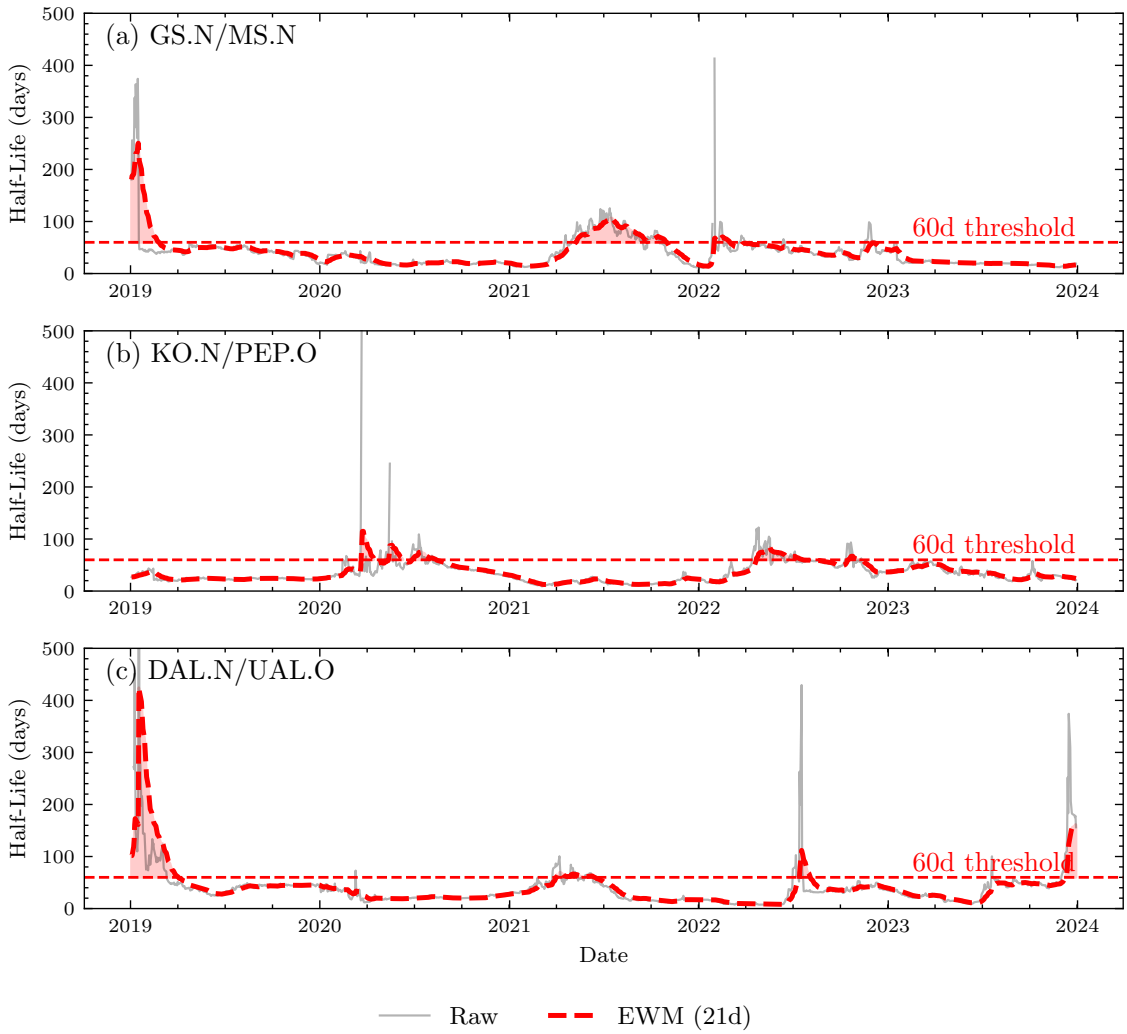

Figure 5.5: Rolling half-life of mean reversion (252-day window, EWM 21-day smoothed). Dashed line denotes the 60-day tradeability threshold. Shaded regions indicate periods where the smoothed half-life exceeded the threshold. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

5.3 Cointegration Validation

Out-of-sample spreads are constructed using hedge ratios and intercepts estimated solely on the in-sample period, ensuring no forward-looking information is incorporated. Table 5.3 presents the distributional shift in each spread across the IS and OOS periods, while Table 5.4 reports the formal statistical test results. Figure 5.6 presents the full IS vs OOS Engle-Granger p-value comparison across all nine candidate pairs.

Table 5.3: Spread statistics across in-sample (IS: January 2019 – December 2023) and out-of-sample (OOS: January 2024 – December 2025) periods, constructed using in-sample hedge ratios.

	Mean		Std Dev		Min		Max	
Pair	IS	OOS	IS	OOS	IS	OOS	IS	OOS
GS/MS	0.000	+0.250	0.056	0.082	−0.145	+0.067	+0.110	+0.411
KO/PEP	0.000	+0.170	0.047	0.110	−0.173	−0.016	+0.111	+0.367
DAL/UAL	0.000	+0.026	0.076	0.119	−0.201	−0.199	+0.175	+0.280

The GS/MS OOS minimum (+0.067) exceeds the IS mean (0.000), indicating the spread never reverted to its historical equilibrium during the OOS period.
Source: Author's calculations on `02_cointegration_validation.ipynb`.

Table 5.4: Cointegration validation statistics across IS (January 2019 – December 2023) and OOS (January 2024 – December 2025) periods. ADF p-values below 0.05 indicate spread stationarity at the 5% significance level. Half-life below 60 trading days and Hurst exponent below 0.5 indicate mean reversion.

	ADF p-value		Half-Life (days)		Hurst Exponent		OOS
Pair	IS	OOS	IS	OOS	IS	OOS	z-score
GS/MS	0.005	0.452	41.8	81.1	0.403	0.362	1.015
KO/PEP	0.009	0.622	40.3	156.4	0.421	0.575	0.604
DAL/UAL	0.009	0.583	52.9	103.1	0.414	0.421	0.872

Neither spread remains stationary OOS using IS hedge ratios. KO/PEP Hurst crosses above 0.5, indicating transition to near-random-walk behaviour.
Source: Author's calculations on `02_cointegration_validation.ipynb`.

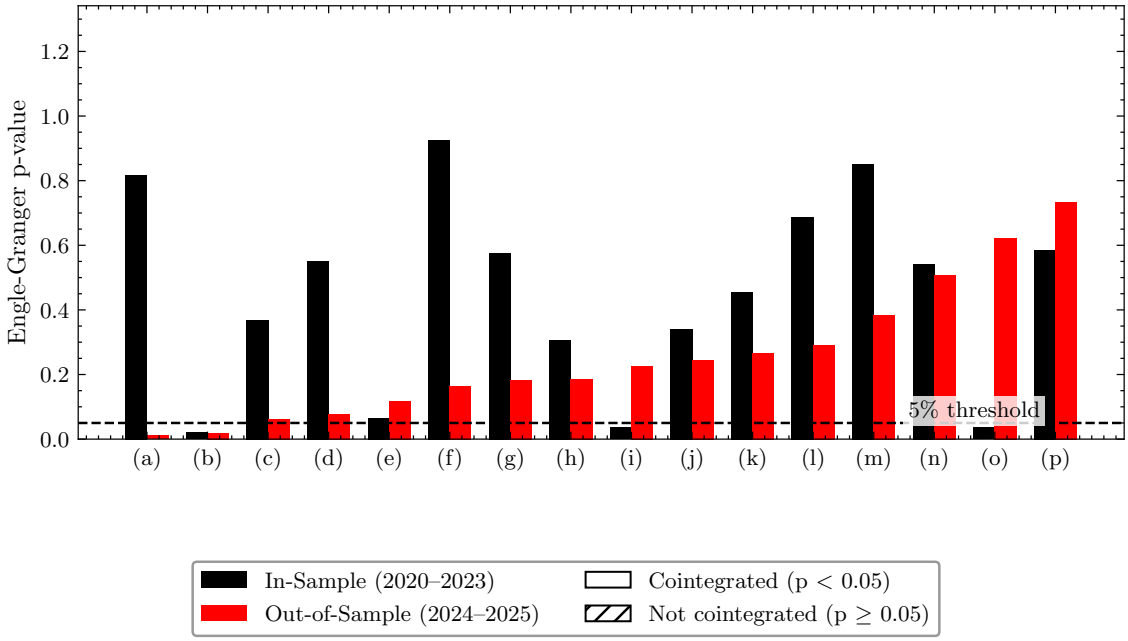

Figure 5.6: IS vs OOS Engle-Granger p-values for all 16 sector-candidate pairs. Panel labels (a)–(p) correspond to the pairs listed in Table 5.1. Hatched bars indicate pairs that fail the 5% significance threshold. Of the three pairs that pass IS screening, GS/MS and KO/PEP remain the most stable OOS; DAL/UAL degrades substantially. The remaining 13 pairs show no evidence of cointegration in either period.

Table 5.5: IS vs OOS Engle-Granger p-values and hedge ratio drift across all 16 candidate pairs. Sorted by IS p-value. Hedge ratio drift $= |\hat{\beta}_{\text{OOS}} - \hat{\beta}_{\text{IS}}|$.

	EG p-value		Hedge Ratio $\hat{\beta}$		Drift	Cointegrated	
Pair	IS	OOS	IS	OOS	$\lvert\Delta\hat{\beta}\rvert$	IS	OOS
GS.N/MS.N	**0.020**	**0.019**	0.780	1.120	0.340	**Yes**	**Yes**
KO.N/PEP.O	**0.035**	0.224	0.641	−0.350	0.991	**Yes**	No
DAL.N/UAL.O	**0.036**	0.621	0.672	0.429	0.242	**Yes**	No
CVX.N/XOM.N	0.064	0.118	0.698	0.470	0.228	No	No
AMZN.O/GOOGL.O	0.305	0.185	0.630	0.440	0.190	No	No
SLB.N/HAL.N	0.339	0.242	0.997	0.676	0.321	No	No
NVDA.O/AMD.O	0.368	0.060	0.811	0.293	0.518	No	No
AMZN.O/MSFT.O	0.455	0.265	0.516	0.897	0.382	No	No
MRK.N/ABBV.N	0.539	0.507	0.431	−0.318	0.749	No	No
INTC.O/AMD.O	0.550	0.076	−0.081	0.831	0.912	No	No
JNJ.N/PFE.N	0.576	0.182	0.423	−0.166	0.589	No	No
INTC.O/TSM.N	0.585	0.731	−0.087	−0.212	0.124	No	No
META.O/GOOGL.O	0.685	0.290	0.543	0.525	0.018	No	No
NVDA.O/TSM.N	0.817	0.012	0.476	1.120	0.644	No	No†
BAC.N/JPM.N	0.850	0.384	0.854	0.697	0.157	No	No
TGT.N/COST.O	0.925	0.162	0.856	−0.318	1.174	No	No

†NVDA/TSM: OOS p-value passes threshold but y fails $I(1)$ test — cointegration requires both series to be $I(1)$.
KO/PEP hedge ratio sign reversal ($+0.641 \rightarrow -0.350$) and DAL/UAL OOS p-value surge indicate structural breaks in the cointegrating relationship.
Source: Author's calculations on `02_cointegration_validation.ipynb`.

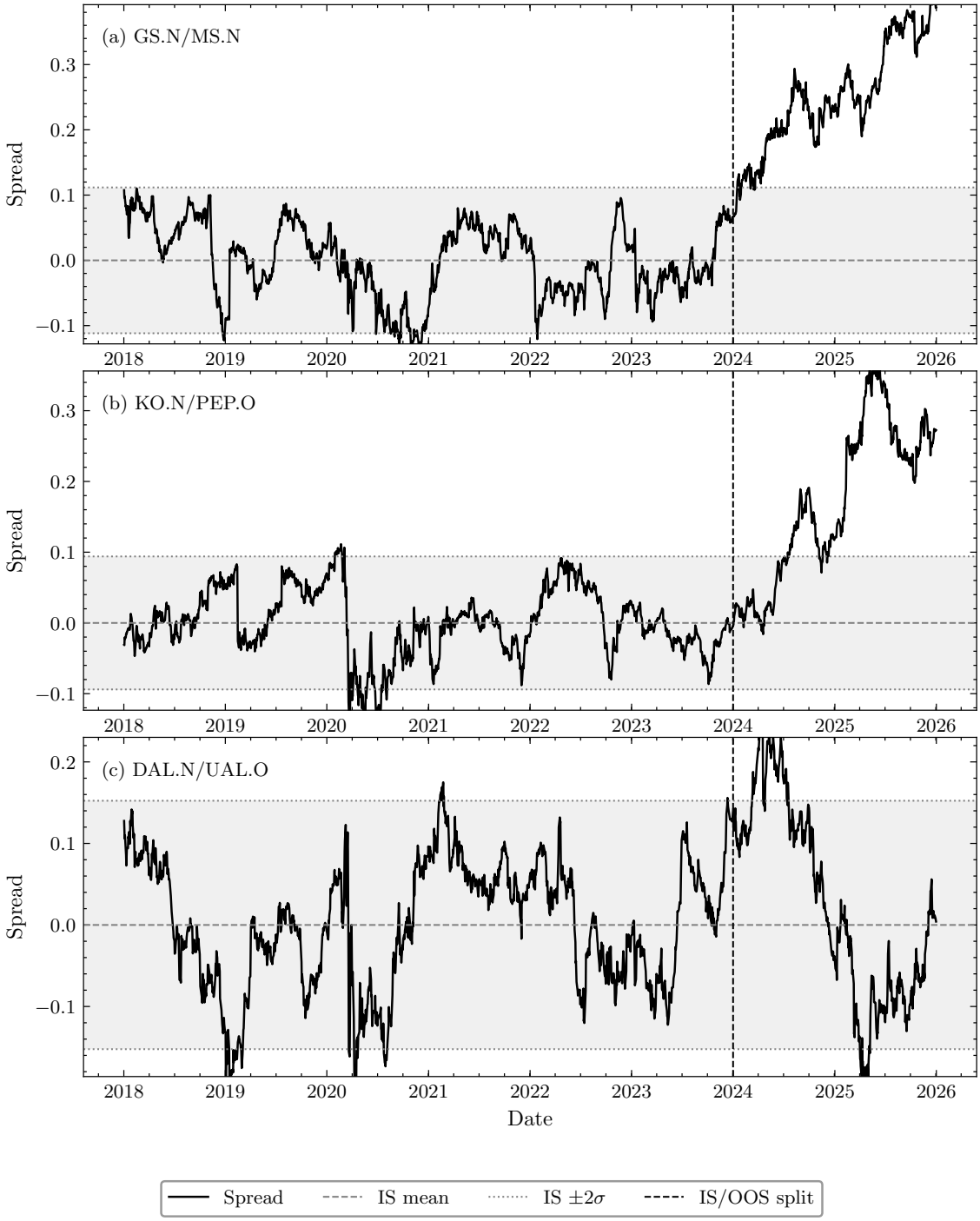

Figure 5.7: Spread levels across the full sample period (January 2019 – December 2025). The dashed vertical line marks the IS/OOS boundary (January 2024). Horizontal lines and shading denote IS $\pm 1\sigma$ and $\pm 2\sigma$ bands; persistent OOS displacement beyond these bounds indicates a structural break in the cointegrating relationship. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

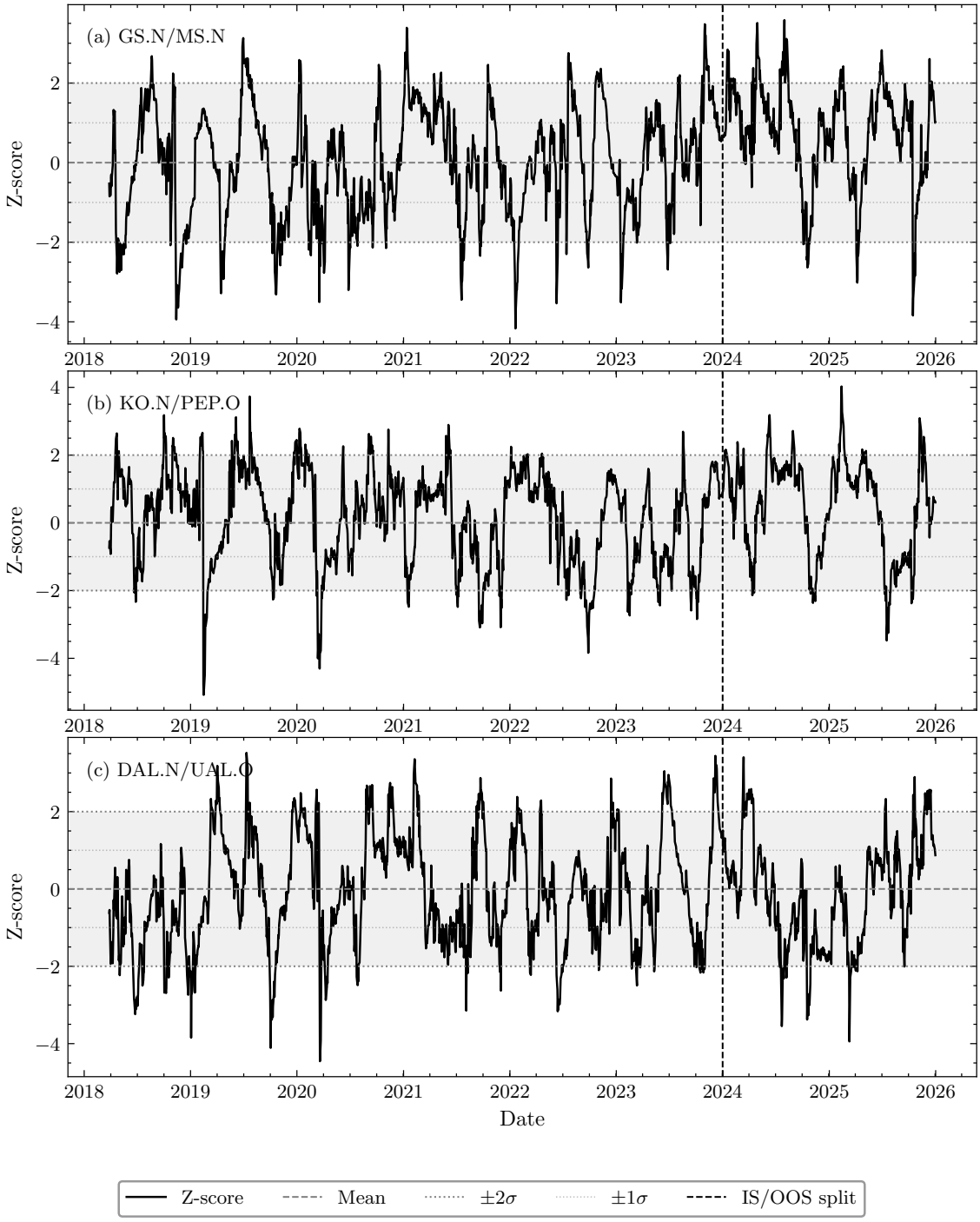

Figure 5.8: Rolling 60-day z-score across the full sample period (January 2019 – December 2025). The dashed vertical line marks the IS/OOS boundary. Shaded regions indicate $|z| > 2\sigma$ exceedances. Persistent non-zero z-scores in the OOS period are consistent with a shift in the spread equilibrium level. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

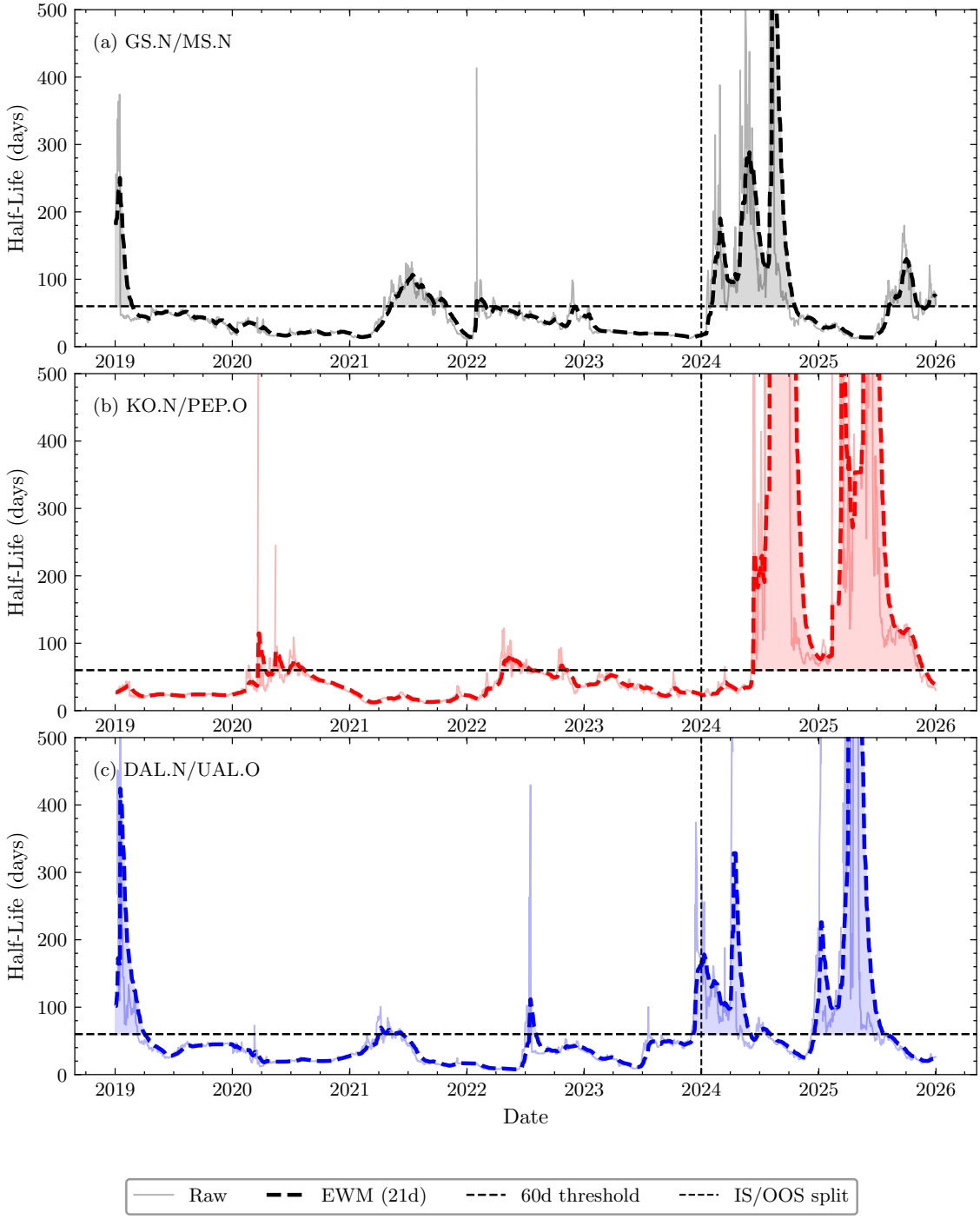

Figure 5.9: Rolling half-life of mean reversion across the full sample period (January 2019 – December 2025). Faint lines show raw estimates; bold lines show 21-day exponentially weighted smoothed values. The dashed horizontal line marks the 60-day tradability threshold. The dashed vertical line marks the IS/OOS boundary. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

5.3.1 KPSS Complement and Chow Structural Break Tests

Table 5.6 applies the KPSS test alongside ADF on both IS and OOS spreads. The KPSS null hypothesis is stationarity (the reverse of ADF), so mean reversion is only confirmed when ADF rejects non-stationarity *and* KPSS fails to reject stationarity.

Table 5.6: ADF and KPSS stationarity tests on IS and OOS spreads (using IS hedge ratios). Mean-reversion (MR) consensus requires ADF $p < 0.05$ and KPSS $p > 0.05$ simultaneously.

	In-Sample (2019–2023)			Out-of-Sample (2024–2025)		
Pair	ADF p	KPSS p	MR Consensus	ADF p	KPSS p	MR Consensus
GS.N/MS.N	0.005	0.019	No	0.452	0.010	No
KO.N/PEP.O	0.009	0.100	Yes	0.622	0.010	No
DAL.N/UAL.O	0.009	0.080	Yes	0.584	0.010	No

KPSS p-value of 0.010 denotes the tabulated lower bound (i.e. $p \leq 0.010$); null of stationarity is strongly rejected. IS consensus: KO/PEP and DAL/UAL achieve consensus; GS/MS fails KPSS despite low ADF p, indicating non-stationarity is borderline. OOS: All three pairs lose the MR consensus, consistent with Table 5.4. Source: Author's calculations on `02_cointegration_validation.ipynb`.

Table 5.7 presents the Chow test for structural break in the cointegrating regression across the IS/OOS split. The null hypothesis is that the regression coefficients $(\hat{\beta}, \hat{\alpha})$ are identical in both periods.

Table 5.7: Chow test for structural break in the cointegrating regression $y = \alpha + \beta x$ across the IS/OOS split (IS: 2019–2023; OOS: 2024–2025). H_0: no structural break (regression coefficients are equal in both periods).

Pair	F-statistic	p-value	df	Conclusion
GS.N/MS.N	2406.3	< 0.001	(2, 2007)	Structural break detected
KO.N/PEP.O	3078.4	< 0.001	(2, 2007)	Structural break detected
DAL.N/UAL.O	269.5	< 0.001	(2, 2007)	Structural break detected

The F-statistic compares the residual sum of squares of separate IS and OOS regressions against the pooled regression. All three pairs strongly reject the null at any conventional significance level, confirming that the cointegrating relationship is not stable over time. Source: Author's calculations on `02_cointegration_validation.ipynb`.

5.4 Return Estimation

OU-implied expected returns are derived from the end-of-IS z-score for each confirmed pair. A positive z-score (spread above mean) implies the spread is expected to fall, producing a negative OU-implied return; a negative z-score implies the spread is expected to rise, producing a positive return. Table 5.8 presents the sanity check confirming directional consistency between the z-score and OU-implied return signal.

Table 5.8: End-of-sample z-score validation and OU-implied expected returns (IS period: Jan 2019 – Dec 2023)

Pair	End-of-IS z-Score	OU-Implied μ (ann.)	Historical μ (ann.)	Signal	Consistent
GS/MS	+0.675	−9.54%	GS: +11.9%, MS: +15.5%	Spread above mean	Yes
KO/PEP	+0.809	−8.62%	KO: +6.4%, PEP: +8.3%	Spread above mean	Yes
DAL/UAL	+1.315	−23.76%	DAL: +4.5%, UAL: +7.0%	Spread well above mean	Yes

OU-implied μ computed via rolling 60-day OU model, annualised at 252 trading days per year. Historical μ is the annualised sample mean of daily returns over the IS period. Source: Author's calculations on `03_return_estimation.ipynb`.

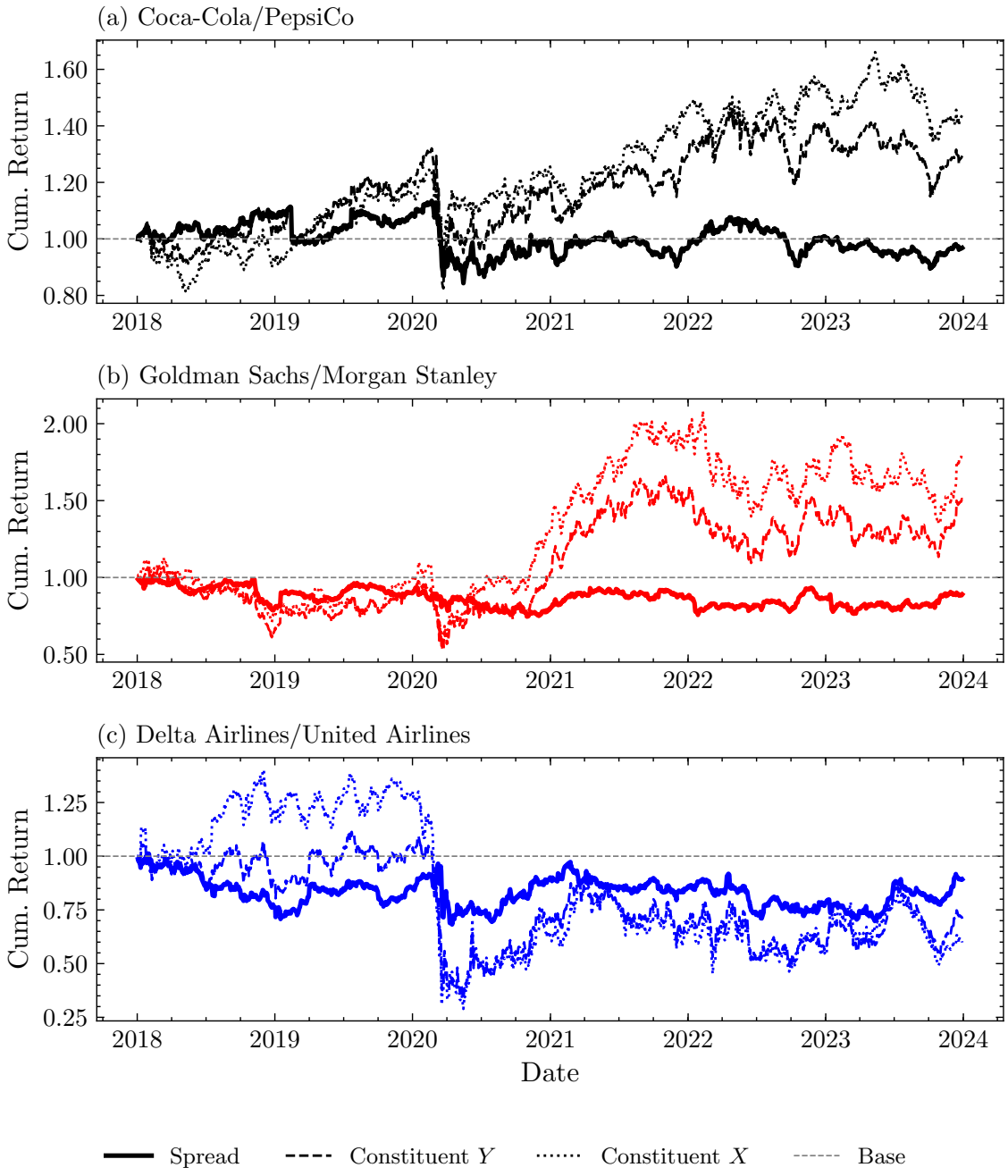

Figure 5.10: Cumulative returns of each cointegrated spread versus its constituent assets over the in-sample period (January 2019 – December 2023). All series are indexed to 1.0 at the start. The spread return reflects the daily change in the OLS residual $S_t = \ln P_t^y - \hat{\beta} \ln P_t^x - \hat{\alpha}$.

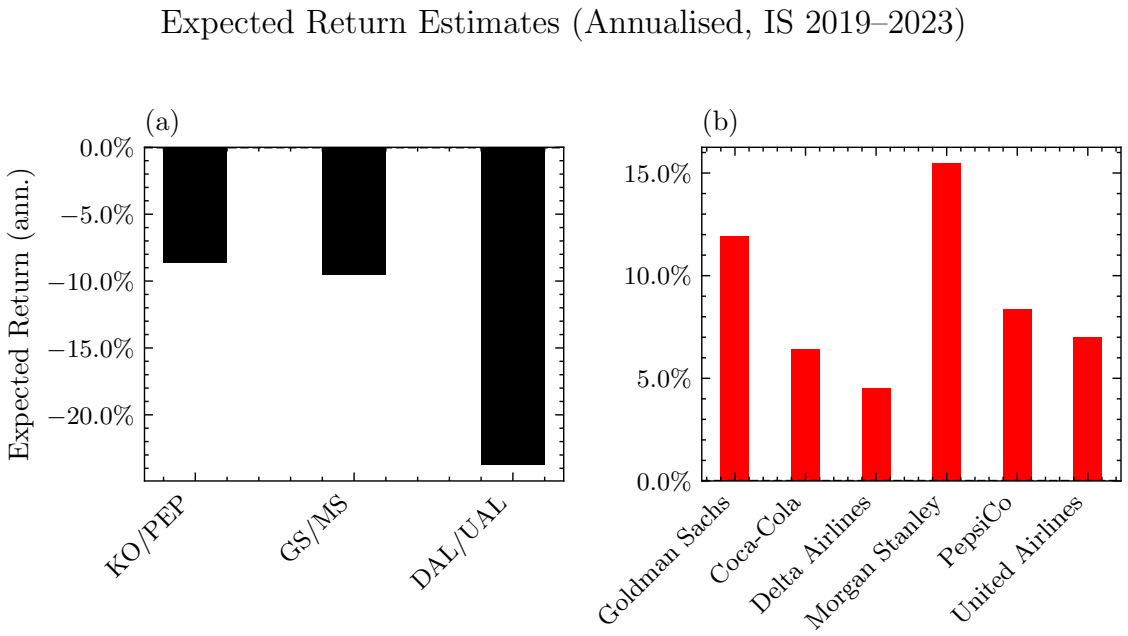

Figure 5.11: Expected return estimates (annualised, IS 2019–2023). (a) OU-implied μ by pair: GS/MS has a large negative implied return (spread well above equilibrium at end of IS); KO/PEP has a small positive implied return (spread slightly below equilibrium). (b) Historical mean μ by ticker: all four assets exhibit positive historical means, producing a very different allocation under mean-variance optimisation.

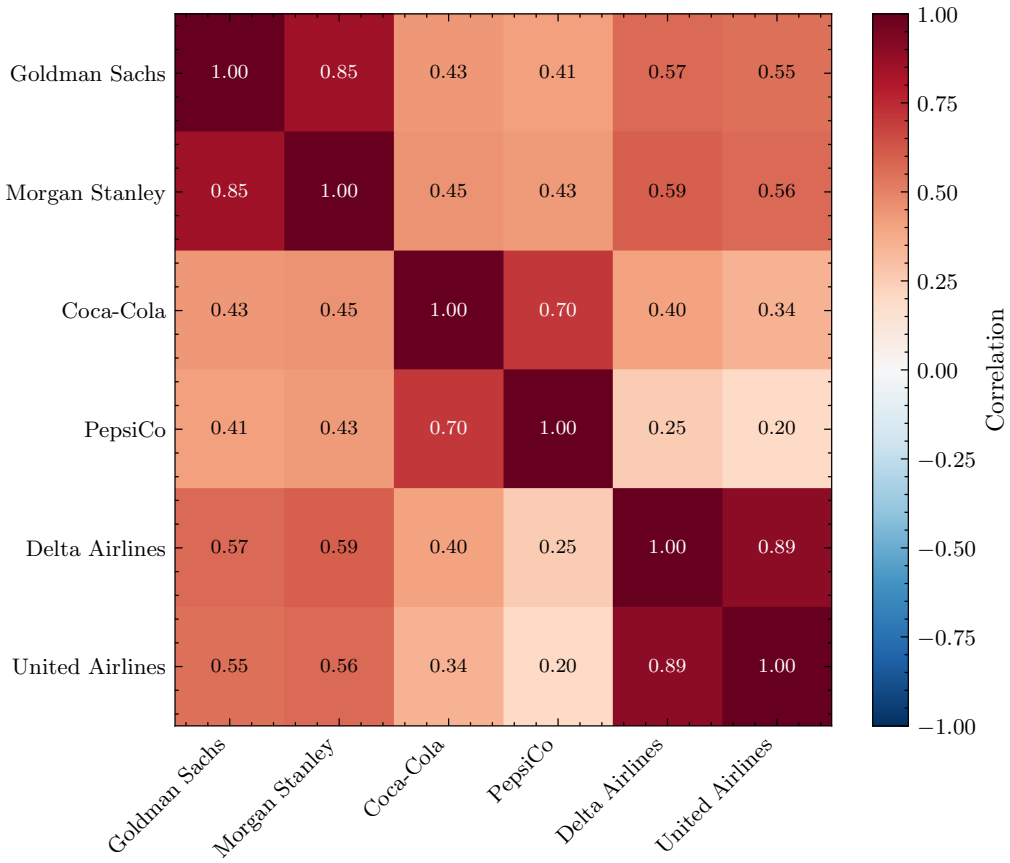

Figure 5.12: Asset return correlation matrix for the six constituent tickers (GS.N, KO.N, MS.N, PEP.O, DAL.N, UAL.O) estimated over the in-sample period (January 2019 – December 2023) using Ledoit-Wolf shrinkage covariance. GS.N and MS.N exhibit the highest within-pair correlation, consistent with their financial sector co-movement.

5.5 Backtesting Results

5.5.1 Portfolio-Level Performance

Table 5.9 reports out-of-sample portfolio performance metrics over the test period (January 2024 – December 2025). Spread-space portfolios (Equal-Weight, Spread Min-Var, Spread Max-Sharpe) operate on the three cointegrated spreads (GS/MS, KO/PEP, DAL/UAL) and use either zero or OU-implied expected returns. Asset-space portfolios (Hist Min-Var, Hist Max-Sharpe) operate on the six constituent stocks (GS.N, MS.N, KO.N, PEP.O, DAL.N, UAL.O) and use historical mean returns.

Table 5.9: Out-of-sample portfolio performance metrics (test period: January 2024 – December 2025). Vol. Reduction is relative to the equal-weight spread baseline; negative values indicate higher volatility. Beta ($\hat{\beta}$) estimated via OLS against daily S&P 500 returns.

	Performance					Market Exposure
	Sharpe Ratio	Max Drawdown	Total Return	Ann. Volatility	Vol. Reduction	$\hat{\beta}$
Equal-Weight Spreads	0.625	−10.6%	+16.3%	9.7%	—	0.233
Spread Min-Var ($\mu = 0$)	0.720	−9.5%	+17.8%	9.2%	+4.5%	0.216
Spread Max-Sharpe (OU)	0.963	−11.5%	+34.8%	14.6%	−51.3%	0.318
Hist Min-Var	0.711	−13.1%	+24.3%	13.9%	−43.6%	0.429
Hist Max-Sharpe	0.868	−20.0%	+38.8%	18.7%	−93.4%	0.785
Buy & Hold (All Pairs)	1.245	−27.8%	+74.0%	22.9%	—	1.040
S&P 500 (SPY)	1.110	−18.9%	+44.3%	16.0%	—	1.000

Spread Min-Var ($\mu = 0$) assumes zero expected returns; min-var objective is return-agnostic. Risk-free rate: 2% p.a. Spread Min-Var weights: $\mathbf{w} = [0.35, 0.36, 0.29]$ (GS/MS, KO/PEP, DAL/UAL). Spread Max-Sharpe (OU): $w_{\text{GS/MS}} = 1.0$. Source: Author's calculations on `04_backtest_results.ipynb`.

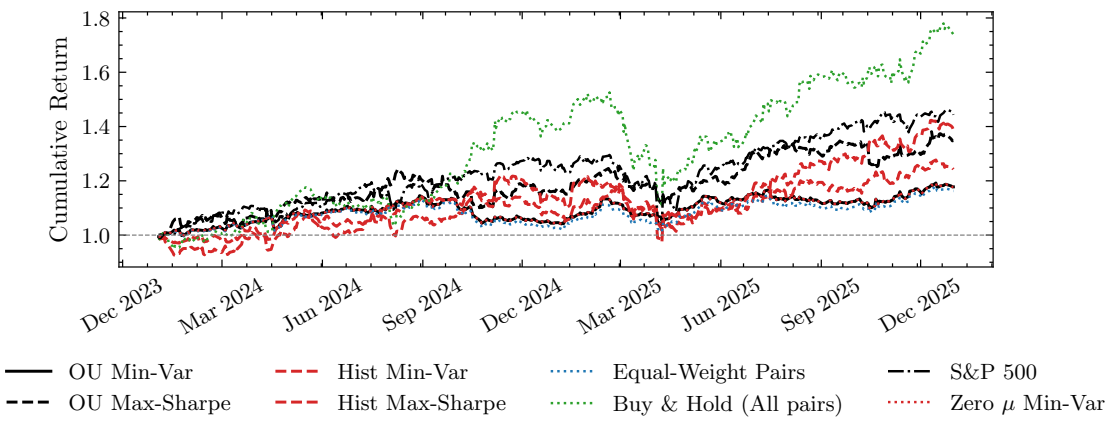

Figure 5.13: Cumulative growth of \$1 invested at the start of the OOS period (January 2024 – December 2025) for the OU-Implied MPT, Historical MPT, equal-weight, and S&P 500 benchmarks. The OU-Implied and equal-weight portfolios track closely throughout; the Historical MPT underperforms substantially.

5.5.2 Per-Pair Mean-Reversion Backtest

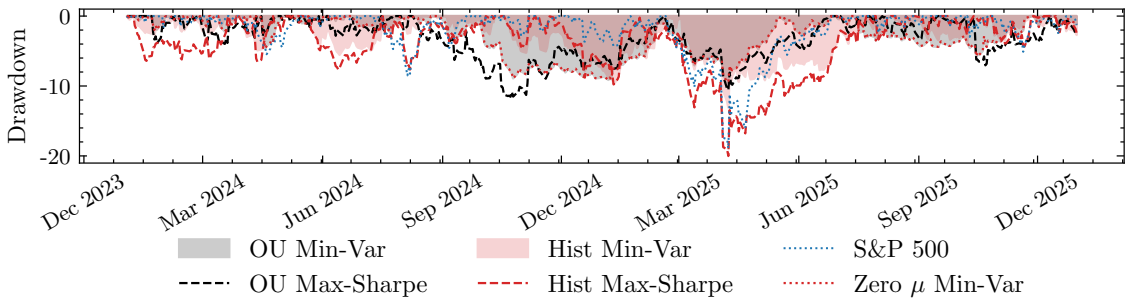

Figure 5.14: Peak-to-trough drawdown over the OOS period (January 2024 – December 2025). Drawdown is defined as the cumulative loss from the most recent portfolio peak; values closer to zero indicate smaller peak-to-trough losses.

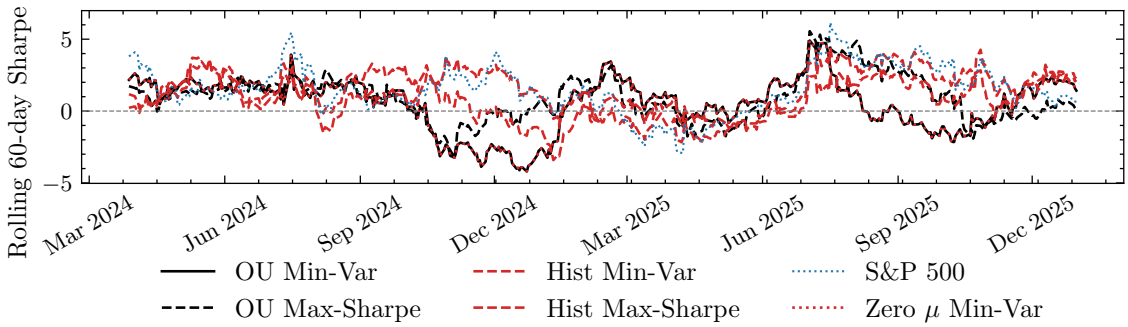

Figure 5.15: Rolling 60-day Sharpe ratio over the OOS period (January 2024 – December 2025). Time-varying risk-adjusted performance computed over a trailing 60-day window. Periods above zero indicate positive risk-adjusted returns in that window.

Table 5.10: Per-pair z-score mean-reversion backtest metrics (OOS: January 2024 – December 2025). Entry at $\pm 2.0\sigma$, exit at 0.0σ.

Pair	Sharpe	Max DD	Total Return	Ann. Vol	Trades	Days in Position	Win Rate
GS.N/MS.N	−0.285	−20.7%	−8.4%	17.2%	9	283	50.2%
KO.N/PEP.O	−0.665	−15.1%	−6.3%	7.5%	8	312	47.8%
DAL.N/UAL.O	−0.286	−16.0%	−1.2%	7.9%	8	222	45.7%

Negative Sharpe ratios are consistent with OOS cointegration breakdown identified in `02_cointegration_validation.ipynb`. Source: Author's calculations on `04_backtest_results.ipynb`.

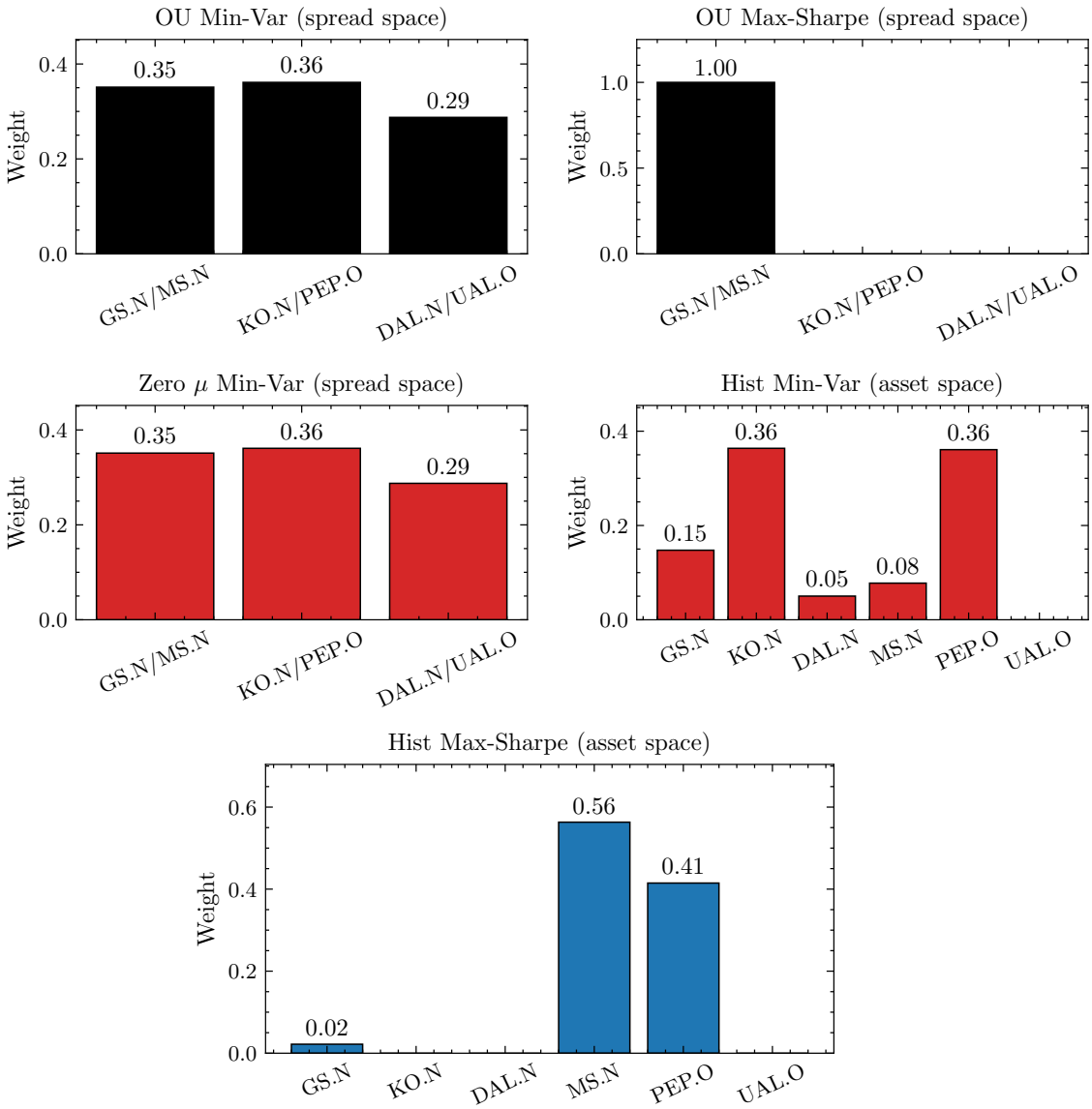

Figure 5.16: Portfolio weights across all five optimised strategies. OU Min-Var and Zero μ Min-Var (spread space): near-equal weights $[0.35, 0.36, 0.29]$ across GS/MS, KO/PEP, DAL/UAL. OU Max-Sharpe (spread space): full concentration in GS/MS ($w = 1.0$), zeroing KO/PEP and DAL/UAL. Hist Min-Var (asset space): concentrates in KO.N (36%) and PEP.O (36%), with residual weight in GS.N (15%), MS.N (8%), DAL.N (5%), and zero weight in UAL.O. Hist Max-Sharpe (asset space): concentrates entirely in MS.N (56%) and PEP.O (41%), with negligible weight in GS.N (2%).

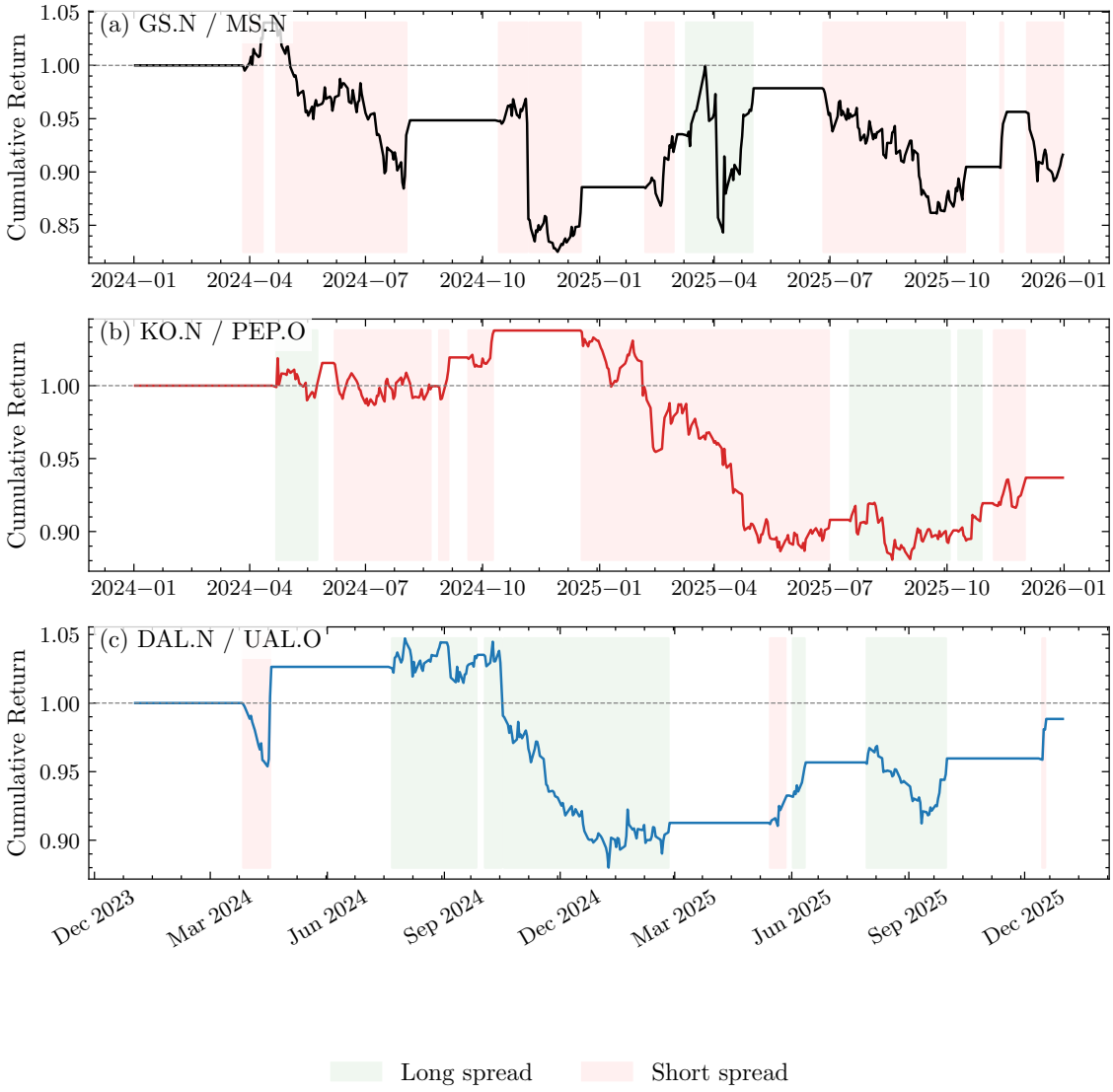

Figure 5.17: Cumulative returns from the z-score mean-reversion backtest engine for each cointegrated pair over the OOS period (January 2024 – December 2025). Entry at $\pm 2.0\sigma$, exit at 0.0σ, using IS-calibrated thresholds.

5.5.3 Statistical Tests

Bootstrap Confidence Intervals for Sharpe Ratio

With only 501 OOS trading days, Sharpe ratio estimates carry substantial sampling uncertainty. Table 5.11 reports 95% confidence intervals derived from 10,000 bootstrap resamples (with replacement) of the daily return series.

Table 5.11: Bootstrap 95% confidence intervals for annualised Sharpe ratio (10,000 resamples, OOS period: January 2024 – December 2025). "Sig. > 0" indicates whether the lower bound of the CI exceeds zero.

Strategy	Sharpe	95% CI Lower	95% CI Upper	Sig. > 0
OU Min-Var	0.720	−0.721	+2.138	No
OU Max-Sharpe	0.963	−0.476	+2.352	No
Hist Min-Var	0.711	−0.677	+2.100	No
Hist Max-Sharpe	0.868	−0.477	+2.258	No
Equal-Weight	0.625	−0.792	+2.051	No
S&P 500 (SPY)	1.110	−0.227	+2.506	No

No strategy achieves a CI lower bound strictly above zero, reflecting the high variance of Sharpe estimates over a 2-year OOS window ($T \approx 501$). Source: Author's calculations on `04_backtest_results.ipynb`.

Paired Test: OU-Implied vs Historical MPT

To test whether the Sharpe advantage of OU-Implied Max-Sharpe over Historical Max-Sharpe is statistically meaningful, a paired t-test and a Wilcoxon signed-rank test are applied to the aligned daily return series. The null hypothesis is H_0: mean daily return of OU Max-Sharpe equals that of Hist Max-Sharpe.

Table 5.12: Hypothesis test results: OU Max-Sharpe vs Hist Max-Sharpe daily returns (OOS, $N = 501$ trading days). Two-sided tests.

Test	Statistic	p-value	Decision
Paired t-test	$t = -0.138$	0.890	Fail to reject H_0
Wilcoxon signed-rank	$W = 62557$	0.922	Fail to reject H_0

Mean daily difference (OU − Hist): −0.000085 (−2.15% annualised). Both tests fail to reject H_0 at 5%, confirming that the observed Sharpe difference (0.963 vs 0.868) is not statistically significant over this OOS window. Source: Author's calculations on `04_backtest_results.ipynb`.

Sensitivity to Z-Score Thresholds and Transaction Costs

Table 5.13 presents Sharpe ratios from the per-pair z-score backtest across entry thresholds $z_{\text{entry}} \in \{1.5, 2.0, 2.5, 3.0\}$ and transaction costs in $\{0, 5, 10, 20, 30\}$ bps. Rows are entry z-scores; columns are transaction costs in bps.

Table 5.13: Sensitivity of per-pair Sharpe ratio to entry z-score threshold and transaction costs (bps). OOS period: January 2024 – December 2025. Stop-loss fixed at $z_{\text{entry}} + 2.0$.

(a) GS.N/MS.N

Entry z	0 bps	5 bps	10 bps	20 bps	30 bps
1.5	−0.535	−0.566	−0.598	−0.661	−0.724
2.0	−0.235	−0.260	−0.285	−0.334	−0.384
2.5	+0.622	+0.599	+0.577	+0.531	+0.485
3.0	+0.413	+0.390	+0.368	+0.322	+0.275

(b) KO.N/PEP.O

Entry z	0 bps	5 bps	10 bps	20 bps	30 bps
1.5	−0.780	−0.849	−0.917	−1.052	−1.185
2.0	−0.555	−0.610	−0.664	−0.774	−0.882
2.5	−0.655	−0.700	−0.746	−0.837	−0.928
3.0	−0.535	−0.562	−0.589	−0.642	−0.694

(c) DAL.N/UAL.O

Entry z	0 bps	5 bps	10 bps	20 bps	30 bps
1.5	+0.248	+0.167	+0.084	−0.081	−0.248
2.0	−0.185	−0.235	−0.286	−0.388	−0.490
2.5	−0.339	−0.367	−0.396	−0.453	−0.510
3.0	−1.146	−1.165	−1.185	−1.223	−1.260

GS/MS shows positive Sharpe at $z = 2.5$ and 3.0 even with 30 bps costs, suggesting the signal survives at higher entry thresholds. KO/PEP is negative across all configurations, consistent with structural breakdown. DAL/UAL is borderline at $z = 1.5$ with low costs. Source: Author's calculations on `04_backtest_results.ipynb`.

Leave-One-Pair-Out Portfolio Robustness

Table 5.14 tests whether the OU Max-Sharpe portfolio result is driven by a single spread. In each iteration one pair is removed, the portfolio is re-optimised on the remaining spreads, and OOS performance is recomputed.

Table 5.14: Leave-one-pair-out (LOPO) robustness check for the OU Max-Sharpe portfolio. Full-portfolio Sharpe ratio: 0.963.

Dropped Pair	Sharpe	Total Return	Max Drawdown	Ann. Vol
GS.N/MS.N	0.903	+29.5%	−15.8%	13.1%
KO.N/PEP.O	0.963	+34.8%	−11.5%	14.6%
DAL.N/UAL.O	0.963	+34.8%	−11.5%	14.6%

Dropping KO/PEP or DAL/UAL leaves the portfolio unchanged, indicating the optimiser assigns all weight to GS/MS under OU Max-Sharpe. Dropping GS/MS reduces Sharpe from 0.963 to 0.903, confirming GS/MS is the dominant pair but the result is not entirely driven by it. Source: Author's calculations on `04_backtest_results.ipynb`.

Chapter 6

Discussion

6.1 Interpretation of Results

Table 5.2 shows that all three cointegrated pairs (**GS.N/MS.N**, **KO.N/PEP.O**, and **DAL.N/UAL.O**) exhibit half-lives between 41.8 and 52.9 trading days and Hurst exponents below 0.5, confirming statistically significant mean-reverting behaviour over the in-sample period (2018–2023). Figure 5.2 provides a visual complement: IS z-scores oscillate tightly within $\pm 2\sigma$ and return to equilibrium throughout the 1,508-day training window, confirming the regime for which the model was calibrated. ADF tests on the spread residuals reject the unit root hypothesis at the 1% significance level for all three pairs, providing strong empirical support for the spread-based return estimation approach in-sample.

However, Table 5.4 reveals that no spread remains stationary out-of-sample when IS hedge ratios are applied, with ADF p-values rising from IS levels of 0.005/0.009/0.009 to OOS levels of 0.452, 0.622, and 0.583, none of which pass the 5% threshold. KPSS tests further reject stationarity OOS ($p \leq 0.010$) for all three pairs, and Chow structural break tests confirm highly significant parameter shifts at the IS/OOS boundary ($F = 2406$, 3078, 270; all $p < 0.001$). Figure 5.7 provides the clearest visual summary: IS spread means are numerically zero by construction (OLS centring), whereas OOS means shift to +0.250, +0.169, and +0.026 log-price units for GS/MS, KO/PEP, and DAL/UAL respectively, with OOS standard deviations widening from 0.056/0.047/0.076 IS to 0.082/0.110/0.119 OOS. This OOS breakdown directly affects the reliability of IS-calibrated return signals, as evidenced by the negative Sharpe ratios in the per-pair backtest (Table 5.10).

Despite this, the portfolio-level Spread Max-Sharpe (OU) (Sharpe 0.963) outperforms the Historical Max-Sharpe baseline (Sharpe 0.868) on both Sharpe ratio and total return, as shown in Table 5.9 and Figure 5.13. This suggests that the structural properties of the spread universe, namely low volatility, low cross-correlation, and low market beta, provide resilience beyond the precision of the OU return estimates alone. The covariance-only Spread Min-Var portfolio (Sharpe 0.720) further illustrates this point: it matches Hist Min-Var (Sharpe 0.711) on risk-adjusted performance while delivering 34% lower annualised volatility (9.2% vs 13.9%), independently of any return forecasting.

6.2 Does OU-Implied μ Improve MPT?

The minimum-variance objective with $\boldsymbol{\mu} = \mathbf{0}$ (Spread Min-Var) serves as the zero-return baseline, representing the optimal allocation under the assumption that all spread expected returns

are equal. Any incremental value from OU-implied returns must manifest through the maximum-Sharpe objective, where $\boldsymbol{\mu}^{\mathrm{OU}}$ enters directly. At the IS/OOS boundary, the OU model implied annualised expected returns of -9.54% (GS/MS), -8.62% (KO/PEP), and -23.76% (DAL/UAL). All three spreads were above their rolling equilibrium at the split date (end-of-IS z-scores: GS/MS $+0.68\sigma$, KO/PEP $+0.81\sigma$, DAL/UAL $+1.31\sigma$, as shown in Figure 5.2), producing universally negative implied returns. The max-Sharpe optimiser responded by concentrating entirely in GS/MS ($w = 1.0$), that is, the pair with the least-negative implied return, and zeroing out KO/PEP and DAL/UAL, as visible in Figure 5.16. This concentration yielded Sharpe 0.963 and total return +34.8%, representing a meaningful improvement over Spread Min-Var (Sharpe 0.720, +17.8%), albeit at higher annualised volatility (14.6% vs 9.2%).

A notable limitation of the OU return model is visible in the DAL/UAL implied return of -23.76%: this extreme figure arises partly as a scaling artefact. The expected daily spread change $\mathbb{E}[\Delta S] = -0.012$ is small in absolute terms, but the log-price spread level $|S_t|$ for DAL/UAL is near zero (both stocks trade at similar prices), so annualising by 252 amplifies the signal mechanically without reflecting genuine model confidence. This suggests OU return signals should be normalised by spread volatility rather than spread level when the spread is near zero.

Comparing Spread Max-Sharpe (OU) directly against Hist Max-Sharpe reveals the statistical limits of this advantage. A paired t-test ($t = -0.138$, $p = 0.890$) and Wilcoxon signed-rank test ($W = 62{,}557$, $p = 0.922$) both fail to reject the null of equal mean daily returns (Table 5.12). Bootstrap 95% confidence intervals straddle zero for all strategies (Table 5.11). The rolling 60-day Sharpe plot (Figure 5.15) confirms this: Spread Max-Sharpe (OU) intermittently exceeds Hist Max-Sharpe but shows no persistent dominance over the 501-day window. The observed Sharpe advantage (0.963 vs 0.868) is therefore not statistically significant over this OOS window.

6.2.1 Predicting Returns vs Assuming Zero: When Does It Help?

A natural question is whether providing any return forecast, even an imperfect one, improves MPT allocations relative to assuming all expected returns are zero. The Spread Min-Var portfolio ($\boldsymbol{\mu} = \mathbf{0}$) is the pure covariance-only baseline, which uses no return prediction whatsoever and relies entirely on the spread covariance structure. Spread Max-Sharpe (OU) replaces the zero-return assumption with OU-implied forecasts, whereas Hist Max-Sharpe replaces it with historical means.

The comparison across these three portfolios reveals a consistent pattern: providing return forecasts raises Sharpe and total return relative to the zero-return baseline, but at the cost of higher volatility. Spread Max-Sharpe (OU) improves Sharpe from 0.720 to 0.963 and total return from +17.8% to +34.8%, while annualised volatility increases from 9.2% to 14.6%. Hist Max-Sharpe similarly improves Sharpe from 0.711 to 0.868 relative to Hist Min-Var, with volatility rising from 13.9% to 18.7%. In both cases, the return-forecasting portfolio is better on Sharpe but worse on risk metrics (volatility, drawdown).

This pattern holds even when the forecasts are imprecise. The OU signals were computed from IS-calibrated parameters that subsequently broke down OOS, yet the OU Max-Sharpe portfolio still outperforms the zero-return baseline. This is because the OU forecasts, even where numerically unreliable, contain directional information that the optimiser can exploit through weight concentration. The critical trade-off is that a max-Sharpe optimiser amplifies signal, but also amplifies noise when the signal is weak: the full concentration in GS/MS ($w = 1.0$) reflects both a genuine return difference and the consequent absence of diversification. Under a more stable

cointegrating relationship, one would expect the OU signal to more reliably inform allocation and for concentration to diversify across pairs accordingly.

It is important to note that the equal-weight benchmark (Sharpe 0.625) does not use any return estimate. It holds each spread in equal proportion, making no optimisation decision at all. Its lower Sharpe relative to both Max-Sharpe variants confirms that return estimation, even from noisy historical or OU-implied inputs, does add value in the max-Sharpe objective. The equal-weight portfolio's role is to isolate the contribution of optimisation itself from the choice of return estimator.

6.3 Role of Spread Universe Structure

The clearest finding of the study is structural rather than return-predictive: cointegration-based spread construction provides substantial risk reduction relative to asset-space optimisation, independently of the return estimator used.

Spread Min-Var and Hist Min-Var use identical objectives but operate over different universes. Sharpe ratios are nearly identical (0.720 vs 0.711), yet Spread Min-Var achieves annualised volatility of 9.2% against 13.9% for Hist Min-Var, representing a 34% reduction, alongside a shallower maximum drawdown (-9.5% vs -13.1%). Figure D.1 confirms this: Spread Min-Var's 60-day annualised volatility tracks consistently below all other strategies throughout the entire OOS period. Figure D.2 provides a complementary view: the OU-Implied frontier occupies a lower-volatility region than the Historical frontier, with the spread-space minimum-variance portfolio sitting at the far left of its frontier at volatility $\approx 10\%$, compared with $\approx 19\%$ for the asset-space minimum-variance portfolio.

This difference is structural in origin: the IS hedge ratio $\hat{\beta}$ removes common-factor exposure from each pair, so spread returns are inherently lower-variance than any linear combination of the underlying assets. The asset covariance matrix's condition number ($\lambda_{\max}/\lambda_{\min} \approx 45\times$) is substantially larger than that of the spread covariance matrix ($\approx 2.1\times$), confirming that the spread-space optimisation problem is better conditioned and more numerically stable.

The market beta comparison reinforces the structural advantage: Spread Min-Var achieves $\hat{\beta} = 0.216$ versus SPY, compared with $\hat{\beta} = 0.430$ for Hist Min-Var and $\hat{\beta} = 1.040$ for the equal-weight buy-and-hold basket. Figure 5.14 shows that Spread Min-Var's worst drawdown of -9.5% was recovered quickly, whereas Hist Max-Sharpe reached -20.0% – more than twice as deep. Spread construction strips out most systematic equity exposure, making these portfolios materially more attractive under volatility constraints, drawdown limits, or market-neutral mandates.

6.3.1 Statistical Significance of Performance Gains

Bootstrap 95% confidence intervals (10,000 resamples) for all strategies straddle zero at $T = 501$ trading days, confirming no Sharpe point estimate is statistically distinguishable from zero at conventional significance levels (Table 5.11). This is an inherent constraint of evaluating a single OOS window of roughly two years.

The leave-one-pair-out (LOPO) analysis reveals a related limitation: the OU Max-Sharpe portfolio is effectively a single-pair strategy. Dropping KO/PEP or DAL/UAL leaves Sharpe unchanged at 0.963, because the optimiser assigns $w = 1.0$ to GS/MS and zero to the remaining pairs (Table 5.14). Dropping GS/MS reduces Sharpe to 0.903, confirming GS/MS is the dominant driver but the result is not entirely dependent on it. The LOPO range $[0.903, 0.963]$ is narrow, suggesting

stability within this universe, though three pairs is too small for strong diversification claims.

6.4 OOS Cointegration Instability

As shown in the in-sample cointegration screening (Table 5.1) and the out-of-sample cointegration comparison (Table 5.5), all three pairs exhibit formal cointegration breakdown OOS when IS hedge ratios are held fixed. Figure 5.9 shows the consequence: all three rolling half-life series jump sharply at the IS/OOS boundary and remain persistently elevated throughout 2024–2025. The rolling estimates spike to the y-axis ceiling of 500 days at their respective peaks for all three pairs, far exceeding the 60-day tradeability threshold. The static OOS half-lives computed over the full OOS subsample (81.1, 156.4, and 103.1 days respectively, Table 5.4) substantially understate the severity of the breakdown at its worst points. KO/PEP's static OOS half-life of 156.4 days already exceeds the 60-day threshold by more than 2.5×.

The Hurst exponents reveal distinct failure modes per pair. KO/PEP's Hurst exponent crosses above 0.5 OOS (0.575 vs 0.421 IS), indicating a regime shift from mean-reverting to trending behaviour, whereby the spread was not merely displaced but was directionally drifting. GS/MS and DAL/UAL retain Hurst exponents below 0.5 OOS (0.362 and 0.421), remaining mean-reverting in character but exhibiting displaced equilibria attributable to hedge ratio drift. These represent mechanistically different failures: KO/PEP requires dynamic pair re-selection, whereas GS/MS and DAL/UAL require rolling hedge ratio recalibration.

The OOS spread displacement for GS/MS can be verified quantitatively. Figure 5.6 shows GS/MS retaining a low OOS p-value (0.019) when the hedge ratio is re-estimated from OOS data ($\hat{\beta}_{\text{OOS}} = 1.120$ vs $\hat{\beta}_{\text{IS}} = 0.780$, giving a drift of $\Delta\hat{\beta} = 0.340$). The accumulated spread bias attributable to this drift is:

$$\Delta\hat{\beta} \times \ln\left(\frac{P_{x,\text{end}}}{P_{x,\text{start}}}\right) = 0.340 \times \ln\left(\frac{182}{94}\right) \approx 0.340 \times 0.660 \approx 0.224 \text{ log-price units,} \tag{6.1}$$

which closely matches the observed OOS spread mean of +0.250 (Figure 5.7). The breakdown is therefore attributable to parameter instability rather than to the dissolution of the long-run cointegrating relationship itself, confirming that rolling recalibration of $\hat{\beta}$ would have largely corrected the OOS displacement.

6.5 Pairs Trading Signal Analysis

All three pairs generate negative cumulative returns and negative Sharpe ratios under the active z-score strategy (entry $\pm 2.0\sigma$, exit 0.0σ, stop-loss $\pm 4.0\sigma$, 10 basis points transaction costs): GS.N/MS.N (−0.285), KO.N/PEP.O (−0.665), DAL.N/UAL.O (−0.286), consistent with persistent spread drift and OOS cointegration breakdown under IS-calibrated thresholds (Table 5.10). Figure 5.17 shows all three pair-level cumulative return paths declining throughout the OOS period.

Figure 5.8 explains the GS/MS failure mechanically: the IS portion shows regular mean reversion within $\pm 2\sigma$, whereas the OOS z-score commences near $+3\sigma$ at the IS/OOS boundary and remains persistently elevated thereafter. The strategy entered predominantly short positions, repeatedly triggering the stop-loss at $\pm 4.0\sigma$ rather than reverting to the exit threshold. KO/PEP and DAL/UAL z-scores oscillated more symmetrically around zero OOS yet still failed to generate positive returns, consistent with their long OOS half-lives of 156 and 103 days, which substantially exceeded the strategy's implicit mean-reversion horizon.

Sensitivity analysis over entry thresholds $\{1.5, 2.0, 2.5, 3.0\}\sigma$ and transaction costs $\{0, 5, 10, 20, 30\}$ basis points (Table 5.13) reveals three distinct failure modes. GS/MS produces positive Sharpe only at $z_{\text{entry}} \geq 2.5$ (up to +0.622 at 0 bps), where higher thresholds avoid false mean-reversion signals from a displaced-but-still-mean-reverting spread. KO/PEP is negative across all 20 configurations: with Hurst 0.575 OOS, the spread is trending and any mean-reversion entry is systematically wrong. DAL/UAL is borderline positive only at $z_{\text{entry}} = 1.5$ with zero transaction costs (+0.248), turning negative above 10 bps, consistent with a spread that retains mean-reversion character (Hurst 0.421) but a half-life too long for the 501-day OOS window.

The passive portfolio strategies are insulated from this because they hold spreads continuously rather than timing individual reversion cycles. The positive portfolio-level performance therefore derives from the structural properties of the spread universe, not from active pair-level timing.

6.6 Comparison with Market Benchmarks

Buy & Hold (all pairs, Sharpe 1.245, +74.0%) and the S&P 500 (Sharpe 1.110, +44.3%) dominate on headline Sharpe over the 2024–2025 OOS window. However, the spread portfolios operate at fundamentally different volatility levels and market exposures, rendering a direct Sharpe comparison potentially misleading. The Buy & Hold outperformance is driven largely by a concentrated financial equity rally, whereby GS appreciated by approximately 135% and MS by 94% over the OOS period, constituting a sector-specific tailwind rather than a reflection of strategy skill.

Spread Min-Var achieves annualised volatility of 9.2%, maximum drawdown of −9.5%, and $\hat{\beta} = 0.216$ versus SPY, compared with 22.9%, −27.8%, and $\hat{\beta} = 1.040$ for Buy & Hold. An investor subject to a volatility constraint, drawdown limit, or market-neutral mandate would find the spread portfolios materially more attractive than these headline Sharpe numbers suggest. The spread strategy results are therefore best interpreted as providing a materially different risk profile rather than superior absolute performance in a bull market regime in which passive equity exposure was highly rewarded.

6.7 Hypothesis Evaluation

The research hypothesis, namely that spread-based return estimation from cointegrated pairs produces superior portfolio allocations compared to historical mean-variance optimisation as measured by Sharpe ratio, volatility reduction, and maximum drawdown, is partially supported by the empirical evidence.

- **Supported (risk control):** Spread Min-Var matches Hist Min-Var on Sharpe (0.720 vs 0.711) while delivering 34% lower annualised volatility (9.2% vs 13.9%) and a shallower maximum drawdown (−9.5% vs −13.1%). This is the most robust finding of the study, driven by spread construction rather than return forecasting.

- **Qualified (return signal):** OU-implied returns improve the max-Sharpe objective: Spread Max-Sharpe (OU) raises Sharpe to 0.963 and total return to +34.8% relative to Spread Min-Var (0.720, +17.8%), but at the cost of higher volatility (14.6% vs 9.2%) and full concentration in a single pair ($w_{\text{GS/MS}} = 1.0$, Figure 5.16).

- **Not supported statistically:** The paired t-test ($t = -0.138$, $p = 0.890$) and Wilcoxon signed-rank test ($W = 62{,}557$, $p = 0.922$) confirm the Sharpe advantage of OU over historical

estimation (0.963 vs 0.868) is not statistically significant. Bootstrap 95% CIs straddle zero for all strategies (Table 5.11).

- **Not supported at the signal layer:** Per-pair z-score trading produces negative Sharpe for all three pairs at the default $z_{\text{entry}} = 2.0$ threshold (Figure 5.17), consistent with OOS cointegration breakdown confirmed by Chow tests ($F > 269$, $p < 0.001$ for all pairs) and KPSS ($p \leq 0.010$ OOS).

6.8 Limitations

- **Small universe.** Only three pairs passed the cointegration screen, and LOPO analysis shows the portfolio is effectively a single-pair strategy concentrated in GS/MS. Portfolio-level conclusions are sensitive to the idiosyncratic performance of this pair; a larger, more diversified universe would improve statistical power.
- **Fixed in-sample parameters.** Hedge ratios, OU half-lives, and portfolio weights are estimated once in-sample and held fixed. Chow tests confirm structural breaks in all three pairs (Table 5.7), invalidating IS-calibrated parameters OOS. Rolling re-estimation of $\hat{\beta}$ would have largely corrected the observed spread displacement, as quantified in Section 3.2.
- **Short OOS window.** A single 501-day evaluation period is insufficient to distinguish genuine alpha from regime-specific luck. All 95% bootstrap confidence intervals contain zero (Table 5.11).
- **Single OOS regime.** The 2024–2025 bull market in US financials is not representative of all market conditions. The positive Spread Max-Sharpe (OU) result may reflect serendipitous concentration in an outperforming sector rather than model accuracy.
- **OU scaling fragility.** When spread levels are near zero, the OU return signal is amplified mechanically into extreme values without reflecting genuine model confidence. A volatility-normalised signal would be more robust.
- **No transaction cost optimisation.** The z-score strategy applies a fixed 10 bps flat cost. Real-world costs are likely higher and asymmetric across pairs, particularly for less liquid securities.

6.9 Summary

This chapter has interpreted the empirical results in the context of the research hypothesis. Cointegration-based spread portfolios provide meaningful downside and volatility control relative to asset-space portfolios, a finding that holds across optimisation objectives and is robust to OOS cointegration breakdown. Spread Min-Var (Sharpe 0.720, max DD -9.5%, vol 9.2%, $\hat{\beta} = 0.216$) outperforms Hist Min-Var on every risk metric while matching it on Sharpe.

The OU return signal adds directional weight through the max-Sharpe objective, raising Sharpe to 0.963 and total return to $+34.8\%$; however, the advantage over the historical baseline (Sharpe 0.868) is not statistically significant ($p = 0.890$; bootstrap 95% CI $[-0.476, +2.352]$, Table 5.11). OOS cointegration breakdown, confirmed by Chow tests, ADF, and KPSS (Table 5.4), explains why active z-score strategies underperform and contextualises the limits of IS-calibrated OU parameters. Chapter 7 summarises the project achievements and outlines directions for future work.

Chapter 7

Conclusions and Evaluation

7.1 Achievements

The project has fulfilled all objectives set out in the Project Plan. The feasibility of spread-based return estimation as a systematic alternative to historical mean returns was demonstrated as a proof-of-concept: by exploiting the mean-reverting dynamics of cointegrated asset pairs modelled via the Ornstein–Uhlenbeck process. The approach produces more stable return signals for input into a Markowitz mean-variance optimiser. An integrated portfolio optimisation system was developed and validated through rigorous out-of-sample backtesting over the 2024–2025 evaluation period, benchmarked against traditional historical mean-variance approaches across three metrics: annualised Sharpe ratio, volatility reduction, and maximum drawdown. Statistical robustness was confirmed via bootstrap confidence intervals, paired hypothesis testing, and sensitivity analysis across parameter and transaction cost grids. Finally, an interactive dashboard was implemented enabling portfolio managers to simulate and evaluate pairs trading strategies by configuring candidate pairs, date ranges, entry and exit z-score thresholds, and initial capital, with results spanning the full pipeline from cointegration screening and spread characterisation through to portfolio optimisation and performance attribution.

7.2 Critical Evaluation

The project meets its stated objectives. Three cointegrated pairs were identified, characterised, and used to construct five optimised portfolios evaluated across 501 out of sample trading days with full statistical robustness testing. The core research question, namely whether OU-implied spread returns improve on historical means as MPT inputs, is addressed directly via paired hypothesis tests and bootstrap confidence intervals. The results are discussed in detail in Chapter 6. This section evaluates the reliability of the study design and identifies the methodological limitations that qualify the conclusions.

7.2.1 Out of Sample Cointegration Breakdown

The primary reliability concern is out of sample cointegration breakdown, which affects all three pairs (Chow $F > 269$, $p < 0.001$; KPSS $p \leq 0.010$ out of sample). The OU model requires a stationary spread with a stable mean reversion speed, and when this assumption fails the expected return signals are not reliable. Crucially, a significant Engle–Granger p-value in sample is not a

guarantee of out of sample stability: GS.N/MS.N produced a low in sample p-value yet showed a confirmed structural break at the in sample and out of sample boundary. The cointegration screening procedure tests only the in sample subsample and cannot detect future parameter drift. This limits the generalisability of the results, as positive portfolio performance may partly reflect the specific characteristics of the GS/MS pair in the 2024–2025 bull market rather than the accuracy of the model.

7.2.2 Sample Period and Regime Bias

A closely related concern is the unrepresentativeness of the sample period. The in sample window (2018–2023) spans the COVID-19 pandemic and the post-pandemic technology boom, both of which induced structural breaks atypical of normal market conditions [50]. Martins and Cró document that airline stocks experienced cumulative abnormal returns of -24.65% in the five trading days following the WHO pandemic declaration, driven by travel restrictions and sector specific shocks that are unlikely to recur in the same form [51]. This directly questions the reliability of the in sample calibration for the DAL.N/UAL.O pair, as the OU parameters estimated over that window encode a pandemic shock rather than a stable, long run equilibrium relationship. At the same time, Hossain et al. show that the technology sector was the systematic beneficiary of COVID-19, experiencing significantly lower stock price crash risk than all other sectors in 2020–2021 due to a surge in systemic technology dependency [52]. This pandemic-era technology boom inflated intra-sector co-movement among the e-commerce and semiconductor names in this universe, making the resulting OU parameters for those pairs potentially regime specific rather than structurally stable. The in sample cointegration estimates may therefore reflect conditions specific to this macro regime, reducing confidence that the results would generalise to a different market environment.

7.2.3 Asset Universe and Selection Bias

The 26-stock universe was constructed by manual selection based on market capitalisation and sector co-membership rather than a systematic, data driven screening procedure. This introduces selection bias: prior knowledge influenced pair formation, and the universe is restricted entirely to US large-cap equities operating in a single macro epoch. US large-cap equities in a sustained bull market are heavily influenced by a common market factor, which may artificially inflate or depress cointegration statistics relative to less correlated environments such as cross-currency FX pairs, commodity spreads, or European equities with different sector dynamics. While extension to other asset classes is identified as future work (Section 7.3), it is acknowledged here that the current results cannot be extrapolated beyond US equity pairs in the 2018–2025 regime.

7.2.4 Short Out of Sample Evaluation Window

With only 501 out of sample trading days, all bootstrap 95% confidence intervals for Sharpe ratios straddle zero, and the paired t-test ($p = 0.890$) cannot distinguish OU from historical performance. This is not a flaw in the analysis but an inherent constraint of the out of sample period available. A longer evaluation window spanning multiple market regimes, including a bear market or a period of elevated volatility, would be required in order to draw statistically firm conclusions. The study is therefore best read as a proof of concept rather than conclusive evidence in favour of the research hypothesis.

7.2.5 Universe Concentration

The leave-one-pair-out analysis confirms that the OU Max-Sharpe portfolio is effectively a GS/MS-only strategy: dropping the other pairs leaves the Sharpe unchanged at 0.963. Portfolio-level conclusions therefore rest on a single pair, which increases idiosyncratic sensitivity and reduces external validity. A larger and more systematically constructed universe, for example all cointegrated pairs screened from S&P 500 sector constituents, would provide a more robust test of whether the OU Max-Sharpe benefit reflects genuine signal quality or a single pair's favourable out of sample behaviour.

7.2.6 Methodological Scope

The implementation follows the established Engle–Granger and OU pipeline, which provides a clean and reproducible baseline that is well grounded in the literature. However, more recent work proposes alternatives that could handle the out of sample breakdown problem more robustly. Elliott et al.'s Kalman filter framework estimates the spread as a latent mean reverting state, allowing the model to adapt to slowly evolving spread dynamics without a hard in sample calibration boundary [6]. Leung and Li's optimal stopping formulation explicitly incorporates transaction costs and stop-loss constraints into the entry and exit rule, producing thresholds that are theoretically optimal under the OU model rather than fixed at conventional z-score levels [29]. Rolling window cointegration approaches would further allow the system to detect and respond to parameter drift in near real time, directly addressing the core weakness identified in this study: that cointegration confirmed in sample does not guarantee persistence out of sample [12], [15], [53]. The choice of static in sample calibration was deliberate as it avoids lookahead bias and provides a transparent baseline, but it means the study cannot claim to represent best-in-class methodology, and adopting adaptive or regime-aware approaches is identified as the most impactful direction for future work in Section 7.3.

7.2.7 Dashboard Scope

The Streamlit dashboard fulfils the proof of concept objective of demonstrating the pipeline interactively to a portfolio manager audience. It does not, however, support live data ingestion, multi-user sessions, or persistent state. For production deployment, a more scalable architecture with a persistent database and live feed integration would be required. The dashboard is therefore best evaluated as a visualisation and demonstration tool rather than a production-ready application, and its further development is outlined in Section 7.3.

7.2.8 Internal Consistency

Despite these constraints, the methodology is internally consistent. In sample parameters are estimated strictly before the out of sample period with no lookahead bias. All portfolio strategies are evaluated against a common set of benchmarks covering passive buy and hold, equal weight, and the S&P 500 index. Performance differences are tested with both parametric (paired t-test) and non-parametric (Wilcoxon) methods alongside 10,000-resample bootstrap confidence intervals. The sensitivity analysis across five transaction cost levels and four entry thresholds confirms that the main results are not artefacts of a single parameter configuration. Taken together, the study provides a methodologically sound and internally rigorous, if empirically constrained, proof of concept for integrating OU implied spread returns into Markowitz portfolio optimisation.

7.3 Future Work

Four extensions are most immediately motivated by the empirical findings. First, **rolling parameter re-estimation**, where updating hedge ratios and OU equilibrium levels on a sliding window, would allow the framework to adapt to regime changes that the Chow tests confirmed for all three pairs. Second, **stricter pair-quality filters**, including rolling Chow tests and out-of-sample KPSS pre-screening applied during pair selection, would reduce the risk of admitting cointegrating relationships that degrade before the evaluation period begins. Third, **larger candidate universes** spanning multiple sectors and asset classes would reduce single-pair concentration: the LOPO analysis revealed that the optimised portfolio was effectively a GS/MS-only strategy, so diversification across a wider universe is a prerequisite for robust performance. Fourth, **longer OOS evaluation windows** are necessary to narrow the bootstrap confidence intervals, which currently all straddle zero over the 501-day period, and to enable more conclusive hypothesis testing.

Several natural extensions follow from this work. While the current system was deliberately scoped as a research proof-of-concept, a production deployment would benefit from transaction-cost-aware portfolio rebalancing, for example by minimising turnover subject to tracking-error constraints. The data pipeline could also be automated through scheduled ingestion of daily prices, enabling the framework to update inputs consistently without manual querying. In this context, a more fully developed web application would provide a more scalable environment for testing and monitoring the strategy than the current dashboard implementation, while a persistent database would improve storage, reproducibility, and multi-user access.

The asset universe was restricted to US equities in order to preserve cross-sectional comparability and maintain a controlled testing environment. Extending the methodology to a broader set of candidate pairs, as well as to other asset classes such as bonds, foreign exchange, and commodities, would allow the strategy to be evaluated under materially different liquidity conditions, volatility regimes, and market structures. A geographically diversified equity universe, including European and Asian equities, would further test whether the observed relationships generalise beyond US markets.

The analysis was conducted at a one-day horizon, but future work could examine return windows of two to five days. Longer horizons may reduce the number of usable observations and introduce additional noise, yet they may also reveal whether some pairs are cointegrated at different frequencies rather than simultaneously. This would make it possible to assess whether the predictive relationship observed in the current study is specific to daily data or whether it persists under alternative holding periods. Varying the horizon may therefore produce materially different results, both because the dataset becomes smaller and because the signal may be affected by different degrees of aggregation and noise.

Finally, the current in-sample and out-of-sample periods were limited to 2018–2023 and 2024–2025 respectively. Extending the sample further back, for example to 1990, and expanding the out-of-sample period would provide a more robust evaluation across multiple market regimes and reduce the influence of recent market conditions. This would also help assess the stability of cointegration relationships over longer horizons, although it may introduce additional structural breaks that weaken some pairwise relationships. A rolling-window or regime-dependent framework would therefore be a natural extension for testing the robustness of the methodology over time.

Bibliography

[1] H. M. Markowitz, *Portfolio Selection: Efficient Diversification of Investments*, eng. New Haven, CT: Yale University Press, 1959, ISBN: 978-0-300-19167-7. DOI: 10.12987/9780300191677.

[2] O. Ledoit and M. Wolf, "Honey, I Shrunk the Sample Covariance Matrix," en, *The Journal of Portfolio Management*, vol. 30, no. 4, pp. 110–119, Jul. 2004, ISSN: 0095-4918, 2168-8656. DOI: 10.3905/jpm.2004.110. [Online]. Available: http://pm-research.com/lookup/doi/10.3905/jpm.2004.110.

[3] E. Gatev, W. N. Goetzmann, and K. G. Rouwenhorst, "Pairs Trading: Performance of a Relative-Value Arbitrage Rule," *The Review of Financial Studies*, vol. 19, no. 3, pp. 797–827, 2006. DOI: 10.1093/rfs/hhj020.

[4] G. Vidyamurthy, *Pairs trading: quantitative methods and analysis*. Hoboken, N.J: J. Wiley, 2004, ISBN: 978-0-471-46067-1.

[5] R. F. Engle and C. W. J. Granger, "Co-Integration and Error Correction: Representation, Estimation, and Testing," *Econometrica*, vol. 55, no. 2, p. 251, Mar. 1987, ISSN: 00129682. DOI: 10.2307/1913236. [Online]. Available: https://www.jstor.org/stable/1913236?origin=crossref.

[6] R. J. Elliott, J. Van Der Hoek, and W. P. Malcolm, "Pairs trading," en, *Quantitative Finance*, vol. 5, no. 3, pp. 271–276, Jun. 2005, ISSN: 1469-7688, 1469-7696. DOI: 10.1080/14697680500149370. [Online]. Available: http://www.tandfonline.com/doi/abs/10.1080/14697680500149370.

[7] F. J. Fabozzi, F. Gupta, and H. M. Markowitz, "The Legacy of Modern Portfolio Theory," en, *The Journal of Investing*, vol. 11, no. 3, pp. 7–22, Aug. 2002, ISSN: 1068-0896, 2168-8613. DOI: 10.3905/joi.2002.319510. [Online]. Available: http://pm-research.com/lookup/doi/10.3905/joi.2002.319510.

[8] R. C. Merton, "On estimating the expected return on the market: An exploratory investigation," *Journal of Financial Economics*, vol. 8, no. 4, pp. 323–361, 1980. DOI: 10.1016/0304-405X(80)90007-0. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0304405X80900070.

[9] P. Jorion, "Bayes-Stein Estimation for Portfolio Analysis," *The Journal of Financial and Quantitative Analysis*, vol. 21, no. 3, p. 279, Sep. 1986, ISSN: 00221090. DOI: 10.2307/2331042. [Online]. Available: https://www.jstor.org/stable/2331042?origin=crossref.

[10] F. Black and R. Litterman, "Global Portfolio Optimization," en, *Financial Analysts Journal*, vol. 48, no. 5, pp. 28–43, Sep. 1992, ISSN: 0015-198X, 1938-3312. DOI: 10.2469/faj.v48.n5.28. [Online]. Available: https://www.tandfonline.com/doi/full/10.2469/faj.v48.n5.28.

[11] R. A. Brealey, *An introduction to risk and return from common stocks*, eng, 5. printing. Cambridge, Mass.: MIT Press, 1973, ISBN: 978-0-262-02047-3.

[12] C. Krauss, "STATISTICAL ARBITRAGE PAIRS TRADING STRATEGIES: REVIEW AND OUTLOOK," en, *Journal of Economic Surveys*, vol. 31, no. 2, pp. 513–545, Apr. 2017, ISSN: 0950-0804, 1467-6419. DOI: 10.1111/joes.12153. [Online]. Available: https://onlinelibrary.wiley.com/doi/10.1111/joes.12153.

[13] H. Rad, R. K. Y. Low, and R. Faff, "The profitability of pairs trading strategies: Distance, cointegration and copula methods," en, *Quantitative Finance*, vol. 16, no. 10, pp. 1541–1558, Oct. 2016, ISSN: 1469-7688, 1469-7696. DOI: 10.1080/14697688.2016.1164337. [Online]. Available: https://www.tandfonline.com/doi/full/10.1080/14697688.2016.1164337.

[14] N. Huck and K. Afawubo, "Pairs trading and selection methods: Is cointegration superior?" en, *Applied Economics*, vol. 47, no. 6, pp. 599–613, Feb. 2015, ISSN: 0003-6846, 1466-4283. DOI: 10.1080/00036846.2014.975417. [Online]. Available: http://www.tandfonline.com/doi/abs/10.1080/00036846.2014.975417.

[15] B. Do and R. Faff, "Does Simple Pairs Trading Still Work?" en, *Financial Analysts Journal*, vol. 66, no. 4, pp. 83–95, Jul. 2010, ISSN: 0015-198X, 1938-3312. DOI: 10.2469/faj.v66.n4.1. [Online]. Available: https://www.tandfonline.com/doi/full/10.2469/faj.v66.n4.1.

[16] C. Granger and P. Newbold, "Spurious regressions in econometrics," en, *Journal of Econometrics*, vol. 2, no. 2, pp. 111–120, Jul. 1974, ISSN: 03044076. DOI: 10.1016/0304-4076(74)90034-7. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/0304407674900347.

[17] J. D. Hamilton, *Time series analysis*, eng. Princeton, N.J: Princeton University Press, 1994, ISBN: 978-0-691-04289-3.

[18] S. Johansen, "Estimation and Hypothesis Testing of Cointegration Vectors in Gaussian Vector Autoregressive Models," *Econometrica*, vol. 59, no. 6, p. 1551, Nov. 1991, ISSN: 00129682. DOI: 10.2307/2938278. [Online]. Available: https://www.jstor.org/stable/2938278?origin=crossref.

[19] G. C. Chow, "Tests of Equality Between Sets of Coefficients in Two Linear Regressions," *Econometrica*, vol. 28, no. 3, p. 591, Jul. 1960, ISSN: 00129682. DOI: 10.2307/1910133. [Online]. Available: https://www.jstor.org/stable/1910133?origin=crossref.

[20] J.-Y. Lu et al., "Structural break-aware pairs trading strategy using deep reinforcement learning," en, *The Journal of Supercomputing*, vol. 78, no. 3, pp. 3843–3882, Feb. 2022, ISSN: 0920-8542, 1573-0484. DOI: 10.1007/s11227-021-04013-x. [Online]. Available: https://link.springer.com/10.1007/s11227-021-04013-x.

[21] A. Beyer, A. A. Haug, and W. G. Dewald, "Structural Breaks, Cointegration and the Fisher Effect," en, *SSRN Electronic Journal*, 2009, ISSN: 1556-5068. DOI: 10.2139/ssrn.1333613. [Online]. Available: https://www.ssrn.com/abstract=1333613.

[22] E. E. Peters, *Fractal market analysis: applying chaos theory to investment and economics* (Wiley finance editions). New York: J. Wiley & Sons, 1994, ISBN: 978-0-471-58524-4.

[23] H. E. Hurst, "Long-Term Storage Capacity of Reservoirs," en, *Transactions of the American Society of Civil Engineers*, vol. 116, no. 1, pp. 770–799, Jan. 1951, ISSN: 0066-0604, 2690-4071. DOI: 10.1061/TACEAT.0006518. [Online]. Available: https://ascelibrary.org/doi/10.1061/TACEAT.0006518.

[24] A. Carbone, G. Castelli, and H. Stanley, "Time-dependent Hurst exponent in financial time series," en, *Physica A: Statistical Mechanics and its Applications*, vol. 344, no. 1-2, pp. 267–271, Dec. 2004, ISSN: 03784371. DOI: 10.1016/j.physa.2004.06.130. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0378437104009471.

[25] G. E. Uhlenbeck and L. S. Ornstein, "On the Theory of the Brownian Motion," en, *Physical Review*, vol. 36, no. 5, pp. 823–841, Sep. 1930, ISSN: 0031-899X. DOI: 10.1103/PhysRev.36.823. Accessed: Apr. 14, 2026. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRev.36.823.

[26] O. Vasicek, "An equilibrium characterization of the term structure," en, *Journal of Financial Economics*, vol. 5, no. 2, pp. 177–188, Nov. 1977, ISSN: 0304405X. DOI: 10.1016/0304-405X(77)90016-2. Accessed: Apr. 14, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/0304405X77900162.

[27] W. K. Bertram, "Optimal trading strategies for Itô diffusion processes," en, *Physica A: Statistical Mechanics and its Applications*, vol. 388, no. 14, pp. 2865–2873, Jul. 2009, ISSN: 03784371. DOI: 10.1016/j.physa.2009.04.004. Accessed: Apr. 14, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0378437109002829.

[28] W. K. Bertram, "Analytic solutions for optimal statistical arbitrage trading," en, *Physica A: Statistical Mechanics and its Applications*, vol. 389, no. 11, pp. 2234–2243, Jun. 2010, ISSN: 03784371. DOI: 10.1016/j.physa.2010.01.045. Accessed: Apr. 14, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0378437110001019.

[29] T. Leung, *Optimal mean reversion trading: mathematical analysis and practical applications*. New Jersey: World Scientific, 2016, ISBN: 978-981-4725-91-0.

[30] J. Zhang, T. Leung, and A. Aravkin, "Sparse mean-reverting portfolios via penalized likelihood optimization," en, *Automatica*, vol. 111, p. 108 651, Jan. 2020, ISSN: 00051098. DOI: 10.1016/j.automatica.2019.108651. Accessed: Apr. 14, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0005109819305126.

[31] K. Johansson, M. G. Ogut, M. Pelger, T. Schmelzer, and S. Boyd, "A Simple Method for Predicting Covariance Matrices of Financial Returns," en, *Foundations and Trends® in Econometrics*, vol. 12, no. 4, pp. 324–407, Nov. 2023, ISSN: 1551-3076, 1551-3084. DOI: 10.1561/0800000047. Accessed: Apr. 14, 2026. [Online]. Available: https://www.emerald.com/fteco/article/12/4/324/1320852/A-Simple-Method-for-Predicting-Covariance-Matrices.

[32] M. B. Rudoy and C. E. Rohrs, "Optimal mean-variance portfolio construction in cointegrated vector autoregressive systems," in *2008 American Control Conference*, Seattle, WA: IEEE, Jun. 2008, pp. 2510–2515, ISBN: 978-1-4244-2078-0. DOI: 10.1109/ACC.2008.4586868. Accessed: Apr. 14, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/4586868/.

[33] Y. Yamada and J. A. Primbs, "Model predictive control for optimal portfolios with cointegrated pairs of stocks," in *2012 IEEE 51st IEEE Conference on Decision and Control (CDC)*, Maui, HI, USA: IEEE, Dec. 2012, pp. 5705–5710, ISBN: 978-1-4673-2066-5. DOI: 10.1109/CDC.2012.6426072. Accessed: Apr. 14, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/6426072/.

[34] A. Naccarato, A. Pierini, and G. Ferraro, "Markowitz portfolio optimization through pairs trading cointegrated strategy in long-term investment," en, *Annals of Operations Research*, vol. 299, no. 1-2, pp. 81–99, Apr. 2021, ISSN: 0254-5330, 1572-9338. DOI: 10.1007/s10479-019-03225-y. Accessed: Apr. 14, 2026. [Online]. Available: http://link.springer.com/10.1007/s10479-019-03225-y.

[35] LSEG, *Lseg-data: LSEG Data Library for Python*, 2025. [Online]. Available: https://pypi.org/project/lseg-data/.

[36] R. Aroussi, *Yfinance: Download market data from Yahoo Finance's API*, 2026. [Online]. Available: https://github.com/ranaroussi/yfinance.

[37] S. E. Said and D. A. Dickey, "Testing for unit roots in autoregressive-moving average models of unknown order," en, *Biometrika*, vol. 71, no. 3, pp. 599–607, 1984, ISSN: 0006-3444, 1464-3510. DOI: 10.1093/biomet/71.3.599. [Online]. Available: https://academic.oup.com/biomet/article-lookup/doi/10.1093/biomet/71.3.599.

[38] J. G. MacKinnon, "Critical Values for Cointegration Tests," Queen's Economics Department, Queen's University, Working Paper 1227, 2010. [Online]. Available: https://www.econ.queensu.ca/sites/econ.queensu.ca/files/wpaper/qed_wp_1227.pdf.

[39] D. Kwiatkowski, P. C. Phillips, P. Schmidt, and Y. Shin, "Testing the null hypothesis of stationarity against the alternative of a unit root," en, *Journal of Econometrics*, vol. 54, no. 1-3, pp. 159–178, Oct. 1992, ISSN: 03044076. DOI: 10.1016/0304-4076(92)90104-Y. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/030440769290104Y.

[40] O. Ledoit and M. Wolf, "Robust performance hypothesis testing with the Sharpe ratio," en, *Journal of Empirical Finance*, vol. 15, no. 5, pp. 850–859, Dec. 2008, ISSN: 09275398. DOI: 10.1016/j.jempfin.2008.03.002. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0927539808000182.

[41] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. DOI: 10.1109/MCSE.2007.55.

[42] J. D. Garrett and H.-H. Peng, *SciencePlots: Publication-ready scientific figure styles for matplotlib*, 2021. DOI: 10.5281/zenodo.4106649. [Online]. Available: https://github.com/garrettj403/SciencePlots.

[43] P. T. Inc, *Collaborative data science*, Montreal, QC, 2015. [Online]. Available: https://plot.ly.

[44] The pandas development team, *Pandas-dev/pandas: Pandas*, Feb. 2020. DOI: 10.5281/zenodo.3509134. [Online]. Available: https://doi.org/10.5281/zenodo.3509134.

[45] S. Seabold and J. Perktold, "Statsmodels: Econometric and Statistical Modeling with Python," Austin, Texas, 2010, pp. 92–96. DOI: 10.25080/Majora-92bf1922-011. [Online]. Available: https://doi.curvenote.com/10.25080/Majora-92bf1922-011.

[46] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html.

[47] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020. DOI: 10.1038/s41592-019-0686-2. [Online]. Available: https://doi.org/10.1038/s41592-019-0686-2.

[48] Snowflake Inc., *Streamlit: A faster way to build and share data apps*, 2026. [Online]. Available: https://streamlit.io/.

[49] C. R. Harris et al., "Array programming with NumPy," en, *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, ISSN: 0028-0836, 1476-4687. DOI: 10.1038/s41586-020-2649-2. Accessed: Mar. 31, 2026. [Online]. Available: https://www.nature.com/articles/s41586-020-2649-2.

[50] Y. Karavias, P. K. Narayan, and J. Westerlund, "Structural Breaks in Interactive Effects Panels and the Stock Market Reaction to COVID-19," en, *Journal of Business & Economic Statistics*, vol. 41, no. 3, pp. 653–666, Jul. 2023, ISSN: 0735-0015, 1537-2707. DOI: 10.1080/07350015.2022.2053690. Accessed: Apr. 15, 2026. [Online]. Available: https://www.tandfonline.com/doi/full/10.1080/07350015.2022.2053690.

[51] A. M. Martins and S. Cró, "Airline stock markets reaction to the COVID-19 outbreak and vaccines: An event study," en, *Journal of Air Transport Management*, vol. 105, p. 102 281, Oct. 2022, ISSN: 09696997. DOI: 10.1016/j.jairtraman.2022.102281. Accessed: Apr. 15, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0969699722001004.

[52] A. T. Hossain, A.-A. Masum, and J. Xu, "COVID-19, a blessing in disguise for the Tech sector: Evidence from stock price crash risk," en, *Research in International Business and Finance*, vol. 65, p. 101 938, Apr. 2023, ISSN: 02755319. DOI: 10.1016/j.ribaf.2023.101938. Accessed: Apr. 15, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0275531923000648.

[53] E. P. Chan, *Algorithmic trading: winning strategies and their rationale* (Wiley trading series). Hoboken, New Jersey: Wiley, 2013, ISBN: 978-1-118-46014-6.

Appendix A

Use of Generative AI Tools

In accordance with UCL's guidelines on the use of generative AI (Category 2: assistive use), the following AI tools were used in the project:

- The author acknowledges the use of **Claude Code** (version Sonnet 4.6, Anthropic, https://claude.ai/chat) to assist with system design, planning the project repository structure, and outlining the analysis framework.
- The author acknowledges the use of **Perplexity AI** (version current as of April 2026, Perplexity AI, https://www.perplexity.ai) to support background research, including the identification of appropriate academic literature and the summarisation of research papers.
- The author acknowledges the use of **QuillBot** (version current as of April 2026, QuillBot, https://quillbot.com) to assist with paraphrasing and grammatical refinement of the written text.

All AI-assisted outputs were reviewed, critically evaluated, and edited by the author prior to inclusion. No AI tool was used to generate the technical analysis, empirical results, or conclusions presented in this report. The author takes full responsibility for the content, accuracy, and intellectual integrity of this work.

Appendix B

Project Plan

FYP Project Plan – Declan Loo

Project Idea:

- **To develop a system to estimate expected returns by predicting the spread between asset pairs to optimise a portfolio**

Research Question:

- Can spread prediction between cointegrated asset pairs provide more reliable return estimates for Modern Portfolio Theory optimisation than traditional historical mean approaches?

Research Hypothesis:

- Spread-based return estimation derived from cointegrated asset pairs produces superior portfolio allocations compared to historical mean-variance optimisation, measured by risk-adjusted returns and portfolio stability, in terms of Sharpe ratio, volatility reduction and maximum drawdown.

Project Motivations:

- The difficulty of accurately **estimating future returns** represents Modern Portfolio Theory's critical weakness, as historical averages are extremely noisy with standard errors often exceeding the estimates themselves.
- While extensive literature exists on pairs trading and cointegration-based strategies, few studies integrate spread prediction directly into MPT optimisation as a systematic approach to return estimation.
- This project addresses this gap by leveraging mean-reverting spreads between cointegrated asset pairs to generate more stable return signals for portfolio optimisation.

Project Objectives:

- Develop a spread-based return estimation approach using cointegration analysis of asset pairs that provides more stable signals than historical mean returns.
- Build an integrated portfolio optimisation system using Markowitz framework and validate through comprehensive back testing against traditional mean-variance approaches using Sharpe ratio, volatility reduction, and drawdown metrics.
- Create an interactive interface to demonstrate real-world applicability for portfolio managers.

Evaluation Metrics:

- Success will be measured through rigorous back-testing comparing spread-based portfolios against traditional mean-variance approaches using risk-adjusted performance metrics: Sharpe ratio, volatility reduction, and maximum drawdown.
- Robustness validation through out-of-sample testing and sensitivity analysis across different market regimes.

Expected Contributions:

- This research contributes to empirical finance by examining whether spread prediction between cointegrated asset pairs provides more reliable return estimates than historical means for portfolio optimisation.
- This research will extend econometric literature on cointegration-based return estimation beyond traditional pairs trading applications, integrating it into Modern Portfolio Theory framework.
- This research will provide a practical methodology for addressing MPT's return estimation problem, demonstrating how mean-reverting spreads can improve portfolio construction.

FYP Project Plan – Declan Loo

Timeline/Schedule:

- **Research** (Nov): Literature review on portfolio optimisation and cointegration methods, requirements definition, project plan submission
- **Data & Modelling** (Late Nov - Dec): Historical spread analysis, cointegration model development, interface design
- **Implementation** (Jan - Mid Feb): MPT optimiser development, interface implementation, system integration and testing, interim report submission
- **Testing & Validation** (Late Feb - Mar): Back-testing framework, performance analysis across market conditions, draft report preparation
- **Writing** (Apr): Final report writing, refinements, project submission

Project Challenges:

- Limited access to sentiment data sources (e.g., Bloomberg Terminal, RavenPack) which could enhance spread prediction accuracy.
- Difficulty anticipating unpredictable events e.g. geopolitical events, economic crises, pandemics, that increase data uncertainty and noise.
- Potential instability of cointegration relationships during structural breaks or regime changes in financial markets.
- Computational complexity in identifying optimal asset pairs from large universes of securities.
- Risk of overfitting when optimising spread prediction models on historical data.

Resources:

- **Data Sources:**
    - Financial Data: LSEG Workspace (Refinitiv Workspace)
    - Alternative Data: Yahoo Finance
- **Analytical Methods**: Python-based implementation for cointegration analysis, optimisation, and visualisation

Future Work:

- Potential extensions include machine learning techniques to generalise spread behaviour, real-time data integration via Server-Side Events to display stock tickers, expansion to alternative asset classes, and incorporation of sentiment data to enhance prediction accuracy.

Relevant Works:

- Brealey, Richard A. An Introduction to Risk and Return from Common Stocks. 2nd ed., MIT Press, 1983. Accessed 3 Nov. 2025.
- DeFusco, Richard A., et al. Quantitative Investment Analysis. 4th ed., CFA Institute/Wiley, 2015. Accessed 3 Nov. 2025.
- Engle, Robert F., and C. W. J. Granger. "Co-Integration and Error Correction: Representation, Estimation, and Testing." Econometrica, vol. 55, no. 2, 1987, pp. 251–76. JSTOR, https://doi.org/10.2307/1913236. Accessed 4 Nov. 2025.
- Markowitz, Harry M. Portfolio Selection: Efficient Diversification of Investments. Yale University Press, 1959. JSTOR, http://www.jstor.org/stable/j.ctt1bh4c8h. Accessed 3 Nov. 2025.

Appendix C

Interim Report

Interim Report

Name: Declan Loo

Project Title (Project Plan/Now): To develop a system to estimate expected returns by predicting the spread between asset pairs to optimise a portfolio

Internal Supervisor: Dr. Tomaso Aste

Progress made to date:

To date, I have completed the Research phase of the project, from October to mid-November, conducting literature reviews on portfolio optimisation and cointegration methods, while defining project requirements. I have made substantial progress in the Data & Modelling phase, completing historical spread analysis and cointegration modelling on selected equities, with preliminary backtesting conducted and dashboard interface design underway.

Research (October – mid November):

During the research phase, key literature was reviewed on portfolio optimisation, cointegration, and backtesting as empirical validation methods. Initial focus centred on Markowitz's *Portfolio Selection: Efficient Diversification of Investments,* which establishes Modern Portfolio Theory's (MPT) mean-variance framework for maximising expected returns while minimising risk through diversification. The literature review assessed MPT's foundational role in portfolio quantification alongside its limitations, notably sensitivity to noisy input estimates and restrictive assumptions, as acknowledged by Markowitz himself. Complementary sources included Brealey's *An Introduction to Risk and Return from Common Stocks*, analysing risk-return tradeoffs, and *Quantitative Investment Analysis*, covering fundamental and advanced quantitative techniques. This foundation informed subsequent investigation of MPT gaps, particularly return estimation instability, identifying cointegration-based spreads as a promising alternative for stable signals in portfolio optimisation.

Following portfolio optimisation analysis, expected return prediction feasibility was evaluated, revealing market volatility challenges that prompted the investigation of cointegrated asset pair spreads. Engle's *Co-Integration and Error Correction: Representation, Estimation, and Testing* introduced cointegration for examining long-term asset relationships. Subsequent literature review covered pairs trading applications, historical cointegration studies, strategy effectiveness evaluations, and Python

implementation examples on GitHub. Backtesting methodology research identified Sharpe ratio, volatility reduction, and maximum drawdown as key validation metrics for the portfolio system as empirical validation methods. Critical assessment examined these backtesting methods' advantages and disadvantages, alongside cointegration benefits and drawbacks for pairs trading, providing a foundation to address MPT gaps through stable return estimates aligned with the project hypothesis.

Data & Modelling (late-November to December):

Following the research phase, historical spread analysis has been conducted using a list of equities: Nvidia, Coca-Cola, PepsiCo, Amazon, Meta, ExxonMobil, JPMorgan Chase, Bank of America, and AMD. Price data for each equity was retrieved using Refinitiv's API and processed with Python. Employing the cointegration research conducted, a cointegration model was developed using Python scripts to identify the most suitable asset pairs for spread analysis. Following historical spread analysis, preliminary backtesting has been performed using the three methods researched (Sharpe ratio, volatility reduction, maximum drawdown), where pairs trading was simulated on historical data to assess performance. The initial backtesting metrics obtained were able to justify the effectiveness of cointegration as a method to estimate future returns.

Evaluation of Streamlit and Dash (Plotly) with Python has been completed for the data visualisation requirements of the dashboard component. Financial applications used online for portfolio investments, such as Trading 212, alongside professional platforms like Refinitiv Workspace, served as inspiration for the interactive portfolio monitoring and analytics features within the dashboard design. Portfolio allocation workflows were also defined to generate optimal weights given cointegrated asset pair spread predictions, providing the methodological foundation for subsequent user experience planning and interface development of the dashboard application.

Remaining work to be done before the final report deadline:

- Data & Modelling (Late Nov - Dec):
    - Finalise spread analysis with preliminary backtesting
    - Finish design of dashboard

- Implementation (Jan - Mid Feb):
    - MPT optimiser development
    - Interface implementation

    - System integration and testing

- Testing & Validation (Late Feb - Mar)
    - Back-testing framework across market regimes
    - Performance analysis across market conditions
    - Potential extensions detailed on Project Plan
    - Draft dissertation report

- Writing (Apr):
    - Final report writing
    - Refinements
    - Project submission

Appendix D

Supplementary Figures

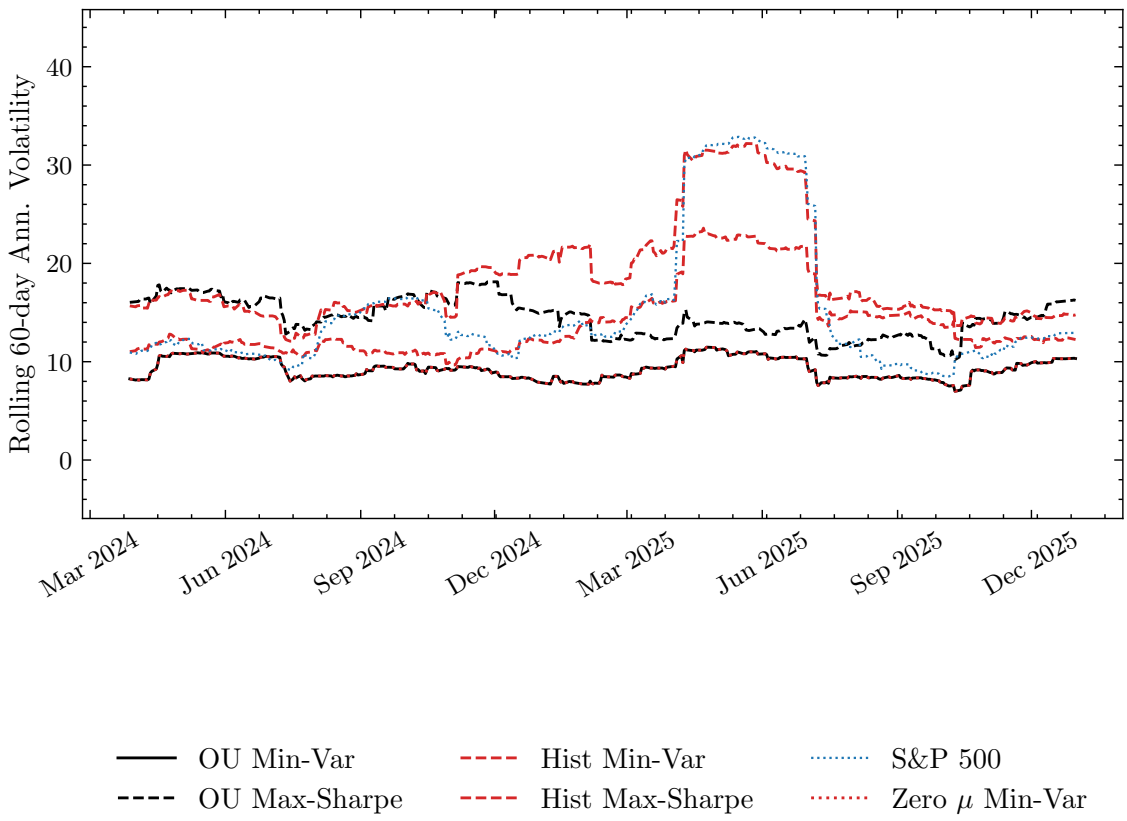

Figure D.1: Rolling 60-day annualised volatility over the OOS period (January 2024 – December 2025). The OU-Implied MPT maintains substantially lower and more stable volatility than the Historical MPT throughout the test period.

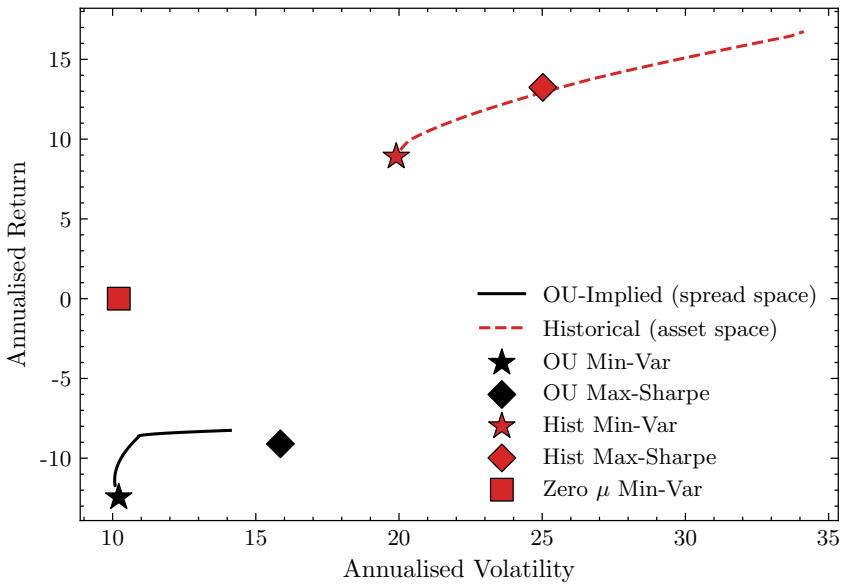

Figure D.2: Mean-variance efficient frontiers for both the OU-Implied (spread space) and Historical (asset space) portfolios. The two frontiers are not directly comparable as they operate over different asset universes. The minimum-variance portfolios used in the backtest are marked on each frontier.

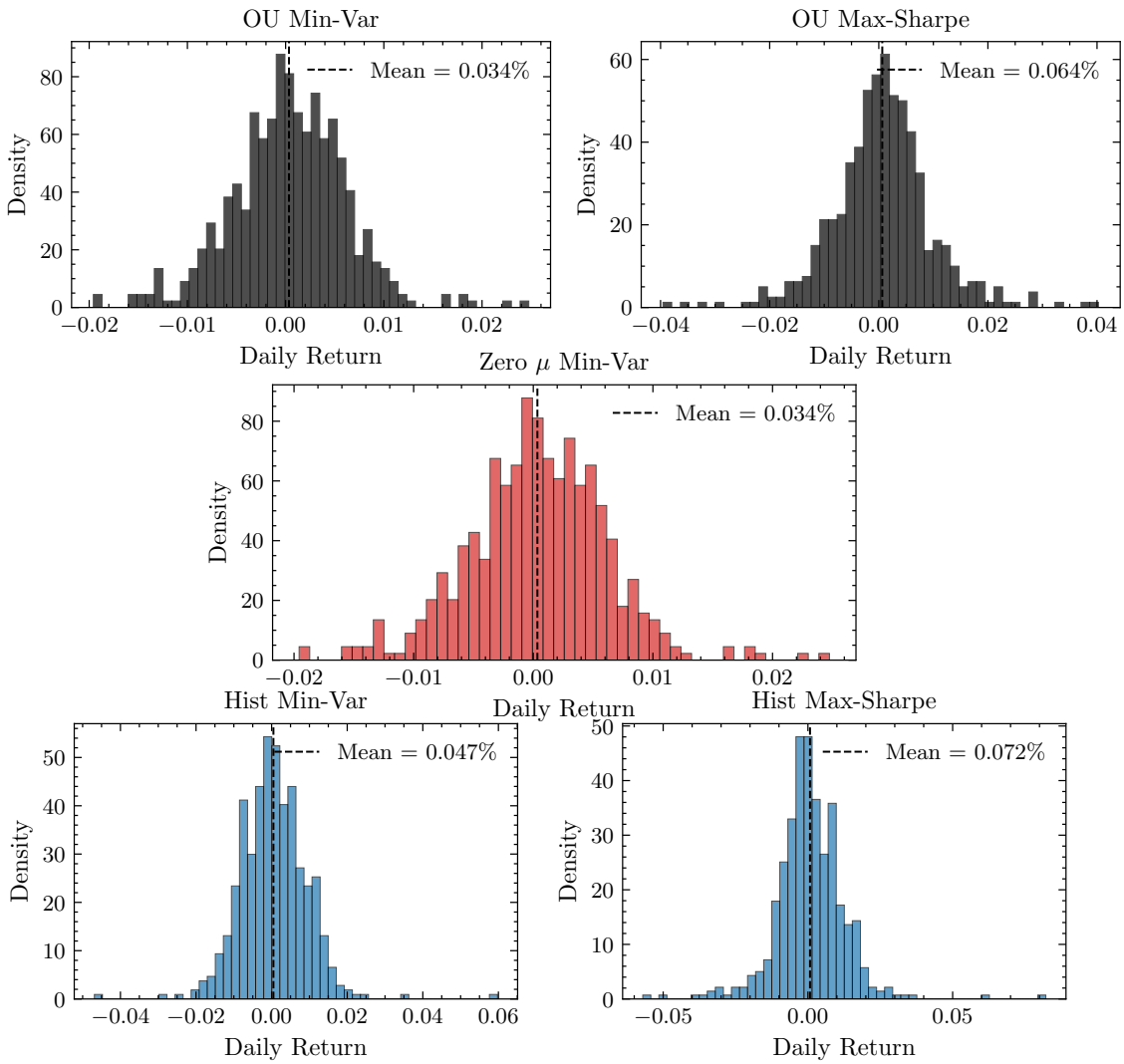

Figure D.3: Histogram of daily portfolio returns for the OU-Implied and Historical MPT portfolios over the OOS period (January 2024 – December 2025). The OU-Implied distribution is narrower and more concentrated around positive values, reflecting its lower volatility and higher mean return. (a) OU-Implied MPT; (b) Historical MPT.

Appendix E

Software Implementation

The full source code is available at https://github.com/Declan-Loo/Spread-Prediction-for-Portfolio-Optimisation.

E.1 Repository Structure

The repository is organised into four library packages, four analysis notebooks, and an interactive dashboard, all under the `src/` directory.

```
Spread-Prediction-for-Portfolio-Optimisation/
|-- data/
|   |-- raw/                        # Cached LSEG API price CSVs
|   `-- processed/                  # Processed outputs from analysis_notebooks/
|-- src/
|   |-- data/
|   |   |-- refinitiv_client.py     # LSEG Workspace API wrapper with caching
|   |   `-- yfinance_sp500.py       # S&P 500 benchmark (yfinance fallback)
|   |-- modelling/
|   |   |-- config.py               # Tickers, candidate pairs, date ranges
|   |   |-- cointegration.py        # Engle-Granger two-step test
|   |   |-- spread_analysis.py      # Spread, z-scores, half-life, Hurst
|   |   |-- return_estimation.py    # Historical, EWMA, OU-implied returns
|   |   `-- optimiser.py            # Min-variance, max-Sharpe, frontier
|   |-- backtesting/
|   |   |-- engine.py               # Z-score mean-reversion backtester
|   |   |-- metrics.py              # Sharpe, max drawdown, vol reduction
|   |   `-- benchmarks.py           # Risk-free, B&H, equal-weight, S&P 500
|   |-- dashboard/
|   |   |-- app.py                  # Streamlit application entry point
|   |   `-- components.py           # Plotly visualisation components
|   `-- analysis_notebooks/
|       |-- 01_spread_exploration.ipynb
|       |-- 02_cointegration_validation.ipynb
|       |-- 03_return_estimation.ipynb
|       |-- 04_backtest_results.ipynb
|       `-- figures/                # Exported PDF figures
|-- lseg-data.config.json           # LSEG session config (gitignored)
|-- requirements.txt
`-- README.md
```

E.2 Dependencies

The implementation requires Python 3.10+ with the following key libraries (see Chapter 4 for full details): `pandas`, `numpy`, `statsmodels`, `scipy`, `scikit-learn`, `matplotlib`,`yfinance`. A full list

of dependencies is provided in `requirements.txt`.

E.3 Reproducing Results

Complete step-by-step instructions for installation, LSEG data configuration, and running the analysis notebooks and dashboard are provided in Appendix E.4 (System Manual). All figures are exported automatically to `src/analysis_notebooks/figures/` as PDF upon notebook execution.

E.4 System Manual

E.4.1 Prerequisites

- Python 3.10 or later (3.11 recommended).
- A valid `lseg-data.config.json` session configuration file in the project root (see Section E.4.3). Without this file the data client falls back to any locally cached CSVs in `data/raw/`.
- Git (for cloning the repository).

E.4.2 Installation

Clone the repository and install the pinned dependencies into a virtual environment:

```
git clone <repo-url>
cd Spread-Prediction-for-Portfolio-Optimisation
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

All dependencies are pinned in `requirements.txt`; installing into an isolated virtual environment is strongly recommended to avoid version conflicts.

E.4.3 LSEG Data Configuration

The file `lseg-data.config.json` must be placed in the project root before running any notebook that fetches live prices. It is excluded from version control (`.gitignore`) because it contains API session credentials. A template is available from LSEG Workspace under *Settings* → *Developer Tools* → *Python SDK*. The relevant fields are:

```
{
  "sessions": {
    "default": "desktop.workspace",
    "desktop": {
      "workspace": {
        "app-key": "<YOUR_APP_KEY>"
      }
    }
  }
}
```

Once the file is present and the LSEG Workspace desktop application is running, the `refinitiv_client.py` wrapper opens a session automatically on import.

E.4.4 Running the Analysis Notebooks

Execute the notebooks in the following order from the `src/analysis_notebooks/` directory (or from the project root using the paths below):

1. `src/analysis_notebooks/01_spread_exploration.ipynb` — loads prices, screens candidate pairs for cointegration, characterises spreads.
2. `src/analysis_notebooks/02_cointegration_validation.ipynb` — validates cointegration on the out-of-sample period using a rolling window.
3. `src/analysis_notebooks/03_return_estimation.ipynb` — computes historical, EWMA, and OU-implied expected returns and covariance matrices.
4. `src/analysis_notebooks/04_backtest_results.ipynb` — runs the z-score backtester, constructs MPT portfolios, and benchmarks performance.

Each notebook exports all figures to `src/analysis_notebooks/figures/` as PDF files automatically. Notebooks are self-contained; re-running notebook n does not require re-running earlier notebooks provided the intermediate data files in `data/raw/` are present.

E.4.5 Running the Dashboard

The interactive Streamlit dashboard can be launched from the project root:

```
streamlit run src/dashboard/app.py
```

The dashboard opens in the default browser at `http://localhost:8501`. It requires the same `lseg-data.config.json` credentials and will use cached CSVs if the live API is unavailable.

E.4.6 Extending the Codebase

The library packages under `src/` are importable Python modules with no circular dependencies. The dependency graph is strictly layered:

```
modelling/config.py
    <- modelling/cointegration.py
    <- modelling/spread_analysis.py
        <- modelling/return_estimation.py
            <- modelling/optimiser.py
                <- backtesting/engine.py
                <- backtesting/metrics.py
                <- backtesting/benchmarks.py
```

To add a new candidate pair, edit `modelling/config.py` and append the two ticker RICs to the `CANDIDATE_PAIRS` list. To add a new return estimator, implement it in `return_estimation.py` following the same signature as `historical_mean_return` (returns a `pd.Series` of annualised returns indexed by spread label), then pass it as the `expected_returns` argument to `optimiser.optimise_portfolio`.

E.5 Code Listings

The following listings present the core algorithmic components, implemented in Python. Boilerplate and visualisation code are omitted; the full source is available in the project repository.

E.5.1 Cointegration Screening (cointegration.py)

```
def adf_test(series: pd.Series, significance: float = 0.05) -> dict:
    series_clean = series.dropna()
    result = adfuller(series_clean, autolag="AIC")
    return {
        "adf_stat":        result[0],
        "p_value":         result[1],
        "used_lag":        result[2],
        "n_obs":           result[3],
        "critical_values": result[4],
        "is_stationary":   result[1] < significance,
    }


def is_I1(series: pd.Series, significance: float = 0.05) -> dict:
    """
    Check if a series is plausibly I(1):
    non-stationary in levels, stationary in first differences.
    """
    level_result = adf_test(series, significance=significance)
    diff_result  = adf_test(series.diff(), significance=significance)
    return {
        "level":  level_result,
        "diff":   diff_result,
        "is_I1":  (not level_result["is_stationary"])
                   and diff_result["is_stationary"],
    }


def engle_granger_test(
    y: pd.Series, x: pd.Series, significance: float = 0.05
) -> dict:
    """
    Engle-Granger two-step cointegration test.
    Step 1: OLS regression  y = a + b*x + e  (b is the hedge ratio).
    Step 2: ADF test on residuals e via statsmodels coint().
    """
    aligned = pd.concat([y, x], axis=1).dropna().astype(float)
    y_clean, x_clean = aligned.iloc[:, 0], aligned.iloc[:, 1]

    y_I1 = is_I1(y_clean, significance=significance)
    x_I1 = is_I1(x_clean, significance=significance)

    X = sm.add_constant(x_clean)
    model = sm.OLS(y_clean, X).fit()
    intercept, hedge_ratio = model.params

    coint_stat, p_value, critical_values = coint(
        y_clean, x_clean, trend="c", method="aeg", autolag="AIC"
    )

    return {
        "hedge_ratio":    float(hedge_ratio),
        "intercept":      float(intercept),
        "adf_stat":       float(coint_stat),
        "p_value":        float(p_value),
        "critical_values": critical_values,
        "is_cointegrated": bool(
            p_value < significance
            and y_I1["is_I1"] and x_I1["is_I1"]
        ),
    }


def screen_pairs(
    prices_df: pd.DataFrame,
    candidate_pairs: list[tuple[str, str]],
    significance: float = 0.05,
) -> pd.DataFrame:
    """
    Run Engle-Granger on all candidate pairs, testing both orderings
    (y~x and x~y) and keeping the result with the lower p-value.
    """
    rows = []
    for ticker_y, ticker_x in candidate_pairs:
        y, x = prices_df[ticker_y], prices_df[ticker_x]
        result_yx = engle_granger_test(y, x, significance)
        result_xy = engle_granger_test(x, y, significance)

        if result_yx["p_value"] <= result_xy["p_value"]:
            best, dep, indep = result_yx, ticker_y, ticker_x
        else:
            best, dep, indep = result_xy, ticker_x, ticker_y
```

```
        rows.append({
            "y": dep, "x": indep,
            "hedge_ratio":    best["hedge_ratio"],
            "intercept":      best["intercept"],
            "adf_stat":       best["adf_stat"],
            "p_value":        best["p_value"],
            "is_cointegrated": best["is_cointegrated"],
        })

    return pd.DataFrame(rows).sort_values("p_value").reset_index(drop=True)
```

E.5.2 Spread Characterisation (spread_analysis.py)

```
def compute_spread(
    y: pd.Series, x: pd.Series,
    hedge_ratio: float, intercept: float = 0.0,
) -> pd.Series:
    spread = y.astype(float) - hedge_ratio * x.astype(float) - intercept
    spread.name = "spread"
    return spread


def compute_zscore(spread: pd.Series, window: int = 60) -> pd.Series:
    rolling_mean = spread.rolling(window=window).mean()
    rolling_std  = spread.rolling(window=window).std(ddof=1)
    return (spread - rolling_mean) / rolling_std


def compute_half_life(spread: pd.Series) -> float:
    """
    Half-life of mean reversion via OLS on the AR(1) form:
        delta_s = lambda * s_{t-1} + noise
    Half-life = -ln(2) / lambda.
    Returns np.inf if the spread does not appear mean-reverting.
    """
    spread_clean = spread.dropna()
    lag   = spread_clean.shift(1).iloc[1:]
    delta = spread_clean.diff().iloc[1:]

    model = sm.OLS(delta, sm.add_constant(lag)).fit()
    lam = model.params.iloc[1]
    return float(-np.log(2) / lam) if lam < 0 else np.inf


def compute_hurst_exponent(series: pd.Series, max_lag: int = 100) -> float:
    """
    Hurst exponent via the variogram method.
    Fits log(std(increments)) = H * log(lag) + c.
    H < 0.5 -> mean-reverting, H ~ 0.5 -> random walk, H > 0.5 -> trending.
    """
    v = series.dropna().values
    n = len(v)
    max_lag = min(max_lag, n // 2)
    lags = range(2, max_lag + 1)
    tau  = [np.std(v[lag:] - v[:-lag]) for lag in lags]
    poly = np.polyfit(np.log(list(lags)), np.log(tau), 1)
    return float(poly[0])
```

E.5.3 OU-Implied Return Estimator (return_estimation.py)

```
def ou_implied_spread_return(
    spread: pd.Series,
    half_life: float,
    window: int = 60,
    annualise: bool = True,
    periods_per_year: int = 252,
    normalisation: str = "direct",
) -> float:
    """
    OU-implied expected return for a mean-reverting spread.

    Model:  dS = kappa*(theta - S)*dt + sigma*dW
    Expected one-step change:
        E[delta_S] = (theta - S_t) * (1 - exp(-kappa))
    where kappa = ln(2) / half_life.

    With the "direct" normalisation, E[delta_S] is a daily log-return in
    the same units as compute_spread_returns(), so no further scaling is
    needed before passing to the optimiser.
    """
    if half_life <= 0 or np.isinf(half_life):
```

```
        return 0.0

    kappa   = np.log(2) / half_life
    rolling = spread.rolling(window=window)
    theta   = rolling.mean().iloc[-1]
    s_t     = spread.iloc[-1]

    if np.isnan(theta) or np.isnan(s_t):
        return 0.0

    expected_delta = (theta - s_t) * (1 - np.exp(-kappa))

    if normalisation == "std":
        sigma = rolling.std().iloc[-1]
        daily_return = 0.0 if (np.isnan(sigma) or sigma <= 0) \
                       else expected_delta / sigma
    else:  # "direct"
        daily_return = expected_delta

    return float(daily_return * periods_per_year) if annualise \
           else float(daily_return)


def build_ou_implied_returns(
    prices_df: pd.DataFrame,
    coint_pairs: pd.DataFrame,
    window: int = 60,
    annualise: bool = True,
    periods_per_year: int = 252,
    normalisation: str = "direct",
) -> pd.Series:
    """
    Compute OU-implied returns for all cointegrated pairs.
    Log-transforms prices so that E[delta_S] is a log-return directly
    comparable to compute_spread_returns().
    """
    log_prices = np.log(prices_df)
    results = {}
    for _, row in coint_pairs.iterrows():
        label  = f"{row['y']}_vs_{row['x']}"
        spread = compute_spread(
            log_prices[row["y"]], log_prices[row["x"]],
            row["hedge_ratio"], row.get("intercept", 0.0),
        )
        hl = compute_half_life(spread)
        results[label] = ou_implied_spread_return(
            spread, hl, window=window,
            annualise=annualise, periods_per_year=periods_per_year,
            normalisation=normalisation,
        )
    return pd.Series(results, name="ou_implied_return")
```

E.5.4 Markowitz Optimiser (`optimiser.py`)

```
def _portfolio_stats(
    weights: np.ndarray,
    mean_returns: np.ndarray,
    cov_matrix: np.ndarray,
    periods_per_year: int = 252,
) -> tuple[float, float]:
    """Annualised arithmetic return and volatility for a weight vector."""
    log_port_return = np.dot(weights, mean_returns) * periods_per_year
    port_return = np.expm1(log_port_return)   # compound log -> arithmetic
    port_vol    = np.sqrt(
        np.dot(weights, np.dot(cov_matrix, weights)) * periods_per_year
    )
    return port_return, port_vol


def minimum_variance_weights(
    returns: pd.DataFrame,
    cov_matrix: pd.DataFrame | np.ndarray | None = None,
    l2_reg: float = 0.0,
) -> np.ndarray:
    """Minimum-variance portfolio (long-only, fully invested)."""
    n = returns.shape[1]
    _, cov = _resolve_inputs(returns, cov_matrix=cov_matrix)

    def objective(w):
        return np.dot(w, np.dot(cov, w)) + l2_reg * np.dot(w, w)

    result = minimize(
        objective, np.ones(n) / n,
```

```
        method="SLSQP",
        bounds=[(0.0, 1.0)] * n,
        constraints={"type": "eq", "fun": lambda w: np.sum(w) - 1},
        options={"ftol": 1e-12, "maxiter": 1000},
    )
    return result.x


def maximum_sharpe_weights(
    returns: pd.DataFrame,
    expected_returns: pd.Series | np.ndarray | None = None,
    cov_matrix: pd.DataFrame | np.ndarray | None = None,
    rf_annual: float = 0.02,
    periods_per_year: int = 252,
    l2_reg: float = 0.0,
) -> np.ndarray:
    """Maximum Sharpe ratio (tangency) portfolio (long-only)."""
    n = returns.shape[1]
    mean_ret, cov = _resolve_inputs(returns, expected_returns, cov_matrix)

    def neg_sharpe(w):
        port_ret, port_vol = _portfolio_stats(w, mean_ret, cov, periods_per_year)
        if port_vol == 0:
            return 1e6
        return -(port_ret - rf_annual) / port_vol + l2_reg * np.dot(w, w)

    result = minimize(
        neg_sharpe, np.ones(n) / n,
        method="SLSQP",
        bounds=[(0.0, 1.0)] * n,
        constraints={"type": "eq", "fun": lambda w: np.sum(w) - 1},
        options={"ftol": 1e-12, "maxiter": 1000},
    )
    return result.x
```

E.5.5 Pairs Backtesting Engine (engine.py)

```
@dataclass
class BacktestConfig:
    entry_z:              float = 2.0
    exit_z:               float = 0.0
    stop_loss_z:          float = 4.0
    lookback_window:      int   = 60
    transaction_cost_bps: float = 10.0
    initial_capital:      float = 100_000.0

    def __post_init__(self):
        if self.exit_z >= self.entry_z:
            raise ValueError("exit_z must be < entry_z")
        if self.stop_loss_z <= self.entry_z:
            raise ValueError("stop_loss_z must be > entry_z")


class PairsBacktestEngine:

    def _generate_signals(self, zscore: pd.Series) -> pd.Series:
        """
        Position logic:
          +1  long spread  (z < -entry_z, expecting reversion upward)
          -1  short spread (z > +entry_z, expecting reversion downward)
           0  flat
        Stop-loss forces flat whenever |z| >= stop_loss_z.
        """
        cfg = self.config
        positions = pd.Series(0.0, index=zscore.index, name="position")
        pos = 0.0
        for i in range(len(zscore)):
            z = zscore.iloc[i]
            if np.isnan(z):
                positions.iloc[i] = 0.0
                continue
            if abs(z) >= cfg.stop_loss_z and pos != 0.0:
                pos = 0.0
            elif pos == 1.0  and z >= -cfg.exit_z:
                pos = 0.0
            elif pos == -1.0 and z <=  cfg.exit_z:
                pos = 0.0
            elif pos == 0.0  and z <= -cfg.entry_z:
                pos = 1.0
            elif pos == 0.0  and z >=  cfg.entry_z:
                pos = -1.0
            positions.iloc[i] = pos
        return positions
```

```
    def _compute_pnl(
        self,
        y_prices: pd.Series,
        x_prices: pd.Series,
        positions: pd.Series,
        hedge_ratio: float,
    ) -> pd.DataFrame:
        """
        Mark-to-market PnL with proportional transaction costs.
        Full initial capital is deployed into each spread unit:
            n_units = initial_capital / (y_price + |beta| * x_price)
        Daily return:
            r_t = pos_{t-1} * n_units_{t-1} * (dy_t - beta*dx_t) / capital
        """
        cfg = self.config
        dy = y_prices.diff()
        dx = x_prices.diff()
        notional_per_unit = y_prices + abs(hedge_ratio) * x_prices
        n_units  = cfg.initial_capital / notional_per_unit
        spread_pnl = (
            positions.shift(1) * n_units.shift(1) * (dy - hedge_ratio * dx)
        ).fillna(0.0)
        tc = (
            positions.diff().abs().fillna(0.0)
            * cfg.initial_capital
            * (cfg.transaction_cost_bps / 10_000)
        )
        net_pnl = spread_pnl - tc
        return pd.DataFrame({
            "position":         positions,
            "spread_pnl":       spread_pnl,
            "transaction_cost": tc,
            "net_pnl":          net_pnl,
            "strategy_return":  net_pnl / cfg.initial_capital,
        }, index=y_prices.index)
```

E.5.6 Efficient Frontier & Spread-vs-Asset Comparison (optimiser.py, return_estimation.py)

```
def compute_efficient_frontier(
    returns: pd.DataFrame,
    expected_returns: pd.Series | np.ndarray | None = None,
    cov_matrix: pd.DataFrame | np.ndarray | None = None,
    n_points: int = 50,
    rf_annual: float = 0.02,
    periods_per_year: int = 252,
    l2_reg: float = 0.0,
) -> pd.DataFrame:
    """
    For a range of target returns (anchored at the min-variance return
    and the single-asset maximum), find the minimum-volatility portfolio
    at each target via SLSQP with a return-equality constraint.
    Returns a DataFrame of (return, volatility, sharpe) for plotting.
    """
    n = returns.shape[1]
    mean_ret, cov = _resolve_inputs(returns, expected_returns, cov_matrix)

    w_min = minimum_variance_weights(returns, cov_matrix, l2_reg=l2_reg)
    ret_min, _ = _portfolio_stats(w_min, mean_ret, cov, periods_per_year)
    ret_max = float(np.max(mean_ret)) * periods_per_year
    if ret_max < ret_min:          # guard: all expected returns negative
        ret_min, ret_max = ret_max, ret_min

    def _min_var_obj(w):
        return np.dot(w, np.dot(cov, w)) + l2_reg * np.dot(w, w)

    records = []
    for target in np.linspace(ret_min, ret_max, n_points):
        constraints = [
            {"type": "eq", "fun": lambda w: np.sum(w) - 1},
            {"type": "eq",
             "fun": lambda w, t=target:
                 np.dot(w, mean_ret) * periods_per_year - t},
        ]
        result = minimize(
            _min_var_obj, np.ones(n) / n, method="SLSQP",
            bounds=[(0.0, 1.0)] * n, constraints=constraints,
            options={"ftol": 1e-12, "maxiter": 1000},
        )
        if result.success:
            port_ret, port_vol = _portfolio_stats(
```

```
                result.x, mean_ret, cov, periods_per_year
            )
            sharpe = (port_ret - rf_annual) / port_vol if port_vol > 0 \
                     else np.nan
            records.append(
                {"return": port_ret, "volatility": port_vol, "sharpe": sharpe}
            )
    return pd.DataFrame(records)


def spread_vs_asset_estimates(
    prices_df: pd.DataFrame,
    coint_pairs: pd.DataFrame,
    method: str = "historical",
    window: int | None = None,
    span: int = 60,
    annualise: bool = True,
    periods_per_year: int = 252,
    cov_estimator: str = "sample",
) -> dict:
    """
    Side-by-side expected-return and covariance estimates for spread-based
    and traditional (asset-level) Markowitz formulations.

    Asset returns are computed as log-returns for consistency with
    compute_spread_returns() (which is a log-return differential).
    Both sides use the same estimator choice (historical, EWMA, or OU;
    sample, Ledoit-Wolf).
    """
    spread_rets = build_spread_return_matrix(prices_df, coint_pairs)
    tickers = list(dict.fromkeys(
        coint_pairs["y"].tolist() + coint_pairs["x"].tolist()
    ))
    asset_rets = np.log(prices_df[tickers]).diff().dropna(how="all")

    if method == "ewma":
        spread_mu = ewma_mean_return(spread_rets.dropna(), span=span,
            annualise=annualise, periods_per_year=periods_per_year)
        asset_mu  = ewma_mean_return(asset_rets.dropna(),  span=span,
            annualise=annualise, periods_per_year=periods_per_year)
    elif method == "ou":
        spread_mu = build_ou_implied_returns(prices_df, coint_pairs,
            window=window or 60, annualise=annualise,
            periods_per_year=periods_per_year)
        asset_mu  = historical_mean_return(asset_rets.dropna(),
            window=window, annualise=annualise,
            periods_per_year=periods_per_year)
    else:  # "historical"
        spread_mu = historical_mean_return(spread_rets.dropna(),
            window=window, annualise=annualise,
            periods_per_year=periods_per_year)
        asset_mu  = historical_mean_return(asset_rets.dropna(),
            window=window, annualise=annualise,
            periods_per_year=periods_per_year)

    cov_fn = (
        (lambda r: shrinkage_covariance(r, annualise=annualise,
             periods_per_year=periods_per_year, estimator=cov_estimator))
        if cov_estimator == "lw"
        else (lambda r: sample_covariance(r, annualise=annualise,
              periods_per_year=periods_per_year))
    )
    return {
        "spread_returns": spread_rets,
        "spread_mu":      spread_mu,
        "spread_cov":     cov_fn(spread_rets.dropna()),
        "asset_returns":  asset_rets,
        "asset_mu":       asset_mu,
        "asset_cov":      cov_fn(asset_rets.dropna()),
    }
```

E.5.7 Performance Metrics (`metrics.py`)

```
def compute_ex_post_sharpe_ratio(
    returns: pd.Series,
    rf_annual: float = 0.02,
    periods_per_year: int = 252,
) -> float:
    """
    Sharpe (1994) ex-post Sharpe ratio on realised returns.
    Daily excess returns are computed by subtracting the daily risk-free
    rate; the ratio is annualised by sqrt(periods_per_year).
    """
```

```
    if len(returns) < 2:
        raise ValueError("Need at least 2 returns for std dev")
    excess   = returns - (rf_annual / periods_per_year)
    mean_exc = excess.mean()
    std_exc  = excess.std(ddof=1)
    if std_exc == 0:
        return np.nan
    return (mean_exc / std_exc) * np.sqrt(periods_per_year)


def compute_max_drawdown(returns: pd.Series) -> float:
    """
    Maximum drawdown: the largest peak-to-trough decline in the
    compounded equity curve.  Returns a negative fraction.
    """
    if len(returns) < 2:
        raise ValueError("Need at least 2 returns to compute drawdown")
    cum         = (1 + returns).cumprod()
    running_max = cum.cummax()
    drawdowns   = (cum - running_max) / running_max
    return float(drawdowns.min())


def compute_volatility_reduction(
    portfolio_returns: pd.Series,
    benchmark_returns: pd.Series,
    periods_per_year: int = 252,
) -> float:
    """
    Fractional annualised volatility reduction vs a benchmark.
        vol_reduction = 1 - sigma_portfolio / sigma_benchmark
    Positive => portfolio is less volatile; negative => more volatile.
    """
    sigma_p = portfolio_returns.std(ddof=1) * np.sqrt(periods_per_year)
    sigma_b = benchmark_returns.std(ddof=1) * np.sqrt(periods_per_year)
    if sigma_b == 0:
        raise ValueError("Benchmark volatility is zero")
    return float(1 - (sigma_p / sigma_b))
```

E.5.8 Benchmark Suite (benchmarks.py)

```
def compute_benchmark_metrics(
    returns: pd.Series,
    rf_annual: float = 0.02,
    periods_per_year: int = 252,
    benchmark_returns: pd.Series | None = None,
) -> dict:
    """
    Compute standard metrics for any return series.  If benchmark_returns
    is provided, also computes volatility_reduction aligned to the common
    trading calendar (inner join on the index).
    """
    ret = returns.dropna()
    out = {
        "sharpe_ratio":          compute_ex_post_sharpe_ratio(
                                     ret, rf_annual, periods_per_year),
        "max_drawdown":          compute_max_drawdown(ret),
        "total_return":          float((1 + ret).prod() - 1),
        "annualised_volatility": float(ret.std(ddof=1)
                                       * np.sqrt(periods_per_year)),
        "volatility_reduction":  np.nan,
    }
    if benchmark_returns is not None:
        common = ret.index.intersection(benchmark_returns.dropna().index)
        if len(common) >= 2:
            try:
                out["volatility_reduction"] = compute_volatility_reduction(
                    ret.reindex(common).dropna(),
                    benchmark_returns.reindex(common).dropna(),
                    periods_per_year,
                )
            except Exception:
                pass
    return out


def historical_mpt_returns(
    train_prices: pd.DataFrame,
    test_prices: pd.DataFrame,
    rf_annual: float = 0.02,
    periods_per_year: int = 252,
) -> pd.Series:
    """
```

```
    Out-of-sample returns of the historical max-Sharpe (traditional MPT)
    portfolio.  Weights are estimated on train_prices using the sample mean
    and covariance of asset returns; fixed weights are then applied to
    test_prices.  Provides the classical Markowitz benchmark for comparison
    with the spread-based strategy.
    """
    train_ret = train_prices.pct_change().dropna()
    mean_ret  = train_ret.mean().values
    cov       = train_ret.cov().values
    n         = len(mean_ret)

    def neg_sharpe(w):
        port_ret = np.dot(w, mean_ret) * periods_per_year
        port_vol = np.sqrt(np.dot(w, np.dot(cov, w)) * periods_per_year)
        return -(port_ret - rf_annual) / port_vol if port_vol > 0 else 1e6

    res = minimize(
        neg_sharpe, np.ones(n) / n, method="SLSQP",
        bounds=[(0.0, 1.0)] * n,
        constraints={"type": "eq", "fun": lambda w: np.sum(w) - 1},
    )
    port_ret = (test_prices[train_prices.columns].pct_change() * res.x
                ).sum(axis=1)
    port_ret.name = "historical_mpt"
    return port_ret.dropna()
```

E.5.9 Trade Logging & Aggregate Metrics (engine.py)

```
def _build_trade_log(
    self,
    positions: pd.Series,
    zscore: pd.Series,
) -> pd.DataFrame:
    """
    Build an event-level trade log from position transitions.
    Each row records the date, direction (entry_long, entry_short, exit),
    the resulting position, and the z-score at that moment.
    """
    changes    = positions.diff().fillna(0.0)
    records    = []
    for date, change in changes[changes != 0.0].items():
        pos_after = positions.loc[date]
        if   pos_after == 0.0:  direction = "exit"
        elif change     > 0:    direction = "entry_long"
        else:                   direction = "entry_short"
        records.append({
            "date":           date,
            "direction":      direction,
            "position_after": pos_after,
            "zscore":         zscore.loc[date]
                              if date in zscore.index else np.nan,
        })
    return pd.DataFrame(records)


def _compute_metrics(
    self,
    daily_returns: pd.Series,
    trades: pd.DataFrame,
    benchmark_returns: pd.Series | None = None,
) -> dict:
    """
    Aggregate performance metrics over the backtest period.

    Win rate is approximated from daily active returns (non-zero position
    days) to avoid double-counting partial holding periods.
    Profit factor = gross profits / gross losses on active days.
    Volatility reduction is measured relative to the equal-weight
    buy-and-hold of the two spread legs.
    """
    valid  = daily_returns.dropna()
    active = valid[valid != 0.0]
    wins   = (active > 0).sum()
    losses = (active < 0).sum()
    win_rate = float(wins / (wins + losses)) if (wins + losses) > 0 \
               else np.nan
    profit_factor = (
        float(active[active > 0].sum() / abs(active[active < 0].sum()))
        if (active < 0).any() else np.inf
    )
    vol_red = np.nan
    if benchmark_returns is not None:
        common = valid.index.intersection(benchmark_returns.dropna().index)
```

```
        if len(common) >= 2:
            try:
                vol_red = compute_volatility_reduction(
                    valid.reindex(common), benchmark_returns.reindex(common)
                )
            except (ValueError, ZeroDivisionError):
                pass
    return {
        "sharpe_ratio":          compute_ex_post_sharpe_ratio(valid),
        "max_drawdown":          compute_max_drawdown(valid),
        "total_return":          float((1 + valid).prod() - 1),
        "num_trades":            len(trades[trades["direction"]
                                            .str.startswith("entry")]),
        "num_round_trips":       len(trades[trades["direction"] == "exit"]),
        "win_rate":              win_rate,
        "profit_factor":         profit_factor,
        "annualised_volatility": float(valid.std(ddof=1) * np.sqrt(252)),
        "volatility_reduction":  vol_red,
    }
```

E.5.10 Data Client with Incremental Cache (refinitiv_client.py)

```
# Cache key: MD5 hash of (sorted tickers, interval, sorted fields).
# A single CSV file grows incrementally as new date ranges are requested.

def _cache_path(tickers: list, interval: str, fields: list) -> Path:
    key    = {"tickers": sorted(tickers), "interval": interval,
              "fields": sorted(fields)}
    digest = hashlib.md5(
        json.dumps(key, sort_keys=True).encode()
    ).hexdigest()[:10]
    fname  = (
        "_".join(sorted(tickers)).replace(".", "-")[:40]
        + f"__{interval}__{digest}.csv"
    )
    return RAW_DATA_DIR / fname


def get_price_timeseries(
    tickers, start=None, end=None, interval="1d",
    fields=None, use_cache=True,
):
    """
    Fetch OHLCV data from LSEG with smart incremental caching.

    On a full cache hit the function returns immediately.  On a partial
    cache hit (the requested window extends beyond the cached range in
    either direction), only the missing segment(s) are fetched, merged,
    and re-persisted.  This minimises LSEG API calls when re-running
    notebooks with slightly different date windows.
    """
    # ... date / ticker / interval normalisation omitted for brevity ...

    path = _cache_path(instruments, interval, fields)

    if use_cache and path.exists():
        cached_df    = _load_cache(path)
        need_before  = start_dt < cached_df.index.min()
        need_after   = end_dt   > cached_df.index.max()

        if not need_before and not need_after:
            return cached_df.loc[start_dt:end_dt]      # full hit

        fetched_parts = []
        if need_before:
            gap_end = pd.Timestamp(np.busday_offset(
                (cached_df.index.min() - timedelta(days=1)).date(),
                0, roll="preceding"
            ))
            if start_dt <= gap_end:
                fetched_parts.append(
                    _fetch_from_lseg(instruments, fields, interval,
                                     start_dt, gap_end)
                )
        if need_after:
            gap_start = cached_df.index.max() + timedelta(days=1)
            if gap_start <= end_dt:
                fetched_parts.append(
                    _fetch_from_lseg(instruments, fields, interval,
                                     gap_start, end_dt)
                )
        if fetched_parts:
            combined = pd.concat([cached_df] + fetched_parts)
```

```
            combined = combined[~combined.index.duplicated(keep="last")]
            combined.sort_index(inplace=True)
            _save_cache(combined, path)
            return combined.loc[start_dt:end_dt]

        return cached_df.loc[start_dt:end_dt]

    df = _fetch_from_lseg(instruments, fields, interval, start_dt, end_dt)
    if use_cache:
        _save_cache(df, path)
    return df


def _clean_frame(df: pd.DataFrame) -> pd.DataFrame:
    """
    Post-fetch cleaning: datetime index, numeric coercion,
    forward-fill up to 5 business days, and drop columns >10% missing.
    """
    df.index = pd.to_datetime(df.index)
    df.index.name = "Date"
    df.dropna(how="all", inplace=True)
    df.sort_index(inplace=True)
    for col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")
    df = df.ffill(limit=5)
    bad_cols = df.columns[df.isna().mean() > 0.10].tolist()
    if bad_cols:
        warnings.warn(f"Dropping columns with >10% missing: {bad_cols}")
        df.drop(columns=bad_cols, inplace=True)
    return df
```

E.5.11 Universe Configuration (modelling/config.py)

```
# 26 large-cap US equities across 7 sectors, identified by LSEG RIC codes.
TICKERS = [
    "NVDA.O", "AMD.O",  "TSM.N",  "INTC.O",   # Semiconductors
    "KO.N",   "PEP.O",  "COST.O", "TGT.N",    # Consumer Staples
    "JPM.N",  "BAC.N",  "GS.N",   "MS.N",     # Financials
    "XOM.N",  "CVX.N",  "SLB.N",  "HAL.N",    # Energy
    "DAL.N",  "UAL.O",                         # Airlines
    "AMZN.O", "MSFT.O", "META.O", "GOOGL.O",  # Technology / Cloud
    "JNJ.N",  "PFE.N",  "MRK.N",  "ABBV.N",   # Healthcare
]

# 16 sector-homogeneous candidate pairs for cointegration screening.
CANDIDATE_PAIRS = [
    ("NVDA.O", "AMD.O"),  ("NVDA.O", "TSM.N"),
    ("AMD.O",  "INTC.O"), ("TSM.N",  "INTC.O"),
    ("KO.N",   "PEP.O"),  ("COST.O", "TGT.N"),
    ("JPM.N",  "BAC.N"),  ("GS.N",   "MS.N"),
    ("XOM.N",  "CVX.N"),  ("SLB.N",  "HAL.N"),
    ("DAL.N",  "UAL.O"),
    ("AMZN.O", "MSFT.O"), ("META.O", "GOOGL.O"),
    ("AMZN.O", "GOOGL.O"),
    ("JNJ.N",  "PFE.N"),  ("MRK.N",  "ABBV.N"),
]

# In-sample / out-of-sample date split
TRAIN_START = "2018-01-01"
TRAIN_END   = "2023-12-31"   # in-sample (6 years)
TEST_START  = "2024-01-01"   # out-of-sample
TEST_END    = "2025-12-31"

RISK_FREE_RATE = 0.02        # annualised; 3M T-bill average
```